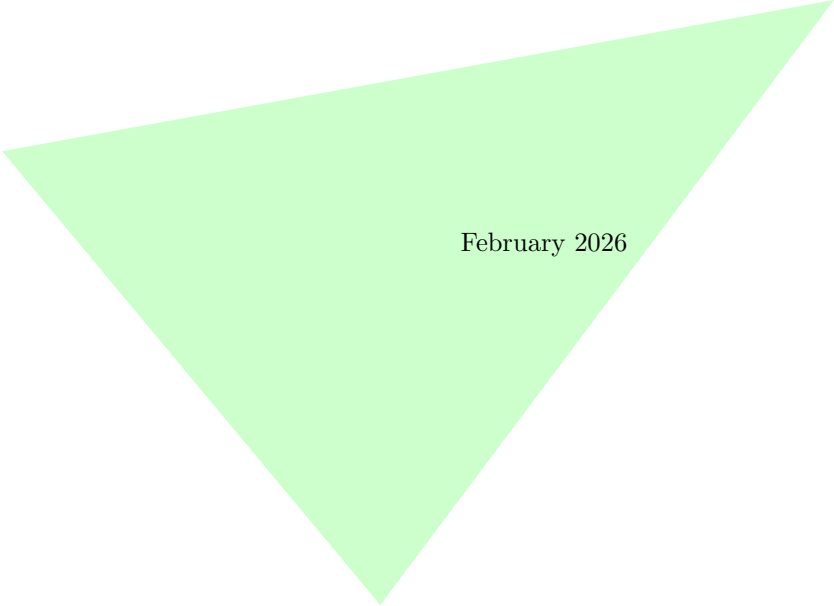
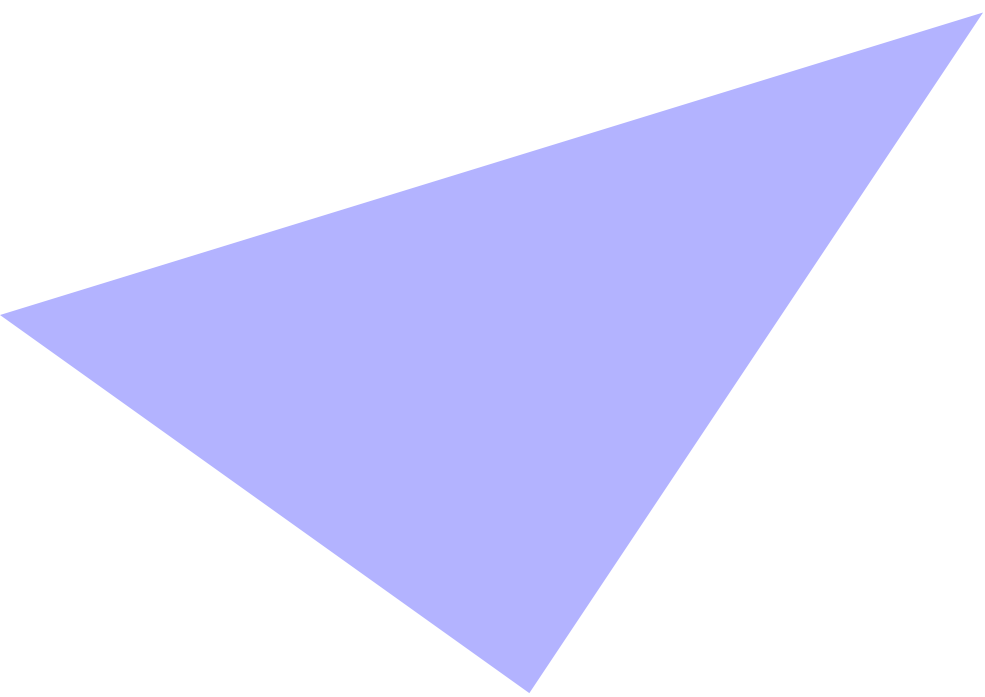




February 2026

C++ Banking Simulation System

An Object-Oriented Banking Application Design Document



Nickolas Black

Financial Systems Programmer



Contents

1	Application Overview	10
1.1	Introduction	10
1.2	Clients	10
1.2.1	BankAccounts.txt	10
1.2.2	Bank Database	11
1.2.3	Bank Statistics	12
1.3	Savings Account	13
1.4	CD Account	13
1.5	Checking Account	14
1.6	Commercial Account	14
1.7	Brokerage Account	14
1.8	Service Transactions	16
1.9	Program Output	16
1.10	Source Files	18
2	Stdafx.hpp	18
2.1	Overview	18
2.2	Standard Libraries	18
2.3	Unit Testing	18
2.4	Exception Headers	19
2.5	Utilities	19
2.6	Implementation	19
3	Testing.hpp	20
3.1	Overview	20
3.2	Standard Libraries	20
3.3	Debug State Management	21
3.4	Logging Macros	21
3.5	Test Execution Macro	21
3.6	Debug Mode Detection	21
3.7	Assertion Macros	21
3.7.1	Release Mode	21
3.7.2	Debug Mode	21
3.8	Internal State	22
3.9	Design Notes	22
4	Exception Classes	22
4.1	Overview	22
4.2	Exception Hierarchy	22
4.3	Financial and Transaction Exceptions	22
4.3.1	SpendingLimitException.hpp	22
4.3.2	InsufficientFundsException.hpp	23
4.3.3	InvalidAmountException.hpp	23
4.4	Account State Exceptions	24
4.4.1	InvalidAccountException.hpp	24
4.4.2	AccountOpenException.hpp	25
4.4.3	AccountClosedException.hpp	25
4.5	Check Processing Exceptions	25
4.5.1	PostDatedCheckException.hpp	25
4.5.2	CheckTooOldException.hpp	26
4.6	Time-Based Financial Exceptions	26
4.6.1	CDMaturityDateException	26
4.6.2	CardExpiredException	27
4.7	Design Notes	27

5	Name.hpp	28
5.1	Overview	28
5.2	Design Rationale	28
5.3	Class Declaration	28
5.4	Constructors	29
5.4.1	Default Constructor	29
5.4.2	Parameterized Constructor	29
5.4.3	Copy Constructor	30
5.5	Member Functions	30
5.5.1	getFirstName()	30
5.5.2	getLastName()	30
5.5.3	printName()	30
5.6	Unit Testing	30
5.6.1	Testing Strategy	31
5.6.2	Test Implementations	31
5.6.3	Test Runner	31
5.7	Implementation	31
5.7.1	Name.hpp	31
5.7.2	Name.cpp	32
5.8	Summary	33
6	Depositor Class	33
6.1	Overview	33
6.2	Design Rationale	33
6.3	Class Declaration	33
6.4	Constructors	34
6.4.1	Default Constructor	34
6.4.2	Parameterized Constructor	34
6.4.3	Copy Constructor	35
6.5	Member Functions	35
6.5.1	getDepositorName()	35
6.5.2	getDepositorSSN()	35
6.5.3	printDepositor()	35
6.6	Unit Testing	35
6.6.1	Testing Strategy	35
6.6.2	Test Implementations	36
6.6.3	Test Runner	36
6.7	Implementation	36
6.7.1	Depositor.hpp	36
6.7.2	Depositor.cpp	37
6.8	Summary	38
7	Calendar Class	38
7.1	Overview	38
7.2	Design Rationale	39
7.3	Class Declaration	39
7.4	Constructors	40
7.4.1	Default Constructor	40
7.4.2	Parameterized Constructor (Full Date)	40
7.4.3	Credit Card Constructor (Month / Year)	40
7.4.4	Copy Constructor	40
7.5	Member Functions	40
7.5.1	getMonth(), getDay(), getYear()	40
7.5.2	before()	40
7.5.3	after()	41
7.5.4	printDate()	41
7.6	Operator Overloading	41
7.6.1	Comparison Operators	41

7.6.2	Arithmetic Operators	41
7.7	Unit Testing	41
7.7.1	Testing Strategy	42
7.7.2	Test Implementations	42
7.7.3	Test Runner	42
7.8	Implementation	42
7.8.1	Calendar.hpp	42
7.8.2	Calendar.cpp	43
7.9	Summary	47
8	TransactionTicket Class	47
8.1	Overview	47
8.2	Design Rationale	48
8.3	TransactionType Enumeration	48
8.4	Class Declaration	48
8.5	Constructors	49
8.5.1	Default Constructor	49
8.5.2	Parameterized Constructor	49
8.5.3	Copy Constructor	50
8.6	Member Functions	50
8.6.1	Getter Series	50
8.6.2	getTransactionType()	51
8.6.3	printTransactionAmount()	51
8.7	Unit Testing	52
8.7.1	Testing Strategy	52
8.7.2	Test Implementation	52
8.7.3	Test Runner	52
8.8	Implementation	52
8.8.1	TransactionTicket.hpp	52
8.8.2	TransactionTicket.cpp	54
8.9	Summary	56
9	TransactionReceipt Class	56
9.1	Overview	56
9.2	Design Rationale	56
9.3	Class Declaration	57
9.4	Constructors	58
9.4.1	Default Constructor	58
9.4.2	Parameterized Constructor (Standard)	58
9.4.3	Parameterized Constructor (CD Version)	58
9.4.4	Copy Constructor	59
9.5	Getter Functions	59
9.6	Formatting Functions	59
9.6.1	printSuccess()	59
9.6.2	printTransactionBalance(double amount)	59
9.6.3	printReceipt()	60
9.6.4	printTransactionReceipt()	60
9.7	Unit Testing	63
9.7.1	Testing Strategy	63
9.7.2	Test Implementation	63
9.7.3	Test Runner	63
9.8	Implementation	63
9.8.1	TransactionReceipt.hpp	63
9.8.2	TransactionReceipt.cpp	65
9.9	Summary	70

10	CreditCard Class	70
10.1	Overview	70
10.2	CardLevel Enumeration	70
10.2.1	Purpose	70
10.3	Class Declaration	71
10.4	Constructors	71
10.4.1	Default Constructor	71
10.4.2	Parameterized Constructor	71
10.5	Getter Functions	72
10.5.1	getCardLimit()	72
10.5.2	getCardLevel()	72
10.6	Formatting Function	72
10.6.1	printCreditChargeAmount()	72
10.7	Design Rationale	73
10.8	Unit Testing	73
10.8.1	Testing Strategy	73
10.8.2	Test Implementation	73
10.8.3	Test Runner	73
10.9	Implementation	74
10.9.1	CreditCard.hpp	74
10.9.2	CreditCard.cpp	74
10.10	Summary	76
11	Check Class	77
11.1	Overview	77
11.2	Design Rationale	77
11.3	Class Declaration	77
11.4	Constructors	78
11.4.1	Default Constructor	78
11.4.2	Parameterized Constructor	78
11.4.3	Copy Constructor	78
11.5	Member Functions	78
11.5.1	getAccountNumber()	79
11.5.2	getCheckAmount()	79
11.5.3	getDateOfCheck()	79
11.6	Unit Testing	79
11.6.1	Testing Strategy	79
11.6.2	Test Implementation	79
11.6.3	Test Runner	79
11.7	Implementation	80
11.7.1	Check.hpp	80
11.7.2	Check.cpp	80
11.8	Summary	81
12	Investment Class	81
12.1	Overview	81
12.2	Design Rationale	81
12.3	Class Declaration	81
12.4	Destructor	82
12.4.1	virtual ~Investment() = default;	82
12.5	Pure Virtual Functions	82
12.5.1	getNumberOfShares()	82
12.5.2	getTotalValue()	82
12.5.3	print()	82
12.6	Shared Utility Function	82
12.6.1	printFormattedBalance(double amount)	82
12.7	Implementation	83
12.7.1	Investment.hpp	83

12.7.2	<code>Investment.cpp</code>	84
12.8	Summary	84
13	Bond Class	84
13.1	Overview	84
13.2	Design Rationale	84
13.3	Class Declaration	85
13.4	Constructors	85
13.4.1	Default Constructor	85
13.4.2	Parameterized Constructor	86
13.4.3	Copy Constructor	86
13.5	Investment Interface Implementation	86
13.5.1	<code>getNumberOfShares()</code>	86
13.5.2	<code>getTotalValue()</code>	86
13.5.3	<code>getAnnualYield()</code>	87
13.5.4	<code>getMaturityDate()</code>	87
13.6	Formatted Output	87
13.6.1	<code>print()</code>	87
13.7	Unit Testing	87
13.7.1	Testing Strategy	87
13.7.2	Test Implementation	88
13.7.3	Test Runner	88
13.8	Implementation	88
13.8.1	<code>Bond.hpp</code>	88
13.8.2	<code>Bond.cpp</code>	89
13.9	Summary	90
14	Option Class	91
14.1	Overview	91
14.2	Design Rationale	91
14.3	Class Declaration	91
14.4	Constructors	92
14.4.1	Default Constructor	92
14.4.2	Parameterized Constructor	92
14.4.3	Copy Constructor	92
14.5	Core Investment Functions	93
14.5.1	<code>getNumberOfShares()</code>	93
14.5.2	<code>getTotalValue()</code>	93
14.5.3	<code>getOptionType()</code>	93
14.6	Formatting Function	93
14.6.1	<code>print()</code>	93
14.7	Unit Testing	94
14.7.1	Testing Strategy	94
14.7.2	Test Implementation	94
14.7.3	Test Runner	94
14.8	Implementation	94
14.8.1	<code>Option.hpp</code>	94
14.8.2	<code>Option.cpp</code>	95
14.9	Summary	97
15	Stock Class	97
15.1	Overview	97
15.2	Design Rationale	98
15.3	Class Declaration	98
15.4	Constructors	99
15.4.1	Default Constructor	99
15.4.2	Parameterized Constructor	99
15.4.3	Copy Constructor	99
15.5	Core Investment Functions	99

15.5.1	<code>getNumberOfShares()</code>	99
15.5.2	<code>getTotalValue()</code>	99
15.5.3	<code>getTickerSymbol()</code>	100
15.6	Formatting Function	100
15.6.1	<code>print()</code>	100
15.7	Unit Testing	100
15.7.1	Testing Strategy	100
15.7.2	Test Implementation	101
15.7.3	Test Runner	101
15.8	Implementation	101
15.8.1	<code>Stock.hpp</code>	101
15.8.2	<code>Stock.cpp</code>	102
15.9	Summary	103
16	CreditAccount Class	103
16.1	Overview	103
16.2	Design Rationale	104
16.3	Class Declaration	104
16.4	Constructors	105
16.4.1	Default Constructor	105
16.4.2	Parameterized Constructor	105
16.4.3	BankAccount Conversion Constructor	105
16.4.4	Copy Constructor	106
16.5	Getter Functions	106
16.6	Formatting Functions	106
16.6.1	<code>printAccount()</code>	106
16.6.2	<code>printAccountBalance()</code>	106
16.6.3	<code>printAccountStatus()</code>	106
16.7	Transaction Operations	107
16.7.1	<code>getBalanceDue()</code>	107
16.7.2	<code>payBalanceDue()</code>	107
16.7.3	<code>creditTransaction()</code>	107
16.7.4	<code>reOpenAccount()</code>	107
16.7.5	<code>closeAccount()</code>	107
16.8	Transaction Logging	107
16.9	Unit Testing	108
16.9.1	Testing Strategy	108
16.9.2	Test Runner	108
16.10	Implementation	108
16.10.1	<code>CreditAccount.hpp</code>	108
16.10.2	<code>CreditAccount.cpp</code>	109
16.11	Summary	111
17	BankAccount Class	111
17.1	Overview	111
17.2	Design Rationale	112
17.3	AccountType Enumeration	112
17.4	Class Declaration	112
17.5	Constructors	113
17.5.1	Default Constructor	113
17.5.2	Parameterized Constructor	113
17.5.3	Copy Constructor	114
17.6	Getter Functions	114
17.7	Transaction Functions	114
17.7.1	<code>getBalance()</code>	114
17.7.2	<code>makeDeposit()</code>	115
17.7.3	<code>makeWithdrawl()</code>	115
17.7.4	<code>closeAccount()</code>	116

17.7.5	<code>reOpenAccount()</code>	117
17.8	Formatting Functions	117
17.8.1	<code>printAccount()</code>	117
17.8.2	<code>printAccountBalance()</code>	117
17.8.3	<code>printAccountStatus()</code>	118
17.8.4	<code>printTransactionHistory()</code>	118
17.9	Unit Testing	118
17.9.1	Testing Strategy	118
17.9.2	Test Runner	118
17.10	Implementation	118
17.10.1	<code>BankAccount.hpp</code>	118
17.10.2	<code>BankAccount.cpp</code>	120
17.11	Summary	129
18	SavingsAccount Class	129
18.1	Overview	129
18.2	Design Rationale	129
18.3	Class Declaration	130
18.4	Constructors	130
18.4.1	Default Constructor	130
18.4.2	Parameterized Constructor	130
18.4.3	Copy Constructor	131
18.5	Inherited Functionality	131
18.6	Unit Testing	131
18.6.1	Testing Strategy	131
18.6.2	Test Implementation	132
18.6.3	Test Runner	132
18.7	Implementation	132
18.7.1	<code>SavingsAccount.hpp</code>	132
18.7.2	<code>SavingsAccount.cpp</code>	132
18.8	Summary	134
19	CheckingAccount Class	134
19.1	Overview	134
19.2	Design Rationale	134
19.3	Class Declaration	135
19.4	Constructors	135
19.4.1	Default Constructor	135
19.4.2	Parameterized Constructor	136
19.4.3	Copy Constructor	136
19.5	Core Checking Function	136
19.5.1	<code>clearCheck()</code>	136
19.6	Unit Testing	137
19.6.1	Testing Strategy	137
19.6.2	Test Implementation	138
19.6.3	Test Runner	138
19.7	Implementation	138
19.7.1	<code>CheckingAccount.hpp</code>	138
19.7.2	<code>CheckingAccount.cpp</code>	139
19.8	Summary	142
20	CDAccount Class	142
20.1	Overview	142
20.2	Design Rationale	142
20.3	Class Declaration	143
20.4	Constructors	143
20.4.1	Default Constructor	143
20.4.2	Parameterized Constructor (CD Term)	144
20.4.3	Parameterized Constructor (Maturity Date)	144

20.4.4	Copy Constructor	144
20.5	Core CD Functions	145
20.5.1	getMaturityDate()	145
20.5.2	makeWithdrawl()	145
20.5.3	makeDeposit()	145
20.6	Formatting Functions	146
20.6.1	printAccount()	146
20.6.2	printAccountReceipt()	146
20.7	Unit Testing	146
20.7.1	Testing Strategy	146
20.7.2	Test Implementation	147
20.7.3	Test Runner	147
20.8	Implementation	147
20.8.1	CDAccount.hpp	147
20.8.2	CDAccount.cpp	148
20.9	Summary	151
21	CommercialAccount Class	152
21.1	Overview	152
21.2	Design Rationale	152
21.3	Class Declaration	152
21.4	Constructors	153
21.4.1	Default Constructor	153
21.4.2	Parameterized Constructor	153
21.4.3	Copy Constructor	153
21.5	Core Commercial Functions	153
21.5.1	getCreditAccount()	153
21.5.2	addTransactionReceipt()	153
21.6	Unit Testing	154
21.6.1	Testing Strategy	154
21.6.2	Test Implementation	154
21.6.3	Test Runner	154
21.7	Implementation	154
21.7.1	CommercialAccount.hpp	154
21.7.2	CommercialAccount.cpp	155
21.8	Summary	157
22	BrokerageAccount Class	157
22.1	Overview	157
22.2	Design Rationale	157
22.3	Class Declaration	157
22.4	Constructors	158
22.4.1	Default Constructor	158
22.4.2	Parameterized Constructor	158
22.4.3	Deleted Copy Constructor	158
22.5	Core Brokerage Functions	158
22.5.1	addInvestment()	158
22.5.2	getTotalValue()	159
22.5.3	getTotalShares()	159
22.5.4	printPortfolio()	159
22.5.5	printTotalValue()	160
22.6	Unit Testing	160
22.6.1	Testing Strategy	160
22.6.2	Test Implementation	160
22.6.3	Test Runner	161
22.7	Implementation	161
22.7.1	BrokerageAccount.hpp	161
22.7.2	BrokerageAccount.cpp	162

22.8 Summary	165
23 RandomGenerator Class	165
23.1 Overview	165
23.2 Design Rationale	166
23.3 Class Declaration	166
23.4 Random Number Generators	167
23.5 Account Generation	167
23.6 Investment Generation	167
23.7 Transaction Generation	167
23.8 Unit Testing	167
23.8.1 Random Values	167
23.8.2 Random Account Generation	167
23.8.3 Random Portfolio Generation	168
23.8.4 Test Runner	168
23.9 Random Generator Implementation	168
23.9.1 RandomGenerator.hpp	168
23.9.2 RandomGenerator.hpp	169
23.10 Summary	176
24 Bank Class	176
24.1 Overview	176
24.2 Member Variables	177
24.3 Constructors	178
24.4 Public Member Functions	178
24.5 Private Member Functions	178
24.6 Transaction Handling and Exceptions	178
24.7 HTML Output	179
24.8 Bank Implementation	179
24.8.1 Bank.hpp	179
24.8.2 Bank.cpp	180
25 Main.cpp	194
25.1 Overview	194
25.2 Libraries Used	194
25.3 Main Function	194
25.4 Execution Flow	195
25.5 Notes	195
25.6 Main Implementation	195

1 Application Overview

1.1 Introduction

C++ was adopted by all the major banking institutions not long after it's introduction in 1989. Since then several modern low level programming languages have been introduced, like we have seen Java and Golang. These new languages offer native improvements to the C++ language. However, new iterations of C++ are periodically introduced as we have seen with C++11-23. Each version includes a new modern technique or commands that allow a competent developer to execute critical tasks faster and avoid expected errors and pitfalls.

Banking institutions largely have not switched to a newer modern programming languages because of profitability. Entire financial platforms, networks, enterprise systems were constructed using C++. The expense of replacing these systems, in conjunction with a potential future drop in profitability caused by a degradation in application performance that may persist for an indefinite period, makes it economically infeasible for companies to make the switch. Companies of this era save money by sticking to C++.

Secondly, there is a direct correlation between the speed that an application completes a transaction and profitability. More transactions over a finite period of time equal more overall profitability. For example, more transactions in a day means more transactions in a month, and, by extension, more transactions in the fiscal year. C++ provides the additional advantage of compiler optimization techniques that moves some computation from runtime to compile time.

This application is meant to be a direct consequence of that truth. First, we have an idea about why we should learn C++ in general. One of the purposes of this application is to provide a high level explanation to someone who is completely unfamiliar with the language, computational finance, high performance computing, or modeling and simulation, and simulate a critical system that they use in their daily lives: a banking application.

Secondly, we not only understand that the performance of our application is essential to our firm's survival, but design choices that we make from the onset are set in stone and could impact our systems in the future, and inevitably, our quality of life. Another purpose of this application is to demonstrate standard object oriented programming paradigms, such as object oriented programming, inheritance, polymorphism, exception handling, and unit testing; as well as taking advantage of specific features that are unique to the C++ programming language, such as operator overloading and static linkage.

This leads to my first programmatic principle:

The best solution to any error is to prevent the error from arising entirely.

This is the single most important sentence in this entire document. This program contains over 3000 lines of code, and If we don't comprehend this concept at the beginning we could end up with an application that does nothing at all or is impossible to optimize in the end. I will refer to this as the first paradigm from this point on.

Now to talk about the features of this application.

1.2 Clients

The program starts by reading the first name, last name, and social security number of the banks clients from a text file. All other data in this application is generated at random. There are no real client or real accounts, however this is the data that we must assure remains the same and we will use some of this information to navigate through the bank account database.

1.2.1 BankAccounts.txt

As stated before the names and social security number of the clients are stored in a text file called BankAccounts.txt. These are the names and social security numbers I used for this documented iteration of the program:

(BankAccounts.txt)

Lewis Hamilton	121212121
Lewis Hamilton	121212121
Lewis Hamilton	121212121
Lewis Hamilton	121212121
Sebastian Priaulx	131313131

Sebastian Priaulx	131313131
Mathias Lauda	454545454
Mathias Lauda	454545454
Mathias Lauda	454545454
Sebastian Buemi	012012012
Sebastian Buemi	012012012
Sebastian Buemi	012012012
Max Verstappen	991777991
Max Verstappen	991777991
Nicolas Lapierre	543543543
Nicolas Lapierre	543543543
Nicolas Lapierre	543543543
Nicolas Lapierre	543543543
Michael Auriemma	789789789
Riley Dickinson	200120010
Riley Dickinson	200120010
Riley Dickinson	200120010
Ashley Freiberg	987654321
Ashley Freiberg	987654321
Madeline Stewart	151515151
Alexandra Veteva	787878787
Alexandra Vetava	787878787
Michael Schumacher	123456789
Michael Schumacher	123456789
Michael Schumacher	123456789
Fernando Alonso	977977977
Fernando Alonso	977977977
Kevin Estre	911911911
Kevin Estre	911911911
Kevin Estre	911911911
Kevin Estre	911911911
Sebastian Vettel	123789123
Sebastian Vettel	123789123
Sebastian Vettel	123789123
Niki Lauda	460460460
Niki Lauda	460460460
Niki Lauda	460460460

Key thing to note here is that all our clients are famous race car drivers with clearly bogus social security numbers (please don't sue me), and at least two of the banks client's has the same first name and the same last name.

1.2.2 Bank Database

Once the client names are read from the file, the program generates a random account data for each client name. The client is given a random account number. Then randomly assigns either a savings account, checking account, certificate of deposit (CD) account, commercial account, or brokerage account. Each account has a random account balance, and for CD accounts, a random maturity date. All accounts are initialized as open. The accounts are read into an account database and printed to a webpage after all the accounts are created. The output should look like this:

Name	SSN	Account#	Type	Status	Balance	Maturity Date

Lewis Hamilton	121212121	110001	Checking	Open	\$78,777.97	
Lewis Hamilton	121212121	210002	Savings	Open	\$13,110.29	
Lewis Hamilton	121212121	510003	Brokerage	Open	\$17,608.49	
Lewis Hamilton	121212121	310004	CD	Open	\$88,880.64	10/1/2027
Sebastian Priaulx	131313131	410005	Commercial	Open	\$82,205.21	
Sebastian Priaulx	131313131	110006	Checking	Open	\$71,237.98	

Mathias Lauda	454545454	110007	Checking	Open	\$47,386.13	
Mathias Lauda	454545454	510008	Brokerage	Open	\$69,433.58	
Mathias Lauda	454545454	310009	CD	Open	\$21,999.46	8/12/2020
Sebastian Buemi	012012012	510010	Brokerage	Open	\$59,025.28	
Sebastian Buemi	012012012	510011	Brokerage	Open	\$91,570.24	
Sebastian Buemi	012012012	310012	CD	Open	\$30,473.46	11/22/2029
Max Verstappen	991777991	210013	Savings	Open	\$83,003.46	
Max Verstappen	991777991	410014	Commercial	Open	\$19,976.88	
Nicolas Lapierre	543543543	110015	Checking	Open	\$56,493.78	
Nicolas Lapierre	543543543	410016	Commercial	Open	\$97,974.12	
Nicolas Lapierre	543543543	510017	Brokerage	Open	\$54,275.74	
Nicolas Lapierre	543543543	410018	Commercial	Open	\$27,727.43	
Michael Auremma	789789789	210019	Savings	Open	\$64,995.26	
Riley Dickinson	200120010	410020	Commercial	Open	\$47,847.43	
Riley Dickinson	200120010	310021	CD	Open	\$30,517.47	1/4/2019
Riley Dickinson	200120010	410022	Commercial	Open	\$89,846.38	
Ashley Freiberg	987654321	410023	Commercial	Open	\$66,378.46	
Ashley Freiberg	987654321	510024	Brokerage	Open	\$46,205.32	
Madeline Stewart	151515151	410025	Commercial	Open	\$38,959.80	
Alexandra Veteva	787878787	510026	Brokerage	Open	\$28,068.20	
Alexandra Vetava	787878787	310027	CD	Open	\$35,281.41	8/25/2028
Michael Schumacher	123456789	110028	Checking	Open	\$51,377.14	
Michael Schumacher	123456789	310029	CD	Open	\$98,560.51	3/25/2020
Michael Schumacher	123456789	510030	Brokerage	Open	\$28,356.31	
Fernando Alonso	977977977	410031	Commercial	Open	\$57,980.90	
Fernando Alonso	977977977	510032	Brokerage	Open	\$43,723.99	
Kevin Estre	911911911	110033	Checking	Open	\$79,341.10	
Kevin Estre	911911911	510034	Brokerage	Open	\$23,383.10	
Kevin Estre	911911911	210035	Savings	Open	\$44,667.90	
Kevin Estre	911911911	410036	Commercial	Open	\$296.22	
Sebastian Vettel	123789123	310037	CD	Open	\$28,748.93	8/25/2028
Sebastian Vettel	123789123	310038	CD	Open	\$70,114.35	3/26/2027
Sebastian Vettel	123789123	510039	Brokerage	Open	\$5,319.12	
Niki Lauda	460460460	410040	Commercial	Open	\$72,645.06	
Niki Lauda	460460460	510041	Brokerage	Open	\$93,285.55	
Niki Lauda	460460460	410042	Commercial	Open	\$98,588.83	

1.2.3 Bank Statistics

As the accounts are read into the banks database the account information is aggregated by its account type. The total amount of assets under management and number of accounts by account type are tracked by static variables. This allows the bank to view it's total position at any point in time.

Total Number of Checking Accounts: 6
 Total Number of Savings Accounts: 4
 Total Number of CD Accounts: 8
 Total Number of Commercial Accounts: 12
 Total Number of Brokerage Accounts: 12
 Total Number of Credit Accounts: 12

Checking Account Total Balance: \$384,614.10
 Savings Account Total Balance: \$205,776.91
 CD Account Total Balance: \$404,576.24
 Commercial Account Total Balance: \$700,426.72
 Brokerage Account Total Balance: \$560,254.92
 Credit Account Total Balance: \$0.00

Total Number of Accounts: 42

Total Assests Under Management: \$2,255,648.89

A key thing to not here is the total balance for credit accounts starts a zero. This is because no is no lending at the start of the application. Next we will dive into the details of each type of account and see the that we have different rules for each transaction and account type.

1.3 Savings Account

If a client is assigned a savings account there are no restrictions on when they can make deposits or withdrawals. The client can perform other typical banking transactions such as open and closing an account, and getting their account balance. Here are the account receipts that a savings account returns.

Transaction Requested: Balance

Client Name: Max Verstappen

Account Number: 210013

Account Type: Savings

Current Balance: \$83,003.46

Transaction Requested: Close Account

Client Name: Lewis Hamilton

Account Number: 210002

Account Type: Savings

Account Closed

Transaction Requested: Re-Open Account

Client Name: Lewis Hamilton

Account Number: 210002

Account Type: Savings

Account Open

Transaction Requested: Withdrawal

Client Name: Max Verstappen

Account Number: 210013

Account Type: Savings

Account Balance: \$83,003.46

| Amount to Withdraw: \$24,310.74

New Balance: \$58,692.72

Transaction Requested: Deposit

Client Name: Kevin Estre

Account Number: 210035

Account Type: Savings

Account Balance: \$44,667.90

| Amount to Deposit: \$4,307.44

New Balance: \$48,975.34

These are the basic bank account transactions. All other bank accounts have the same functionality and could return a similar receipt. The depositor performs a full withdrawal whenever their account is closed. We will explore that more later in the `BankAccount` class. Next, we will examine the CD accounts and their features.

1.4 CD Account

A certificate of deposit account, or CD account, is a low risk interest earning savings account that locks in your money for a specific term at a fixed rate. In this program a client with a CD account can perform the same transactions as with a savings account, however, there are restrictions on when they can perform deposits and withdrawals. Once an initial deposit is made the client can not make another deposit, and withdrawals are only permitted after the maturity date has be reached. These are some of the possible receipts generated by a CD account:

Transaction Requested: Deposit

Client Name: Michael Schumacher

Account Number: 310029

Account Type: CD

Maturity Date: 2/17/2043

Account Balance: \$98,560.51

| Amount to Deposit: \$12,264.11

New Balance: \$110,824.62

Transaction Requested: Withdrawal

Client Name: Lewis Hamilton

Account Number: 310004

Account Type: CD

Maturity Date: 10/1/2027

Current Balance: \$88,880.64

Transaction Amount: \$558.34

Maturity Date: 10/1/2027 not reached yet.

Key things to note here are that these examples demonstrate a deposit on an account with a maturity date that already passed and a new maturity date was assigned. It also demonstrates that an incorrect withdrawal on a CD account whose maturity date has not been reached yet. Also notice that the maturity date is a printed on the receipt.

I will go into more detail about the different user defined exception classes in a later section. For now understand that a CD account is a savings account with rules on deposits and withdrawals. Next, we examine the checking accounts.

1.5 Checking Account

When a client is assigned a checking account they can perform the same transactions as a savings account without restrictions on deposits and withdrawals. Clients with a checking account have the added benefit of performing checking transactions. Essentially, the account performs a typical withdrawal using a check. The account charges a penalty for a transaction failure from insufficient funds.

Here is a possible checking account transaction receipt:

Transaction Requested: Clear Check

Client Name: Nicolas Lapierre

Account Number: 110015

Account Type: Checking

Account Balance: \$56,493.78

| Amount of Check: \$24,709.95

New Balance: \$31,783.83

There are a few circumstance where checks will not be cleared, such as a post-dated check or a check that is older than six months. As mentioned before, I'll go into greater detail about the user defined checking account exception classes in a later section.

The key takeaway here is that checking accounts clear checks. Now I'll move on to commercial accounts.

1.6 Commercial Account

A commercial account in this context is a combination of a credit account and a checking account. The client can perform credit transactions using a credit card, along with all the transactions of a checking account, including check transactions. Credit transactions result in a balance that is due to the bank. If the client has reached their credit limit, then the transaction will be denied. They also has the ability to pay down their balance.

Here is a possible commercial transaction receipt:

Transaction Requested: Credit Transaction

Client Name: Max Verstappen

Account Number: 410014

Account Type: Commercial

Credit Amount: \$928.62

Balance Due: \$928.62

Now I'll walk through the final account type, Brokerage account.

1.7 Brokerage Account

A brokerage account is a commercial account with an investment portfolio. The program generates an investment portfolio for the client once they are assigned a brokerage account. A portfolio can consist of exactly three types of investments: stocks, bonds, and options. They can view the total value of their portfolio and the total number of shares for each investment.

A possible brokerage account transaction receipt could look like this:

Transaction Requested: Portfolio

Client Name: Alexandra Veteva Account Number: 510026

Account Type: Brokerage

Client Name	SSN	Acct Num	Acct Type	Status	Balance	Balance Due
Alexandra Veteva	787878787	510026	Brokerage	Open	\$28,068.20	\$0.00

***** Investment Portfolio *****

Ticker: #G00G

Shares: 142

Price: \$467.75 Total Value: \$66,421.09

Option: Call

Contracts: 3

Premium: \$9.09 Total Value: \$2,726.97

Option: Put

Contracts: 3

Premium: \$10.55 Total Value: \$3,165.11

Option: Call

Contracts: 12

Premium: \$9.71 Total Value: \$11,654.84

Bond

Quantity: 35

Face Value: \$1,000.00

Coupon: 3.68236%

Maturity Date: 2/17/2042 Total Value: \$35,000.00

Ticker: #BA

Shares: 36

Price: \$233.97 Total Value: \$8,423.00

Bond

Quantity: 45

Face Value: \$1,000.00

Coupon: 8.09214%

Maturity Date: 2/17/2049 Total Value: \$45,000.00

Ticker: #MSFT

Shares: 333

Price: \$114.00 Total Value: \$37,963.28

Option: Put

Contracts: 13

Premium: \$2.77 Total Value: \$3,598.92

Option: Put

Contracts: 4

Premium: \$14.56 Total Value: \$5,825.25

Total Value: \$219,778.46

Total Number of Shares: 4091

Key takeaway here a brokerage account could perform checking transactions, credit transactions, and view their current portfolio position.

1.8 Service Transactions

The bank itself could also provide special service transactions on the account in the database. Sometimes they may want to add a new account or remove an account. Those transactions aren't necessary for this applications. However, clients can view their transaction history and check all their accounts. In this application clients can see all the accounts they have (why some clients have more than one account), and they can view all of their transaction receipts. A service transaction might look like:

Transaction Requested: Client Information

Client Name: Sebastian Buemi Account Number: 310012

Account Type: CD Maturity Date: 11/22/2029

Name	SSN	Acct Num	Type	Status	Balance	Maturity Date

Sebastian Buemi	012012012	510010	Brokerage	Open	\$59,025.28	
Sebastian Buemi	012012012	510011	Brokerage	Open	\$91,570.24	
Sebastian Buemi	012012012	310012	CD	Open	\$30,473.46	11/22/2029

Number of Accounts: 3						

Transaction Requested: Transaction History

Client Name: Lewis Hamilton Account Number: 210002

Account Type: Savings

Client Name	SSN	Acct Num	Acct Type	Status	Balance
Lewis Hamilton	121212121	210002	Savings	Open	\$0.00
***** Transaction History *****					
Date	Transaction	Amount	Status	Balance	Reason for Failure
2/17/2026	Close Account	\$0.00	Done	\$0.00	
2/17/2026	Balance	\$0.00	Done	\$0.00	
2/17/2026	Re-Open Account	\$0.00	Done	\$0.00	
2/17/2026	Balance	\$0.00	Done	\$0.00	

Generally, logging, and logging efficiently, is an essential feature of any enterprise application. For this application the program output, which I will discuss in the next subsection, serves as the log. This concludes all of the applications transactions.

1.9 Program Output

The program out put starts with the bank database as stated before. Next it writes the the receipts of all the user specified number of random transactions to the same webpage. Finally, it writes the new position of the bank at the end end of the webpage like this:

Name	SSN	Account#	Type	Status	Balance	Maturity Date

Lewis Hamilton	121212121	110001	Checking	Open	\$36,722.19	
Lewis Hamilton	121212121	210002	Savings	Open	\$38,969.10	
Lewis Hamilton	121212121	510003	Brokerage	Closed	\$0.00	
Lewis Hamilton	121212121	310004	CD	Open	\$0.00	10/1/2027
Sebastian Priaulx	131313131	410005	Commercial	Open	\$82,921.64	
Sebastian Priaulx	131313131	110006	Checking	Closed	\$-2.50	
Mathias Lauda	454545454	110007	Checking	Open	\$7,852.81	
Mathias Lauda	454545454	510008	Brokerage	Open	\$86,949.26	
Mathias Lauda	454545454	310009	CD	Closed	\$0.00	2/17/2050
Sebastian Buemi	012012012	510010	Brokerage	Open	\$65,440.39	
Sebastian Buemi	012012012	510011	Brokerage	Open	\$71,227.46	
Sebastian Buemi	012012012	310012	CD	Closed	\$0.00	11/22/2029
Max Verstappen	991777991	210013	Savings	Open	\$19,041.41	
Max Verstappen	991777991	410014	Commercial	Closed	\$0.00	

Nicolas Lapierre	543543543	110015	Checking	Open	\$3,539.75	
Nicolas Lapierre	543543543	410016	Commercial	Closed	\$5,167.99	
Nicolas Lapierre	543543543	510017	Brokerage	Closed	\$0.00	
Nicolas Lapierre	543543543	410018	Commercial	Open	\$38,232.37	
Michael Auriemma	789789789	210019	Savings	Open	\$104,259.62	
Riley Dickinson	200120010	410020	Commercial	Open	\$47,847.43	
Riley Dickinson	200120010	310021	CD	Open	\$0.00	2/17/2040
Riley Dickinson	200120010	410022	Commercial	Open	\$0.00	
Ashley Freiberg	987654321	410023	Commercial	Open	\$66,378.46	
Ashley Freiberg	987654321	510024	Brokerage	Closed	\$0.00	
Madeline Stewart	151515151	410025	Commercial	Open	\$0.00	
Alexandra Veteve	787878787	510026	Brokerage	Closed	\$13,962.73	
Alexandra Vetava	787878787	310027	CD	Closed	\$0.00	8/25/2028
Michael Schumacher	123456789	110028	Checking	Open	\$37,047.86	
Michael Schumacher	123456789	310029	CD	Open	\$0.00	2/17/2043
Michael Schumacher	123456789	510030	Brokerage	Closed	\$1,912.62	
Fernando Alonso	977977977	410031	Commercial	Open	\$57,980.90	
Fernando Alonso	977977977	510032	Brokerage	Open	\$67,738.90	
Kevin Estre	911911911	110033	Checking	Open	\$32,095.06	
Kevin Estre	911911911	510034	Brokerage	Open	\$35,974.80	
Kevin Estre	911911911	210035	Savings	Closed	\$0.00	
Kevin Estre	911911911	410036	Commercial	Closed	\$0.00	
Sebastian Vettel	123789123	310037	CD	Open	\$0.00	8/25/2028
Sebastian Vettel	123789123	310038	CD	Closed	\$0.00	3/26/2027
Sebastian Vettel	123789123	510039	Brokerage	Open	\$5,319.12	
Niki Lauda	460460460	410040	Commercial	Open	\$72,645.06	
Niki Lauda	460460460	510041	Brokerage	Open	\$93,285.55	
Niki Lauda	460460460	410042	Commercial	Open	\$22,546.29	

Total Number of Checking Accounts: 6
 Total Number of Savings Accounts: 4
 Total Number of CD Accounts: 8
 Total Number of Commercial Accounts: 12
 Total Number of Brokerage Accounts: 12
 Total Number of Credit Accounts: 12

Checking Account Total Balance: \$215,534.68
 Savings Account Total Balance: \$283,048.48
 CD Account Total Balance: \$426,909.08
 Commercial Account Total Balance: \$732,495.24
 Brokerage Account Total Balance: \$621,720.30
 Credit Account Total Balance: \$33,591.85

Total Number of Accounts: 42
 Total Assests Under Management: \$2,246,115.94

A very important thing to note here is that the program output is written to a webpage. This program is meant to demonstrate modeling and simulation using C++, so I must tell you that you should expect to perform tens of thousands of computations. A Monte Carlo simulation is a textbook example of computational modeling in practice.

We will see later how we can specify the number of random transactions generated. There is a cap at around 1000 transactions, because anything above that will crash the program due to a lack of random access memory (RAM) to render the webpage. Generally, web pages don't serve as logs. It's here for people who are reading this document and outside observers to have a simplified way of viewing the programs output, and to demonstrate how C++ ties in with HTML.

1.10 Source Files

For this application I used this build command

```
clang++ -std=c++20 *.cpp -o program
```

All the files can be found here:

https://github.com/NickolasBlack/Banking_Application_Output

2 Stdafx.hpp

2.1 Overview

This part is little more advanced, but the purpose is to reduce repetitive includes, enforce consistency across modules, and simplify dependency management. It serves as the central location of all the standard libraries, and using this we can give our files access to the libraries in one central location.

Generally, we only want to include the libraries that we need in the file (and we will), however as we add more and more files to the package, dependency management becomes unwieldy. While this is certainly computationally suboptimal, it also serves as an example of a passive debugging technique. Certain linker errors could arise due to dependency inconsistencies that are not only hard to resolve, but the process of debugging them could introduce new linker errors and add more time until we get a working program. In the event the event that a linker error does arise, we know exactly where the bug could be. Remember the first paradigm.

2.2 Standard Libraries

The following C++ standard library headers are included to support core functionality used throughout the system:

- `<iostream>` — Console input and output streams
- `<cmath>` — Mathematical functions and constants
- `<string>` — String manipulation and storage
- `<vector>` — Dynamic array container
- `<chrono>` — Time points, durations, and clocks
- `<sstream>` — String-based stream processing
- `<iomanip>` — Output formatting utilities
- `<memory>` — Smart pointers and memory management
- `<fstream>` — File input and output streams

2.3 Unit Testing

This file also contains the unit testing framework in `Testing.hpp`, which we will explore in greater detail in the next section. Including it here allows us to include the unit testing macro commands in each file. We can develop test classes for the classes and all but guarantee the accuracy and functionality of the program.

- `Testing.hpp` — Defines assertion utilities and test helpers

We will also see how it also serves as a useful debugging tool and help us triangulate exactly where the logic error is located.

2.4 Exception Headers

By design not every transaction that is generated is intended to be valid. In some circumstances they may be instances where we want to gracefully handle the error or even prevent an invalid transaction from occurring. For example, when we go to the bank and withdraw money from our accounts the ATM does not let us withdraw more money than our account balance. We have exceptions for these unique circumstances that we might encounter. These are the exception header files:

- AccountClosedException.hpp
- CDMaturityDateException.hpp
- CheckTooOldException.hpp
- InsufficientFundsException.hpp
- InvalidAccountException.hpp
- InvalidAmountException.hpp
- PostDatedCheckException.hpp
- AccountOpenException.hpp
- CardExpiredException.hpp
- SpendingLimitException.hpp

We will explore the exception classes in greater detail in Section 4.

2.5 Utilities

C++20 does have a date class in the `<chrono>` library. However, I'm using a Mac computer and the date class object isn't available on ARM64 chips. So we did the next best thing and developed our own date class called `Calendar`, which is preferred. I'll go into more detail about the `Calendar` class and its functionality in the `Calendar.hpp` section.

- `Calendar.hpp` — Date and time utilities used for transaction validation, maturity checks, and expiration logic

For now, the key takeaway is the reduced overhead of including date class methods and all our class have access to calendar objects.

2.6 Implementation

Some additional notes about this file

- This header is protected by `#pragma once` to prevent multiple inclusion.
- No implementation logic should be added to this file.
- Changes to this file should be made cautiously, as it affects a wide range of compilation units.

Here are the declarations:

```
#pragma once

// Standard Library
#include <iostream>
#include <cmath>
#include <string>
#include <vector>
#include <chrono>
#include <sstream>
#include <iomanip>
```

```
#include <memory>
#include <fstream>

// Unit Testing
#include "Testing.hpp"

// Exceptions
#include "AccountClosedException.hpp"
#include "CDMaturityDateException.hpp"
#include "CheckTooOldException.hpp"
#include "InsufficientFundsException.hpp"
#include "InvalidAccountException.hpp"
#include "InvalidAmountException.hpp"
#include "PostDatedCheckException.hpp"
#include "AccountOpenException.hpp"
#include "CardExpiredException.hpp"
#include "SpendingLimitException.hpp"

// Utilities
#include "Calendar.hpp"
```

This concludes the section on Stdafx.hpp.

3 Testing.hpp

3.1 Overview

This is the part where we introduce the second programmatic paradigm:

They have the acceptance criteria. You have test-driven software.

Unit testing is one phase in the Application Development Life Cycle (ADLC), but wouldn't it be helpful if we had a program that did it for us? That's the purpose of this header file. It defines a lightweight debugging, assertion, and unit testing framework used throughout the application. It allows us to configurable debug logging, assertion macros, and a standardized mechanism for executing test functions with clear pass/fail reporting.

We could make our own unit testing framework or include the `<assert>` library, with the former being preferred. I took the unit testing framework from C++ for Financial Mathematics by John Armstrong. He provides a reusable unit testing framework and in this section we will examine it in high level detail.

3.2 Standard Libraries

Remember when we were talking about centralizing the standard libraries? Well, that file also includes the unit testing framework, so we don't want to include Stdafx.hpp here. Now, also recall the linker errors that we mentioned.

The following standard library headers are included to support exception handling, logging, and diagnostics:

- `<stdexcept>` — Runtime exception types
- `<sstream>` — String stream construction for error messages
- `<iostream>` — Diagnostic output streams
- `<cstdlib>` — Process termination utilities
- `<cassert>` — Standard assertion support

3.3 Debug State Management

Debug output is controlled via a global debug flag and accessor functions:

- `bool isEnabled()` — Returns whether debug output is enabled
- `void setDebugEnabled(bool enabled)` — Enables or disables debug output

This allows debug logging to be dynamically controlled at runtime without recompilation.

3.4 Logging Macros

The header defines a macro for informational logging:

- `INFO(A)` — Writes informational messages to `std::cerr`, including file name and line number metadata

This macro is intended for high-level execution tracing and diagnostics.

3.5 Test Execution Macro

A standardized macro is provided for executing unit tests:

- `TEST(f)` — Executes a test function `f()`, reports success or failure, and terminates the program upon uncaught exceptions

This ensures consistent test output and immediate feedback when a test fails.

3.6 Debug Mode Detection

Debug mode is automatically enabled based on platform- and build-specific preprocessor definitions:

- `_DEBUG` — Commonly defined in Windows debug builds
- `_GLIBCXX_DEBUG` — Used in GNU/Linux debug builds
- `DEBUG` — Generic debug flag

If any of these macros are defined, `DEBUG_MODE` is enabled.

3.7 Assertion Macros

Assertion behavior differs depending on whether debug mode is active.

3.7.1 Release Mode

When debug mode is disabled, assertion and debug macros expand to no-ops:

- `DEBUG_PRINT(A)`
- `ASSERT(c)`
- `ASSERT_APPROX_EQUAL(x, y, tolerance)`

This ensures zero runtime overhead in production builds.

3.7.2 Debug Mode

When debug mode is enabled, assertions provide detailed diagnostics and throw runtime exceptions upon failure:

- `DEBUG_PRINT(A)` — Conditionally logs debug messages
- `ASSERT(c)` — Verifies boolean conditions
- `ASSERT_APPROX_EQUAL(x, y, tolerance)` — Validates approximate equality for floating-point comparisons

All assertion failures report file and line number information to aid debugging.

3.8 Internal State

A static boolean variable maintains the current debug state:

- `debugEnabled` — Controls whether debug messages are emitted

This variable is accessed exclusively through the provided accessor functions.

3.9 Design Notes

The design emphasizes minimal runtime overhead when debugging is disabled while offering detailed diagnostics during development and testing.

- The header is protected using `#pragma once`.
- Macros are used to eliminate overhead when debugging is disabled.
- Assertions throw exceptions rather than aborting, allowing test harnesses to catch failures cleanly.
- Diagnostic output is directed to `std::cerr` to separate it from program output.

This was a high level discussion of the unit testing framework. We will see how it is used in practice through out this application. The key takeaways here is that we have a unit testing framework and this is going to allow us to extended the application further and further with confidence.

4 Exception Classes

4.1 Overview

We will discuss all the user defined exception classes here. Words cannot underscore how critical it is to gracefully handle exceptions that arise during execution. The resulting consequences could be catastrophic if our applications don't work properly. It's not just a matter of software, it's our bank accounts and our entire way of life.

The exception classes serves as the third section after unit testing for a reason. Developing our exception classes early on saves time later and documents our system requirements in stone. More importantly, the unit testing and exception classes form the bedrock of our application, which improves maintainability.

Here we define a collection of custom exception classes to represent error conditions encountered during our transactions. All exceptions inherit from `std::runtime_error`, allowing them to integrate seamlessly with standard C++ exception handling mechanisms while providing meaningful, contextual error messages.

4.2 Exception Hierarchy

As stated before all exception classes derive directly from:

- `std::runtime_error`

This design allows exceptions to be thrown without requiring explicit compile-time handling while still supporting detailed diagnostics.

4.3 Financial and Transaction Exceptions

4.3.1 SpendingLimitException.hpp

As we will see later each credit card is randomly assigned either a Basic, Gold, Platinum level. Each level has it's own spending limits. This error gets thrown when a credit card transaction exceeds the allowed spending limit for the card's level.

This is its implementation:

```
#include <stdexcept>

/*
    Spending Limit exception
    gets thrown when a credit card limit is reached
*/
class SpendingLimitException: public std::runtime_error {
public:
    // Spending limit exception constructor
    SpendingLimitException(std::string limitLevel): std::runtime_error(
        // Write error message
        "Over spending limit reached for card level: "
        + limitLevel
    ){}
};
```

The key things to note here are:

- Accepts the card's limit level as a constructor argument
- Generates a descriptive error message identifying the limit tier

We will see how this exception is used in `CreditCard.hpp` and `CreditAccount.hpp`.

4.3.2 InsufficientFundsException.hpp

This is the most obvious exception of them all. We do not want clients to overdraw from their accounts. No matter the transaction, this exception gets thrown when there is an insufficient balance in the account to complete the transaction. The error is logged to the accounts transaction history.

This is the implementation:

```
#include <stdexcept>
#include <string>

class InsufficientFundsException : public std::runtime_error{
    /*
        Insufficient fund exception
        gets throw when there is a withdral from the account
        but not enough funds in the account
    */
public:
    InsufficientFundsException(std::string balance):
        std::runtime_error(
            "Insufficient funds. Account has a balance of "
            + balance
        ) {}
};
```

The key items to note here are:

- Accepts the current account balance as a string
- Provides contextual feedback indicating available funds

We will see examples of the the insufficient funds exception getting thrown throughout the application.

4.3.3 InvalidAmountException.hpp

This application randomly generates the amount of money for each transaction instead of us directly specifying the transaction amount. By design it generates numbers from the set of integers $\mathbf{Z}$. This means that our numbers can be signed and unsigned, so negative transaction amounts may occur.

We do not want the application to perform a transaction on a negative transaction amount. This exception gets thrown when a transaction with an invalid monetary amount get generated.

This is the implementation:

```
#include <stdexcept>
#include <string>

class InvalidAmountException : public std::runtime_error {
    /*
        Invalid amount exception
        gets thrown on transactions with an invalid amount
    */
public:
    // Invalid amount exception constructor
    InvalidAmountException(std::string transactionAmount)
        : std::runtime_error(
            transactionAmount + " is an invalid amount."
        ){}
};
```

The key things to not here are:

- Accepts the invalid transaction amount
- Reports the rejected value directly in the error message so that we could display it later

We will see this exception thrown throughout the application, as any potential transaction could potentially attempt to transact with an invalid dollar amount.

4.4 Account State Exceptions

4.4.1 InvalidAccountException.hpp

There may be circumstanced where we try to perform a transactions with an incorrect account type. For example, if we tried to perform a checking transaction with a savings account. This exception gets thrown when a transaction is attempted on an inappropriate or incorrect account.

This is the implementation:

```
#include <stdexcept>
#include <string>

class InvalidAccountException : public std::runtime_error {
    /*
        Invalid Account Error
        Gets thrown when the user attempts to perform
        a transaction on the wrong account
    */
public:
    // Invalid Account Exception Constructor
    InvalidAccountException(): std::runtime_error(
        // write error message
        "Invalid account for this transaction."
    ){}

    InvalidAccountException(std::string errorMessage): std::runtime_error(
        // Invalid Account Exception error message constructor
        errorMessage // print error message
    ){}
};
```

The key things to note here are:

- The exception provides a default error message
- Allows a custom error message to be provided for greater detail

We shouldn't expect to see this exception to many times.

4.4.2 AccountOpenException.hpp

This exception is pretty straightforward. It gets thrown when an attempt is made to open an account that is already open.

Here is the implementation:

```
#include <stdexcept>

/*
    Account Open Exception
    Gets thrown when an account that is already open is
    re-opened
*/
class AccountOpenException: public std::runtime_error{
public:
    // Account open Exception constructor
    AccountOpenException(): std::runtime_error(
        "Account already open."
    ){}
};
```

As we can see the exception provides an error message that will appear on the receipt in the event that the exception is thrown. We will go into how later in this document.

4.4.3 AccountClosedException.hpp

Same as the exception the one before except it gets thrown when an operation attempts to close an account that is already closed.

Here is the implementation:

```
#include <stdexcept>

class AccountClosedException : public std::runtime_error {
/*
    Account Closed Exception class
    Gets thrown when an account that is closed get closed
    again
*/
public:
    // AccountClosedException constructor
    AccountClosedException(): std::runtime_error(
        "Account is already closed."
    ){}
};
```

Note the same custom error message as the pervious exceptions

4.5 Check Processing Exceptions

4.5.1 PostDatedCheckException.hpp

Every check has a date on it, and sometimes people write checks that they expect to be cashed at some time in the future. We will see that the dates that are assigned to the checks in our checking transaction are random and some of the checks may be dated for a future date. This exception is thrown when a check is post-dated and therefore invalid for processing.

Here is the implementation:

```
#include <stdexcept>

/*
    Post Dated Check Exception
    gets thrown when a check is post dated
*/
class PostDatedCheckException: public std::runtime_error{
public:
    // Post dated check exception constructor
    PostDatedCheckException(): std::runtime_error(
        "Check is Post Dated"
    ){}
};
```

Note the custom exception message.

4.5.2 CheckTooOldException.hpp

Sometimes people wait too long to cash checks. The application generates random dates older than six months. This exception get thrown when a check is older than the allowable processing window.

Here is the implementation:

```
#include <stdexcept>
#include <string>

class CheckTooOldException : public std::runtime_error{
/*
    CheckTooOldException
    get through if a check is too old to clear
*/
public:
    // Check Too old Exception constructor
    CheckTooOldException(std::string checkDate)
        : runtime_error(
            // Write error
            checkDate + " was more than 6 months ago"
        ){}
};
```

Some key things to note here:

- Accepts the original check date as a parameter
- Indicates that the check exceeds the six-month validity period

This detailed error message will provide us with greater fidelity when the exception is thrown. We can know the exact date that was on the check.

4.6 Time-Based Financial Exceptions

4.6.1 CDMaturityDateException

Remember the rules for CD account transactions. Deposits and withdrawals cannot be made before the accounts maturity date. We have a exception explicitly for any of these circumstances. This exception gets thrown when a transaction is attempted on a Certificate of Deposit before its maturity date.

Here is the implementation:

```
#include <stdexcept>
#include <string>

class CDMaturityDateException : public std::runtime_error {
```

```

    /*
        CD Maturity Date Exception
        get thrown whenever there is a transaction before the
        CD maturity date
    */
    public:
        // CD Maturity Date Exception Constructor
        CDMaturityDateException(std::string maturityDate)
            : std::runtime_error(
                // write error message
                "Maturity Date: " + maturityDate +
                " not reached yet."
            ){}
};

```

Key things to note here:

- Accepts the CD maturity date
- Reports that the maturity condition has not yet been met

As before this exception provides us clarity on the error. We can see what maturity date was randomly generated.

4.6.2 CardExpiredException

Lastly, we will see that each credit card is assigned a randomly generated expiration date. This exception gets thrown when a credit card is used after its expiration date.

Here is the implementation:

```

#include <stdexcept>

/*
    Card Expired Exception
    Gets thrown when a credit card expires
*/
class CardExpiredException: public std::runtime_error{
    public:
        // CardExpired Exception constructor
        CardExpiredException(): std::runtime_error{
            // Write error message
            "Card Expired"
        }{}
};

```

We have a custom error message. We could included the date, but for now we just don't want the card to function and get a simple error message. We know what is wrong.

4.7 Design Notes

This conclude our section on the exception classes. In summary, using specialized exception types improves error clarity, enforces business rules, and enables precise error handling in higher layers of the system. We will see in later sections exactly where the exceptions are used. Here are some of the key features we discussed:

- All exceptions inherit from `std::runtime_error` for consistency and simplicity.
- Exception messages are constructed at throw-time to preserve contextual information.
- Domain-specific exception types improve readability and maintainability compared to generic error handling.

- Exceptions are intended to be caught at service or controller layers rather than at the point of failure.

We made our exceptions early on. Now, all we have to do is throw them as we build the rest of the application like it were a native library.

5 Name.hpp

5.1 Overview

This header file contains the `Name` class, which represents a depositor's personal name, consisting of a first name and a last name. This gives us flexibility in the way that we store and reference names. The name of a client is its own object with specifications about how it should be written or displayed. Instead of manually hard coding how we want to print our names each time or getting creative in the ways we reference the first and last name, we define those specifications here in the `Name` class. This class encapsulates name-related data and provides controlled access through getter and formatting functions.

5.2 Design Rationale

We will go into more design detail as we walk through the instructions, but these are the primary purposes to pay attention to:

- Encapsulation ensures name data is accessed only through defined member functions.
- Constructors support flexible object creation, including default, parameterized, and copy semantics.
- A dedicated formatting function avoids repeated string concatenation logic throughout the system.

5.3 Class Declaration

Here we have the class declarations of the `Name` class. Notice the naming convention when we declared the private member variables. Using descriptive variable names is always heavily emphasized, but it truly is an art and a science. We use `firstName` and `lastName` to represent the depositor's first name and last name, respectively. We do not want to keep returning to this file to recall the exact spelling of the first name and last name variables. We will see exactly where this is helpful in later sections, remember the first programmatic paradigm.

Header Definitions

```
(Name.hpp)
#pragma once

#include "Stdafx.hpp"

class Name {
public:
    Name();
    Name(std::string firstName, std::string lastName);
    Name(const Name& otherName);

    std::string getFirstName();
    std::string getLastName();
    std::string printName();

private:
    std::string firstName;
    std::string lastName;
};
```

Unit Testing

```
// Unit Testing
void testName();
```

Notice that the getter functions include the same naming convention of the variable that it is meant to return. You probably also noticed that there are no setter functions. We will understand why in the next part.

5.4 Constructors

This is the part where we have to take moment and understand what we are actually writing in stone. There are a several different types of constructors in C++. We could choose to construct our objects a hundred different ways with some being more efficient than others. Generally, we do not want to have a long list of parameters for our constructors. Hence, we use setters to deliberately set each variable and guarantee that our variables are getting initialized, which is crucial.

However, this practice introduces a security vulnerability if not used correctly. Our getter methods are public, so using a **public** setter method would, by design, give anyone the ability to mutate the data at the address that the first name and last name data is stored in. After all, we would not want anyone other than the dedicated bank employees whose sole job function is to manipulate account balances to set our account balances to anything.

The proper way to offset this security vulnerability is to use **private** setter methods, but those can only be used in **Name** class methods. That is perfectly fine, because the helper methods do not need to be in the header file anyway. Not that it is computationally crucial, but rather the client just does not care.

For this application we will use a no argument constructor or default, keyword argument constructor, and a copy constructor.

5.4.1 Default Constructor

This is the **Name** default constructor. It does not take any arguments, and it is critical to remember that the member variables still need to get initialized. Using the colon is the fastest way to initialize those variables. We could initialize the variables in their declarations, but generally we do not want anyone to know the variables value and this gives us greater variable setting accuracy. We gain greater confidence that our variables are going to get set to *something* and that *something* is what we intend it to be. This is our first line of defense against null pointer errors.

```
Name::Name() : firstName(" "), lastName(" ") {}
```

- Ensures the object is always in a valid state
- Useful for deferred assignment or testing

Lastly, we used double quotation marks with a space and not "".

5.4.2 Parameterized Constructor

As I stated before, we generally do not want a long list of parameters, because doing so could introduce the potential that we accidentally mismatch the parameters and the variables that we expect them to be assigned to. The key word here is expect, because it could be extremely difficult to figure out and locate the problem if that error arises. That's why we have one constructor with both data members. This constructor in conjunction with our default constructor makes it impossible for us to encounter null pointer errors.

If a name exists, it's members have a value. So we do not need to worry that at some point during the computation there will be a function call on a **Name** object that we inadvertently initialized and it doesn't return anything. The application still works even if it returns a value that we do not want.

Here is the parameterized constructor declaration:

```
Name::Name(std::string firstName, std::string lastName)
: firstName(firstName), lastName(lastName) {}
```

Again we keep the naming convention of the parameters as close to the spelling of the data member that it references so that there is no confusion.

- `firstName` — The depositor's given name
- `lastName` — The depositor's family name

5.4.3 Copy Constructor

The copy constructor creates a deep copy of an existing `Name` object. This makes it easier to return data from an existing object without concern that the information belonging to that specific object will be mutated. It also prevents anyone else aside from the people that we want to know the exact location of our account information from accessing it.

```
Name::Name(const Name& otherName)
: firstName(otherName.firstName),
  lastName(otherName.lastName) {}
```

5.5 Member Functions

This are the name class member functions. This class is pretty small, so a bunch of member functions is not needed. A key thing to note here is the naming conventions of the member functions. The getter methods are directly aligned with variables that they are meant to return and the start with the word "get", now we don't have to remember what we called our getter methods. All we have to do is type "get" and the name of the variable that it is programmed to return. Remember the first programmatic paradigm. Let examine what we do have.

5.5.1 `getFirstName()`

This is the get first name function, it returns the stored first name, a string.

```
std::string Name::getFirstName() {
    return firstName;
```

5.5.2 `getLastName()`

This is that get last name function, it returns the stored last name, a string.

```
std::string Name::getLastName() {
    return lastName;
```

5.5.3 `printName()`

This is the print first name method. It returns a formatted full name in the form:

`<FirstName> <LastName>`

This is a concatenation with a " " in between the first and last names.

```
std::string Name::printName() {
    return firstName + " " + lastName;
}
```

5.6 Unit Testing

Now we can beginning tying in our unit testing framework with our first class. Remember the second programmatic paradigm. We can test this by initializing a name object and printing it to the console to confirm the `Name` object creation. However, we would need to re-run that same console test if we make any changes to the program.

5.6.1 Testing Strategy

These are the important features of the `Name` unit tests that we want to verify:

- Correct storage of first name
- Correct storage of last name
- Correct formatting of full name output

To perform the test we are going to call the testing macros that are defined in the `Testing.hpp` header file.

5.6.2 Test Implementations

Test: First Name Getter The first test we run is to confirm the first name getter.

```
static void testGetFirstName() {
    Name testFirstName("Nickolas", "Black");
    ASSERT(testFirstName.getFirstName() == "Nickolas");
}
```

This confirms that the name "Nickolas" is getting stored in the `firstName` variable and the `getFirstName()` member function is returning what is located at `firstName`.

Test: Last Name Getter The second test we run is to confirm the last name getter is functioning properly.

```
static void testGetLastName() {
    Name testLastName("Nickolas", "Black");
    ASSERT(testLastName.getLastName() == "Black");
}
```

This confirms that the name "Black" is getting stored in the `lastName` data member and the `getLastName()` member function is returning what is located at `lastName`.

Test: Name Formatting Next we test that the `printName()` member function is return the concatenated string as we expect.

```
static void testPrintName() {
    Name testName("Nickolas", "Black");
    ASSERT(testName.printName() == "Nickolas Black");
}
```

5.6.3 Test Runner

Lastly, we have our voided `testName()` member function, which when called will run test all the specific unit test cases for the entire name class. This allow use to run several unit test with one function.

```
void testName() {
    TEST(testGetFirstName);
    TEST(testGetLastName);
    TEST(testPrintName);
}
```

5.7 Implementation

5.7.1 Name.hpp

The entire `Name.hpp` header file should look like this:


```

#pragma once

#include "Stdafx.hpp"

class Name{
    /*
        Name Class
        Contains the first and last name of a depositor
    */
public:
    // No args constructor
    Name();
    // Keyword Args constructor
    Name(std::string firstName, std::string lastName);
    // Copy constructor
    Name(const Name& otherName);

    // Name Getter functions
    std::string getFirstName(); // returns first name
    std::string getLastName(); // returns last name

    // Name Print
    std::string printName(); // returns {firstName} {lastName}
private:
    std::string firstName; // first name
    std::string lastName; // last name
};

// Unit Testing
void testName();

```

5.7.2 Name.cpp

The entire Name.cpp with the Name class function implementations should appear as the following:

```

#include "Name.hpp"

using namespace std;

// Name No Args constructor
Name::Name() : firstName(" "), lastName (" "){}
// Name Keyword args constructor

Name::Name(string firstName, string lastName) :
    firstName(firstName), lastName(lastName) {}

// Name copy constructor
Name::Name(const Name& otherName) :
    firstName(otherName.firstName), lastName(otherName.lastName) {}

// Name getter functions
string Name::getFirstName(){return firstName;} // return first name
string Name::getLastName(){return lastName;} // return last name

// Name print function
string Name::printName(){
    // return {firstName} {lastName}
    return firstName + " " + lastName;
}

```

```

// Unit Testing Series
static void testGetFirstName(){
    // Get first name unit test
    Name testFirstName = Name("Nickolas", "Black");
    ASSERT(testFirstName.getFirstName() == "Nickolas");
}

static void testGetLastName(){
    // Get last name unit test
    Name testLastName = Name("Nickolas", "Black");
    ASSERT(testLastName.getLastName() == "Black");
}

static void testPrintName(){
    // Print Name unit test
    Name testName = Name("Nickolas", "Black");
    ASSERT(testName.printName() == "Nickolas Black");
}

void testName() {
    TEST(testGetFirstName); // Test get first Name
    TEST(testGetLastName); // test get last name
    TEST(testPrintName); // test print name
}

```

5.8 Summary

The `Name` class provides a clean, minimal abstraction for handling personal name data. Its design emphasizes encapsulation, clarity, and ease of testing, making it a reliable foundational component within the banking system.

6 Depositor Class

6.1 Overview

Next we have the `Depositor` class. This represents an individual client of the banking system. It encapsulates the depositor's identity by combining a `Name` object with a Social Security Number (SSN).

This class serves as a foundational identity component for accounts and transactions, ensuring depositor information is managed consistently and securely. Each account will have a depositor and this can be extended to include an address or other personal identifying information.

6.2 Design Rationale

The key ideas here are :

- Composition is used to embed a `Name` object rather than duplicating name fields.
- Encapsulation restricts direct access to sensitive data such as Social Security Numbers.
- Copy semantics allow safe duplication of depositor records.
- A dedicated print function standardizes depositor output formatting.

6.3 Class Declaration

Here we have the class declarations for the `Depositor` class. Note our three constructor and the same getter and setter rationale. Also notice that our Social Security number is of data type `std::string`, this is because although a social security number should be a integer, we should not expect any arithmetic to occur on depositor's social security numbers.

Header Definitions

```
#pragma once

#include "Stdafx.hpp"
#include "Name.hpp"

class Depositor {
public:
    Depositor();
    Depositor(const Name& depositorName, std::string ssn);
    Depositor(const Depositor& otherDepositor);

    Name getDepositorName();
    std::string getDepositorSSN();

    std::string printDepositor();

private:
    Name depositorName;
    std::string socialSecurityNumber;
};
```

Unit Testing

```
// Unit testing
void testDepositor();
```

Same as before in the `Name` class. There is no real unique difference between `Depositor` class and the `Name` class.

6.4 Constructors

6.4.1 Default Constructor

This is the `Depositor` class default constructor. Sticking with the same purpose and rationale as before, notice that the `socialSecurityNumber` (you know what that is) is initialized to a " ".

```
Depositor::Depositor()
: depositorName(Name()),
  socialSecurityNumber(" ") {}
```

Another key thing to note is how the `depositorName` gets initialized with a `Name()` object. Recall, when we implemented the `Name` class default constructor we specified how the `firstName` and `lastName` variables are initialized to " ". In other words, it's a user defined null name. We did the same thing here for `depositor`, so when we initialize a null `depositor` using the default constructor we are also initializing a null name. This is helpful, because if we inadvertently call `Name` functions on a null `Depositor` object some value will be returned.

6.4.2 Parameterized Constructor

Here is the keyword argument constructor. It takes a copy of a `Name` object address and a `std::string` of the social security number named `ssn` here.

```
Depositor::Depositor(const Name& depositorName, std::string ssn)
: depositorName(depositorName),
  socialSecurityNumber(ssn) {}
```

These are the parameters of the `Depositor` keyword argument constructor in getter detail:

- `depositorName` — The depositor's full name, contains their first name and last name, along with an ability to print it in a formatted method
- `ssn` — The depositor's Social Security Number

6.4.3 Copy Constructor

This is the `Depositor` class copy constructor. It creates a deep copy of an existing `Depositor` object and allows us to see the information without compromising our data. We will see this in practice in the next part.

```
Depositor::Depositor(const Depositor& otherDepositor)
    : depositorName(otherDepositor.depositorName),
      socialSecurityNumber(otherDepositor.socialSecurityNumber) {}
```

6.5 Member Functions

6.5.1 getDepositorName()

This function returns a copy of the depositor's name. Notice the use of the copy constructor. When called this function return the address of a copy of the depositor object.

```
Name Depositor::getDepositorName() {
    return Name(depositorName);
}
```

6.5.2 getDepositorSSN()

This method returns the depositor's Social Security Number.

```
std::string Depositor::getDepositorSSN() {
    return socialSecurityNumber;
}
```

6.5.3 printDepositor()

This function returns a formatted string representation of the depositor. It builds on top of the `Name` class `printName()` method. Recall that we specified exactly how a name should appear. We add the formatted name with the depositor social security number and a some detail.

Client Name: <First Last> SSN: <Number>

Here is the implementation:

```
std::string Depositor::printDepositor() {
    return "Client Name: " + depositorName.printName() +
           "\t\tSSN: " + socialSecurityNumber;
}
```

6.6 Unit Testing

For the same reasons as before, we will include a unit testing framework to ensure the functionality of the `Depositor` class.

6.6.1 Testing Strategy

These are the functionality that the `Depositor` unit tests verify:

- Correct storage and retrieval of depositor name
- Correct storage and retrieval of SSN
- Proper formatting of depositor output

6.6.2 Test Implementations

Test: Depositor Name The first test we will implement tests the `Depositor` class' `Name` object initialization and `getDepositorName()` member function.

```
static void testGetDepositorName() {
    Name testName("Nickolas", "Black");
    Depositor testDepositor(testName, "123456789");
    ASSERT(testDepositor.getDepositorName().printName() ==
           "Nickolas Black");
}
```

Test: Depositor SSN The next test is to confirm the social security number and test the `getDepositorSSN()` member function.

```
static void testGetDepositorSSN() {
    Depositor testDepositor(Name(), "123456789");
    ASSERT(testDepositor.getDepositorSSN() == "123456789");
}
```

Test: Depositor Print Next we test the `printDepositor()` member function. A key thing to note here is that testing print methods will become slightly infeasible and we combine print methods like we have here. It is helpful to test the core methods and test the bigger printed statements visually.

```
static void testPrintDepositor() {
    Depositor testDepositor(
        Name("Nickolas", "Black"),
        "123456789"
    );
    ASSERT(testDepositor.printDepositor() ==
           "Client Name: Nickolas Black\t\tSSN: 123456789"
    );
}
```

6.6.3 Test Runner

We have another master unit testing voided function for the `Depositor` class. We will see how these functions are deployed in later sections.

```
void testDepositor() {
    TEST(testGetDepositorName);
    TEST(testGetDepositorSSN);
    TEST(testPrintDepositor);
}
```

6.7 Implementation

6.7.1 Depositor.hpp

The entire `Depositor.hpp` header file should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "Name.hpp"

class Depositor{
    /*
        Depositor class
        contains the Name of a depositor and their ssn,
```

```

    */
public:
    // Depositor no Args constructor
    Depositor();
    // Depositor keyword args constructor
    Depositor(const Name& depositorName, std::string ssn);
    // Depositor copy constructor
    Depositor(const Depositor& otherDepositor);

    // Depositor getter series
    Name getDepositorName(); // returns depositor name
    std::string getDepositorSSN(); // returns depositor ssn

    // Print depositor
    std::string printDepositor(); // returns {name} \t\t {ssn}
private:
    Name depositorName; // Depositor Name
    std::string socialSecurityNumber; // ssn
};

// Unit testing
void testDepositor();

```

6.7.2 Depositor.cpp

The entire Depositor.cpp source file should look like this:

```

#include "Depositor.hpp"

using namespace std;

// Depositor Constructor series
Depositor::Depositor():
    // Depositor no args constructor
    depositorName(Name()), socialSecurityNumber(" "){}

Depositor::Depositor(
    // Depositor key word Args constructor
    const Name& depositorName, // depositor name
    string ssn // social security number
): depositorName(depositorName), socialSecurityNumber(ssn){}

Depositor::Depositor(const Depositor& otherDepositor):
    // Depositor copy constructor
    depositorName(otherDepositor.depositorName), // copy depositor name
    socialSecurityNumber(otherDepositor.socialSecurityNumber){} // copy ssn

// Depositor getter series
Name Depositor::getDepositorName(){return Name(depositorName);} // return depositor name
string Depositor::getDepositorSSN(){return socialSecurityNumber;} // return ssn

// Depositor print function
string Depositor::printDepositor(){
    return "Client Name: " + depositorName.printName() + // Client Name:
           "\t\tSSN: " + socialSecurityNumber; // SSN:
}

/* Unit Testing Series */
static void testGetDepositorName(){

```

```

    // Get despoitor name unit test
    Name testName = Name("Nickolas", "Black");
    Depositor testDepositor = Depositor(testName, "123456789");
    ASSERT(testDepositor.getDepositorName() == "Nickolas Black");
}

static void testGetDepositorSSN(){
    // get depositor ssn unit test
    Depositor testDepositor = Depositor(
        Name(), // null name
        "123456789" // ssn
    );
    ASSERT(testDepositor.getDepositorSSN == "123456789");
}

static void testPrintDepositor(){
    // Print depositor unit test
    Depositor testDepositor = Depositor(
        Name("Nickolas", "Black"), // test name
        "123456789" // test ssn
    );
    ASSERT(testDepositor.printDepositor() ==
        "Client Name: Nickolas Black          SSN: 123456789"
    );
}

void testDepositor(){
    // Test Depositor unit test. Tests all functions
    TEST(testGetDepositorName); // test get name
    TEST(testGetDepositorSSN); // test get ssn
    TEST(testPrintDepositor); // test print depositor
}

```

6.8 Summary

The `Depositor` class provides a clean abstraction for managing client identity data. By leveraging composition and strong encapsulation, it supports safe data handling, consistent formatting, and reliable testing throughout the application. It also gives us the ability to categorize accounts by the client associated with the account. This is how we will be able to aggregate all the accounts for a particular depositor.

7 Calendar Class

7.1 Overview

Next we have the `Calendar` class. This represents a date object used throughout the banking system to timestamp transactions and compare dates. In java the calendar class is a part of the `java.util` library. C++ has a similar class in the `<chrono>` library, however, as stated before it is unavailable on Mac ARM64 chips. We created our own calendar class, which has a particular benefit: we eliminated the overhead of importing an external dependency. All of our calendar logic binaries is in our package, which makes our application faster than if we had imported it from an external dependency.

It encapsulates three fundamental components of a date:

- Month
- Day
- Year

The `Calendar` class allows users to determine whether one date occurs before or after another, provides overloaded comparison operators, and supports month-based arithmetic operations. This makes it especially useful for validating transactions such as post-dated checks, expired cards, and CD maturity dates.

7.2 Design Rationale

Aside from helping us manage time logic, the key ideas here are:

- Encapsulation ensures date components are accessed only through controlled methods.
- Operator overloading allows natural chronological comparisons.
- Dedicated comparison methods (`before`, `after`) improve readability.
- Month arithmetic operators support expiration and maturity calculations.
- A print formatter standardizes how dates appear throughout the system.

7.3 Class Declaration

Header Definitions Here we have the class declaration for the `Calendar` class. Notice the constructor overloads, comparison methods, and arithmetic operators.

```
#pragma once

#include "Stdafx.hpp"

class Calendar{
public:
    Calendar();
    Calendar(int month, int day, int year);
    Calendar(int month, int year);
    Calendar(const Calendar& otherCalendar);

    int getMonth();
    int getDay();
    int getYear();

    bool before(Calendar otherCalendar) const;
    bool after(Calendar otherCalendar) const;

    std::string printDate();

    bool operator<(const Calendar& otherCalendar) const;
    bool operator>(const Calendar& otherCalendar) const;
    bool operator<=(const Calendar& otherCalendar) const;
    bool operator>=(const Calendar& otherCalendar) const;
    bool operator==(const Calendar& otherCalendar) const;
    bool operator!=(const Calendar& otherCalendar) const;

    Calendar operator+(int month) const;
    Calendar operator-(int month) const;

private:
    int month;
    int day;
    int year;
};
```


Unit Testing

```
void testCalendar();
```

7.4 Constructors

7.4.1 Default Constructor

The default constructor initializes a “null” or placeholder date. As stated before, this is useful when creating temporary objects before assigning meaningful values.

```
Calendar::Calendar()
    : month(1), day(1), year(1900) {}
```

7.4.2 Parameterized Constructor (Full Date)

This constructor initializes a complete date with month, day, and year.

```
Calendar::Calendar(int month, int day, int year)
    : month(month), day(day), year(year) {}
```

- month — Month value (1–12)
- day — Day value (1–31)
- year — Four-digit year

7.4.3 Credit Card Constructor (Month / Year)

This constructor is designed for expiration-based logic (such as credit cards), where only month and year are relevant. We will implement `CreditCard.hpp` in a later section

```
Calendar::Calendar(int month, int year)
    : month(month), day(1), year(year) {}
```

7.4.4 Copy Constructor

Creates a deep copy of another `Calendar` object.

```
Calendar::Calendar(const Calendar& otherCalendar)
    : month(otherCalendar.month),
      day(otherCalendar.day),
      year(otherCalendar.year) {}
```

7.5 Member Functions

7.5.1 `getMonth()`, `getDay()`, `getYear()`

These functions return the corresponding date component.

```
int Calendar::getMonth(){ return month; }
int Calendar::getDay(){ return day; }
int Calendar::getYear(){ return year; }
```

7.5.2 `before()`

This member function determines if the current date occurs before another date. Return turns `true` if it is before and `false` if it is after.

```
bool Calendar::before(Calendar otherCalendar) const {
    return (*this < otherCalendar);
}
```

7.5.3 after()

This member function determines if the current date occurs after another date. Return turns `false` if it is before and `true` if it is after.

```
bool Calendar::after(Calendar otherCalendar) const {
    return (*this > otherCalendar);
}
```

7.5.4 printDate()

This member function returns a formatted string representation of the date in:

MM/DD/YYYY

```
std::string Calendar::printDate(){
    return std::to_string(month) + "/" +
           std::to_string(day) + "/" +
           std::to_string(year);
}
```

Just like in the classes before we just specified how all printed dates should appear whenever printed.

7.6 Operator Overloading

These are overloaded operators that allow natural chronological comparisons.

7.6.1 Comparison Operators

```
bool Calendar::operator<(const Calendar& other) const {
    if(year != other.year) return year < other.year;
    if(month != other.month) return month < other.month;
    return day < other.day;
}
```

Other comparison operators are defined in terms of `<` and `>`.

7.6.2 Arithmetic Operators

Month-based arithmetic is useful for maturity dates and expiration checks. We these overloaded operates we could add years or months to a `Calendar` object with `+` or `-`.

```
Calendar Calendar::operator+(int months) const {
    int newMonth = month + months;
    int newYear = year + (newMonth - 1) / 12;
    newMonth = ((newMonth - 1) % 12) + 1;
    return Calendar(newMonth, day, newYear);
}

Calendar Calendar::operator-(int months) const {
    return (*this + (-months));
}
```

7.7 Unit Testing

The unit testing here is critical. Many of our transactions will rely on date logic and if the class does not function properly we could expect inconsistent results.

7.7.1 Testing Strategy

The `Calendar` unit tests verify:

- Correct initialization of dates
- Accurate chronological comparisons
- Correct month arithmetic
- Proper formatted output

7.7.2 Test Implementations

Test: Before Comparison Here we test the `before()` function by creating two `Calendar` objects and confirming that one is before the other.

```
static void testBefore(){
    Calendar date1(1,1,2024);
    Calendar date2(2,1,2024);
    ASSERT(date1.before(date2));
}
```

Test: Operator Comparison This is the same test as before except we confirm the same result using the overloaded operator itself.

```
static void testOperatorLess(){
    Calendar date1(1,1,2024);
    Calendar date2(1,1,2025);
    ASSERT(date1 < date2);
}
```

Test: Month Arithmetic This confirms our ability to add months to a `Calendar` object.

```
static void testMonthAddition(){
    Calendar date(11,1,2024);
    Calendar result = date + 2;
    ASSERT(result.getMonth() == 1 &&
           result.getYear() == 2025);
}
```

7.7.3 Test Runner

As before, we have a unique method that will run all the tests in the `Calendar` class. This is here to confirm complete functionality.

```
void testCalendar(){
    TEST(testBefore);
    TEST(testOperatorLess);
    TEST(testMonthAddition);
}
```

7.8 Implementation

7.8.1 `Calendar.hpp`

This is the entire implementation of the `Calendar.hpp` header file:

```

#pragma once

#include "Stdafx.hpp"

/*
    Calendar Class
    This is a date object for transactions
    it contains a month, day and year variable.
    Users can test if a date is before or after another
*/
class Calendar{
public:
    // Calendar constructor series
    Calendar(); // No args constructor
    Calendar(int month, int day, int year); // Keyword args constructor
    Calendar(int month, int year); // credit car constructor
    Calendar(const Calendar& otherCalendar);

    // Get getter Series
    int getMonth(); // returns month
    int getDay(); // returns day
    int getYear(); // returns year

    // Calendar computational series
    bool before(Calendar otherCalendar) const; // checks before
    bool after(Calendar otherCalendar) const; // checks after

    // Formatter
    std::string printDate(); // returns {mm/dd/yyyy}

    // Calendar overloaded operators
    bool operator<(const Calendar& otherCalendar) const; // <
    bool operator>(const Calendar& otherCalendar) const; // >
    bool operator<=(const Calendar& otherCalendar) const; // <=
    bool operator>=(const Calendar& otherCalendar) const; // >=
    bool operator==(const Calendar& otherCalendar) const; // ==
    bool operator!=(const Calendar& otherCalendar) const; // !=
    Calendar operator+(int month) const; // +
    Calendar operator-(int month) const; // -

private:
    int month; // month
    int day; // day
    int year; // year
};

// unit testing
void testCalendar();

```

7.8.2 Calendar.cpp

This is the entire implementation of the `Calendar.cpp` source file:

```

#include "Calendar.hpp"

using namespace std;

// Calendar class constructor series
Calendar::Calendar(){

```

```

    // Get time now
    auto now = std::chrono::system_clock::now();
    std::time_t t = std::chrono::system_clock::to_time_t(now);
    std::tm local = *std::localtime(&t);

    // Set month year day month to time now
    year = local.tm_year + 1900; // years since 1900
    month = local.tm_mon + 1;    // 0-11 → 1-12
    day = local.tm_mday;
}

Calendar::Calendar(int month, int day, int year):
    // Key word args constructor
    month(month), day(day), year(year){}

Calendar::Calendar(int month, int year):
    // Canlendar Credit Card date constructor
    month(month), day(01), year(year){}

Calendar::Calendar(const Calendar& otherCalendar):
    // Calendar copy constructor
    month(otherCalendar.month), // copy month
    day(otherCalendar.day), // copy day
    year(otherCalendar.year) {} // copy year

/* Calendar getter series */
int Calendar::getMonth(){
    // Calendar get month. returns month
    return month; // return month
}

int Calendar::getDay(){
    // Calendar get day object. returns day
    return day; // return day
}

int Calendar::getYear(){
    // Calendar get year object. returns years
    return year; // return year
}

/* Calendar Computational Series */
bool Calendar::after(Calendar otherCalendar) const {
    // Calendar before function. Checks if a calendar object is before
    // the other
    return *this > otherCalendar;
}

bool Calendar::before(Calendar otherCalendar) const {
    // Calendar after method. chekcs if a calendar date is after another
    return *this < otherCalendar;
}

/* Calendar Formatter series */
string Calendar::printDate(){
    // Calendar print date method. prints a formatted string of the date
    return to_string(month) + "/" + // print month
        to_string(day) + "/" + // print day

```

```

        to_string(year); // print year
    }

    /* Calendar Overloaded operator series */
    static int daysInMonth(int month, int year){
        // Calendar days In month method. returns the days in a particular month
        if (month == 2){
            // Account for number of days in february
            bool leap = (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0));
            return leap ? 29 : 28; // return number of february days
        }
        if (month == 4 || month == 6 || month == 9 || month == 11){
            return 30; // april, june, september, november
        }
        return 31; // 31 days
    }

    bool Calendar::operator<(const Calendar& otherCalendar) const{
        // Calendar class overloaded < operator method
        if (year != otherCalendar.year){
            // Check year
            return year < otherCalendar.year;
        }
        if (month != otherCalendar.month){
            // check month
            return month < otherCalendar.month;
        }
        // check day
        return day < otherCalendar.day;
    }

    bool Calendar::operator>(const Calendar& otherCalendar) const{
        // Calendar class overloaded > operator
        if (year != otherCalendar.year){
            // check year
            return year > otherCalendar.year;
        }
        if (month != otherCalendar.month){
            // check month
            return month > otherCalendar.month;
        }
        // check day
        return day > otherCalendar.day;
    }

    bool Calendar::operator<=(const Calendar& otherCalendar) const{
        // Calendar class overloaded <= operator
        return !(*this > otherCalendar);
    }

    bool Calendar::operator>=(const Calendar& otherCalendar) const{
        // Calendar class Overloaded >= operator
        return !(*this < otherCalendar);
    }

    bool Calendar::operator==(const Calendar& otherCalendar) const{
        // Calendar class overloaded == operator
        return year == otherCalendar.year && // years equal

```

```

        month == otherCalendar.month && // month equal
        day == otherCalendar.day; // day equal
    }

bool Calendar::operator!=(const Calendar& otherCalendar) const {
    // Calendar class overloaded != operator
    return !(*this == otherCalendar);
}

Calendar Calendar::operator+(int months) const {
    // Calendar class overload + operator. Adds months and returns
    // new calendar object
    int newMonth = month + months; // compute new month
    int newYear = year; // copy year
    // Compute year
    while (newMonth > 12){
        newMonth -= 12;
        newYear++;
    }
    // Compute number of days in month
    int newDay = day; // copy day
    int maxDay = daysInMonth(newMonth, newYear);
    if(newDay > maxDay){
        newDay = maxDay;
    }
    // Return new Calendar
    return Calendar(newMonth, newDay, newYear);
}

Calendar Calendar::operator-(int months) const {
    // Calendar Class overloaded - operator. subtracts months and
    // returns a new calendar object
    int newMonth = month - months; // subtract month
    int newYear = year; // copy year
    // compute year
    while (newMonth <= 0) {
        newMonth += 12; // add months
        newYear--; // subtract year
    }
    // Compute number of days in month
    int newDay = day; // copy day
    int maxDay = daysInMonth(newMonth, newYear);
    if (newDay > maxDay){
        newDay = maxDay;
    }
    // return new calendar
    return Calendar(newMonth, newDay, newYear);
}

/* Unit Testing Series */
static void testDate(){
    // create test calendar
    Calendar testDate = Calendar(01, 01, 2026);
    // assert
    ASSERT(testDate.getMonth() == 1);
    ASSERT(testDate.getDay() == 1);
    ASSERT(testDate.getYear() == 2026);
}

```

```

static void testAfter(){
    // Unit test for calendar after method
    Calendar testDate = Calendar(2, 15, 2025);
    Calendar otherTestDate = Calendar(7, 18, 2024);
    // Assert
    ASSERT(testDate.after(otherTestDate) == true);
}

static void testBefore(){
    // Unit test for calendar before method
    Calendar testDate = Calendar(8, 22, 2025);
    Calendar otherTestDate = Calendar(7, 13, 2026);
    // Assert
    ASSERT(testDate.before(otherTestDate));
}

static void testOperators(){
    // Calendar class unit test for operators
    Calendar a(1, 15, 2025); // test calendar
    Calendar b(1, 15, 2025);
    Calendar c(2, 1, 2025);
    // Assert
    ASSERT(a == b);
    ASSERT(c > a);
    ASSERT(a < c);
    ASSERT(a <= b);
    ASSERT(a >= b);
}

void testCalendar(){
    // Unit Test for Calendar class
    TEST(testDate); // test date
    TEST(testAfter); // test after function
    TEST(testBefore); // test before function
    TEST(testOperators); // test operators
}

```

7.9 Summary

The `Calendar` class provides a clean abstraction for managing chronological data within the banking system. Through encapsulation, operator overloading, and month arithmetic, it supports reliable date comparison and expiration logic. This class plays a crucial role in validating transactions, enforcing time-based rules, and ensuring system consistency.

8 TransactionTicket Class

8.1 Overview

Each transaction in our system begins by recording the transaction request. We accomplish this process by utilizing a `TransactionTicket`. Here we explore in detail the `TransactionTicket` class along with the `TransactionType` enumeration. This class represents a request to perform a transaction on a bank account. It serves as an immutable instruction object that carries all relevant transaction data before processing.

A `TransactionTicket` contains:

- Account number
- Transaction type

- Transaction amount
- Transaction date
- CD term (if applicable)

This abstraction separates transaction input data from account logic, ensuring clean architecture and traceable operations.

8.2 Design Rationale

The key ideas here are:

- An enumeration defines all valid transaction types, enforcing type safety.
- Encapsulation protects transaction data from accidental modification.
- Separation of ticket and receipt models mirrors real-world banking systems.
- A formatting function standardizes monetary output.
- Copy semantics allow safe duplication when passing tickets between objects.

8.3 TransactionType Enumeration

When can not context switch over strings in C++ like we can in java. We will define the transaction type in binaries using an enumeration. The `TransactionType` enum defines all supported transaction operations in the banking system. A key to note here, enumerations generate extensibility errors. They are one of the major drawbacks in C++, because extensions to the any enumeration requires extending some dependency else where. Each element in the enumeration not only needs further handling, but could limit our ability to run tests due to branch prediction, if not fully implemented. This enumeration ensures that only valid transaction categories may be used throughout the system.

```
enum TransactionType {  
    Deposit,  
    Withdrawl,  
    Balance,  
    Close,  
    Open,  
    BalanceDue,  
    PayBalance,  
    CreditTransaction,  
    ClearCheck,  
    Portfolio,  
    TransactionHistory,  
    ClientInfo  
};
```

This is every type of transaction that our system can perform. When making enumerations it is best to know all the possible elements beforehand and define them.

8.4 Class Declaration

Header Definitions These are the header definitions of the `TransactionTicket` class. Sticking with our naming conventions, notice we use the full name of the variable to describe the data being referenced and our getter methods also have the full variable name.

```
#pragma once  
  
#include "Stdafx.hpp"  
  
class TransactionTicket{
```

```

public:
    TransactionTicket();
    TransactionTicket(
        TransactionType transactionType,
        int accountNumber,
        double transactionAmount,
        Calendar transactionDate,
        int cdTerm
    );
    TransactionTicket(const TransactionTicket& otherTransactionTicket);

    int getAccountNumber();
    std::string getTransactionType();
    TransactionType getType();
    double getTransactionAmount();
    Calendar getTransactionDate();
    int getCDTerm();

    std::string printTransactionAmount();

private:
    int accountNumber;
    TransactionType transactionType;
    double transactionAmount;
    Calendar transactionDate;
    int cdTerm;
};

```

Unit Testing We add our unit testing function.

```
void testTransactionTicket();
```

8.5 Constructors

8.5.1 Default Constructor

The default constructor initializes a placeholder transaction ticket, where the account number is 000000, the transaction type is `Balance`, the transaction amount is 0.0, and the transaction date is the time that the ticket is initialized, and the term of CD account is 0.

```

TransactionTicket::TransactionTicket():
    accountNumber(000000),
    transactionType(TransactionType::Balance),
    transactionAmount(0.0),
    transactionDate(Calendar()),
    cdTerm(0) {}

```

This is useful when creating temporary tickets before assigning meaningful values.

8.5.2 Parameterized Constructor

This constructor initializes a fully specified transaction ticket. Notice the matching references so that there is no confusion.

```

TransactionTicket::TransactionTicket(
    TransactionType transactionType,
    int accountNumber,
    double transactionAmount,
    Calendar transactionDate,

```

```

    int cdTerm
):
    transactionType(transactionType),
    accountNumber(accountNumber),
    transactionAmount(transactionAmount),
    transactionDate(transactionDate),
    cdTerm(cdTerm) {}

```

Parameter details:

- `transactionType` — The type of transaction requested
- `accountNumber` — Account on which the transaction applies
- `transactionAmount` — Dollar amount involved
- `transactionDate` — Date of the transaction
- `cdTerm` — Term of CD (if applicable)

8.5.3 Copy Constructor

Creates a deep copy of an existing transaction ticket which is needed for reasons mentioned before.

```

TransactionTicket::TransactionTicket(
    const TransactionTicket& otherTransactionTicket
):
    transactionType(otherTransactionTicket.transactionType),
    accountNumber(otherTransactionTicket.accountNumber),
    transactionAmount(otherTransactionTicket.transactionAmount),
    transactionDate(otherTransactionTicket.transactionDate),
    cdTerm(otherTransactionTicket.cdTerm) {}

```

8.6 Member Functions

8.6.1 Getter Series

These functions provide controlled access to ticket data.

```

int TransactionTicket::getAccountNumber(){
    return accountNumber;
}

double TransactionTicket::getTransactionAmount(){
    return transactionAmount;
}

Calendar TransactionTicket::getTransactionDate(){
    return transactionDate;
}

int TransactionTicket::getCDTerm(){
    return cdTerm;
}

TransactionType TransactionTicket::getType(){
    return transactionType;
}

```

8.6.2 getTransactionType()

This method converts the TransactionType enum into a readable string.

```
string TransactionTicket::getTransactionType(){
    switch(transactionType){
        case TransactionType::Balance:
            return "Balance";
        case TransactionType::Deposit:
            return "Deposit";
        case TransactionType::Withdrawl:
            return "Withdrawl";
        case TransactionType::Open:
            return "Re-Open Account";
        case TransactionType::Close:
            return "Close Account";
        case TransactionType::ClearCheck:
            return "Clear Check";
        case TransactionType::BalanceDue:
            return "Balance Due";
        case TransactionType::PayBalance:
            return "Make Payment";
        case TransactionType::CreditTransaction:
            return "Credit Transaction";
        case TransactionType::Portfolio:
            return "Portfolio";
        case TransactionType::TransactionHistory:
            return "Transaction History";
        case TransactionType::ClientInfo:
            return "Client Information";
        default:
            return " ";
    }
}
```

8.6.3 printTransactionAmount()

This function formats the transaction amount as currency with two decimal places and comma separators. We could add this to our utility class, but it helps to have it specified in the class. The instructions for how to print a formatted balance are closer in memory as a member function that it would as a non-static member function in another class.

\$1,234.56

```
string TransactionTicket::printTransactionAmount(){
    ostringstream amount;
    amount << "$" << fixed << setprecision(2)
        << transactionAmount;

    string formattedBalance = amount.str();

    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance;
}
```

8.7 Unit Testing

8.7.1 Testing Strategy

We've already stated the importance of performing unit tests on our essential classes. The `TransactionTicket` unit tests are here to verify:

- Correct storage of transaction type
- Accurate account number retrieval
- Proper amount handling
- Correct date storage
- CD term accuracy
- Proper formatting of monetary values

8.7.2 Test Implementation

```
static void testTicket(){
    TransactionTicket transactionTicket =
        TransactionTicket(
            TransactionType::Balance,
            000000,
            100.0,
            Calendar(1,1,2001),
            12
        );

    ASSERT(transactionTicket.getTransactionType() == "Balance");
    ASSERT(transactionTicket.getAccountNumber() == 000000);
    ASSERT(transactionTicket.getTransactionAmount() == 100.0);
    ASSERT(transactionTicket.getTransactionDate().printDate() ==
        "1/1/2001");
    ASSERT(transactionTicket.getCDTerm() == 12);
}
```

8.7.3 Test Runner

Here is the implementation of our voided function to run all tests.

```
void testTransactionTicket(){
    TEST(testTicket);
}
```

8.8 Implementation

8.8.1 TransactionTicket.hpp

The `TransactionTicket.hpp` header file should look like this in its entirety:

```
#pragma once

#include "Stdafx.hpp"

/*
    Transaction Type enum
    Defines all the types of transactions
*/
enum TransactionType {
    Deposit, // Deposit
```

```

Withdrawl, // withdrawl
Balance, // balance
Close, // close account
Open, // open account
BalanceDue, // Balance Due
PayBalance, // Pay Balance
CreditTransaction, // credit Transaction
ClearCheck, // clear check
Portfolio, // Portfolio
TransactionHistory, // transaction History
ClientInfo // client info
};

class TransactionTicket{
    /*
        Transaction Ticket class
        contains an accountNumber, transaction date, transaction type,
        amount of transaction and term of cd
    */
public:
    // Transaction Ticket constructor series
    TransactionTicket(); // No args constructor
    TransactionTicket(
        // Transaction Ticket Keyword args constructor
        TransactionType transactionType, // Transaction type
        int accountNumber, // account Number
        double transactionAmount, // transaction amount
        Calendar transactionDate, // tranaction date
        int cdTerm // cd term
    );
    // TransactionTicket copy constructor
    TransactionTicket(const TransactionTicket& otherTransactionTicket);

    // Transaction ticket getter series
    int getAccountNumber(); // return ticket
    std::string getTransactionType(); // return transaction type
    TransactionType getType(); // returns transaction type enum
    double getTransactionAmount(); // return amount
    Calendar getTransactionDate(); // return date
    int getCDTerm(); // return cd term

    // Transaction Ticket formatter series
    std::string printTransactionAmount();

private:
    int accountNumber; // account number
    TransactionType transactionType; // transaction type
    double transactionAmount; // transaction amount
    Calendar transactionDate; // transaction date
    int cdTerm; // Term of CD
};

// Unit testing
void testTransactionTicket();

```

8.8.2 TransactionTicket.cpp

The complete source file includes constructor definitions, getter implementations, formatting logic, and unit tests should look like this in its entirety:

```
#include "TransactionTicket.hpp"

using namespace std;

// Transaction Ticket constructor series
TransactionTicket::TransactionTicket():
    // transactionTicket no args constructor
    accountNumber(000000), transactionType(TransactionType::Balance),
    transactionAmount(0.0), transactionDate(Calendar()),
    cdTerm(0) {}

TransactionTicket::TransactionTicket(
    // transaction ticket key word args constructor
    TransactionType transactionType, // transaction type
    int accountNumber, // account Number
    double transactionAmount, // transaction amount
    Calendar transactionDate, // transaction date
    int cdTerm // cd term
):
    transactionType (transactionType), // set type
    accountNumber (accountNumber), // set number
    transactionAmount(transactionAmount), // set amount
    transactionDate(transactionDate), // set date
    cdTerm (cdTerm) // set term
{}

TransactionTicket::TransactionTicket(
    // Transaction ticket copy constructor
    const TransactionTicket& otherTransactionTicket // other ticket
):
    transactionType(otherTransactionTicket.transactionType), // set type
    accountNumber(otherTransactionTicket.accountNumber), // set number
    transactionAmount(otherTransactionTicket.transactionAmount), // set amount
    transactionDate(otherTransactionTicket.transactionDate), // set date
    cdTerm (otherTransactionTicket.cdTerm) // set term
{}

// Transaction Ticket getter series
int TransactionTicket::getAccountNumber(){return accountNumber;}
double TransactionTicket::getTransactionAmount(){return transactionAmount;}
Calendar TransactionTicket::getTransactionDate(){return transactionDate;}
int TransactionTicket::getCDTerm(){return cdTerm;}
TransactionType TransactionTicket::getType(){return transactionType;}
string TransactionTicket::getTransactionType(){
    // Get transaction type. Returns a string of the transaction type
    switch(transactionType){
        // Get balance transactions
        case TransactionType::Balance:
            return "Balance"; // return balance
        // Make deposit transactions
        case TransactionType::Deposit:
            return "Deposit"; // return deposit
        // Make With drawls
        case TransactionType::Withdrawl:
```

```

        return "Withdrawl"; // return withdrawl
    // Open account
    case TransactionType::Open:
        return "Re-Open Account"; // return reopen account
    // Close account
    case TransactionType::Close:
        return "Close Account"; // return close account
    // Clear Check
    case TransactionType::ClearCheck:
        return "Clear Check"; // return clear check
    // Get Balance due
    case TransactionType::BalanceDue:
        return "Balance Due"; // return balance due
    // Pay Balance
    case TransactionType::PayBalance:
        return "Make Payment"; // return make payment
    // Credit Transaction
    case TransactionType::CreditTransaction:
        return "Credit Transaction";
    // Return Portfolio
    case TransactionType::Portfolio:
        return "Portolio";
    case TransactionType::TransactionHistory:
        return "Transaction History"; // return transaction history
    case TransactionType::ClientInfo:
        return "Client Information";
    default:
        return " ";
    }
}

// Transaction Ticket formatter series
string TransactionTicket::printTransactionAmount(){
    // Transaction Ticket print transaction method. Prints a fomatted
    // balance
    ostringstream amount; // write string stream
    amount << "$" << fixed << setprecision(2) << transactionAmount;
    // convert to string
    string formattedBalance = amount.str();

    // Insert commas manually
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance; // return balance
}

// Unit Testing
static void testTicket(){
    // test ticket method. Tests the creation of a transaction ticket
    TransactionTicket transactionTicket = TransactionTicket(
        // Initialize transaction ticket
        TransactionType::Balance, // transaction ticket
        000000, // account number
        100.0, // transaction amount
        Calendar(01,01,2001), // transaction date

```



```

        12 // CD term
    );
    // run assertions
    ASSERT(transactionTicket.getTransactionType() == "Balance");
    ASSERT(transactionTicket.getAccountNumber() == 000000);
    ASSERT(transactionTicket.getTransactionAmount() == 100.0);
    ASSERT(transactionTicket.getTransactionDate().printDate == "1/1/2001");
    ASSERT(transactionTicket.getCDTerm() == 12);
}

void testTransactionTicket(){
    // Unit test for transaction ticket class
    TEST(testTicket); // test ticket creation
}

```

8.9 Summary

The `TransactionTicket` class acts as a structured request object for banking operations. By encapsulating transaction metadata and separating it from account logic, it promotes modularity, maintainability, and testability.

Combined with the `TransactionReceipt` class, this design mirrors real-world financial systems where a transaction request produces a verified response. This abstraction improves clarity, traceability, and long-term system scalability.

9 TransactionReceipt Class

9.1 Overview

Next we have the `TransactionReceipt` class. This class represents the result of processing a `TransactionTicket`. While the ticket acts as a transaction request, the receipt acts as the transaction response. Each account has a log of transaction receipts and a client can view there transaction history. Every transaction produces a transaction receipt.

A `TransactionReceipt` contains:

- The original `TransactionTicket`
- Account type
- Pre-transaction balance
- Post-transaction balance
- Success indicator flag
- Reason for failure (if applicable)
- Optional new maturity date (for CD accounts)

This mirrors real-world banking systems where every transaction request generates a formal receipt documenting the outcome.

9.2 Design Rationale

The key ideas here are:

- Separation of concerns between request (`TransactionTicket`) and response (`TransactionReceipt`).
- Strong encapsulation of transaction results.
- Clear success/failure reporting using a boolean flag.
- Support for CD accounts via optional maturity date updates.
- Centralized formatting logic for standardized receipt output.

9.3 Class Declaration

Header Definitions These are the header declarations for the `TransactionReceipt` class. We again have our naming conventions and a function to print the formatted balance. Also notice the `newMaturityDate` variable. Our `TransactionTicket` class had a variable for the new CD term for valid CD account transactions. If the maturity date on a CD accounts passed, then we need to compute a new maturity date for the account. The `TransactionReceipt` also contains the `TransactionTicket` request.

```
#pragma once

#include "Stdafx.hpp"
#include "TransactionTicket.hpp"

class TransactionReceipt{
public:
    TransactionReceipt();
    TransactionReceipt(
        const TransactionTicket& transactionTicket,
        std::string accountType,
        double preTransactionBalance,
        double postTransactionBalance,
        bool transactionSuccess,
        std::string reasonForFailure
    );
    TransactionReceipt(
        const TransactionTicket& transactionTicket,
        std::string accountType,
        double preTransactionBalance,
        double postTransactionBalance,
        bool transactionSuccess,
        std::string reasonForFailure,
        Calendar newMaturityDate
    );
    TransactionReceipt(
        const TransactionReceipt& otherTransactionReceipt
    );

    TransactionTicket getTransactionTicket();
    std::string getAccountType();
    double getPreTransactionBalance();
    double getPostTransactionBalance();
    bool getTransactionSuccess();
    Calendar getMaturityDate();
    std::string getReasonForFailure();

    std::string printReceipt();
    std::string printSuccess();
    std::string printTransactionBalance(double amount);
    std::string printTransactionReceipt();

private:
    TransactionTicket transactionTicket;
    std::string accountType;
    double preTransactionBalance;
    double postTransactionBalance;
    std::string reasonForFailure;
    bool transactionSuccess;
    Calendar newMaturityDate;
```

```
};
```

Unit Testing

```
void testTransactionReceipt();
```

9.4 Constructors

9.4.1 Default Constructor

Initializes a null receipt with default values for reasons that we have explained before. The key thing to note here is that every null transaction receipt is a transaction failure by default.

```
TransactionReceipt::TransactionReceipt():
    transactionTicket(TransactionTicket()),
    accountType(" "),
    preTransactionBalance(0.0),
    postTransactionBalance(0.0),
    transactionSuccess(false),
    reasonForFailure(" ") {}
```

This constructor is useful for temporary placeholders.

9.4.2 Parameterized Constructor (Standard)

Used for most transactions that do not involve CD maturity updates.

```
TransactionReceipt::TransactionReceipt(
    const TransactionTicket& transactionTicket,
    string accountType,
    double preTransactionBalance,
    double postTransactionBalance,
    bool transactionSuccess,
    string reasonForFailure
):
    transactionTicket(transactionTicket),
    accountType(accountType),
    preTransactionBalance(preTransactionBalance),
    postTransactionBalance(postTransactionBalance),
    transactionSuccess(transactionSuccess),
    reasonForFailure(reasonForFailure) {}
```

9.4.3 Parameterized Constructor (CD Version)

Remember, we want to limit the amount of ways that we can construct objects to build confidence. We add one more constructor for a CD Account so that we can construct CD Accounts transaction receipts like any other account constructs transaction receipts.

```
TransactionReceipt::TransactionReceipt(
    const TransactionTicket& transactionTicket,
    string accountType,
    double preTransactionBalance,
    double postTransactionBalance,
    bool transactionSuccess,
    string reasonForFailure,
    Calendar newMaturityDate
):
    transactionTicket(transactionTicket),
    accountType(accountType),
    preTransactionBalance(preTransactionBalance),
```

```

    postTransactionBalance(postTransactionBalance),
    transactionSuccess(transactionSuccess),
    reasonForFailure(reasonForFailure),
    newMaturityDate(newMaturityDate) {}

```

9.4.4 Copy Constructor

This creates a deep copy of an existing receipt for reasons we stated before.

```

TransactionReceipt::TransactionReceipt(
    const TransactionReceipt& otherTransactionReceipt
):
    transactionTicket(otherTransactionReceipt.transactionTicket),
    accountType(otherTransactionReceipt.accountType),
    preTransactionBalance(otherTransactionReceipt.preTransactionBalance),
    postTransactionBalance(otherTransactionReceipt.postTransactionBalance),
    transactionSuccess(otherTransactionReceipt.transactionSuccess),
    reasonForFailure(otherTransactionReceipt.reasonForFailure) {}

```

9.5 Getter Functions

```

TransactionTicket TransactionReceipt::getTransactionTicket();
string TransactionReceipt::getAccountType();
double TransactionReceipt::getPreTransactionBalance();
double TransactionReceipt::getPostTransactionBalance();
bool TransactionReceipt::getTransactionSuccess();
string TransactionReceipt::getReasonForFailure();

```

These functions provide controlled access to receipt data while preserving encapsulation.

9.6 Formatting Functions

9.6.1 printSuccess()

Here we specify what string to return based on transaction success status.

```

string TransactionReceipt::printSuccess(){
    if (transactionSuccess)
        return "Done";
    else
        return "Failed";
}

```

9.6.2 printTransactionBalance(double amount)

We have our monetary format function for values with commas and two decimal places.

\$1,234,567.89

```

string TransactionReceipt::printTransactionBalance(double amount){
    ostringstream transactionBalance;
    transactionBalance << "$" << fixed << setprecision(2) << amount;

    string formattedBalance = transactionBalance.str();

    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance;
}

```

9.6.3 printReceipt()

Receipts will need to be printed several different ways. First, we have this member function that prints a receipt as if it were an account. This is going to be used when we print a receipt for the account transaction history. Generates a formatted row suitable for transaction history logs.

It includes:

- Transaction date
- Transaction type
- Amount
- Status
- Post-transaction balance
- Failure reason (if applicable)

```
string TransactionReceipt::printReceipt(){
    // Transaction Receipt print receipt method. prints
    // a formatted string of the receipt
    ostream receipt; // write string stream
    receipt << left << setw(13)
        << transactionTicket.getTransactionDate().printDate()
        << setw(20) << transactionTicket.getTransactionType()
        << setw(12)
        << printTransactionBalance(transactionTicket.getTransactionAmount())
        << setw(10) << printSuccess()
        << setw(10) << printTransactionBalance(postTransactionBalance);
    if (transactionTicket.getType() != TransactionType::Portfolio ||
        transactionTicket.getType() != TransactionType::TransactionHistory ||
        transactionTicket.getType() != TransactionType::ClientInfo){
        receipt << setw(20) << reasonForFailure;
    }

    return receipt.str(); // return receipt
}
```

9.6.4 printTransactionReceipt()

Next we have a print method that prints a transaction ticket after every transaction. This is analogous to the transaction receipts that come out of an ATM each time we withdraw or deposit money. This member function generates a detailed human-readable receipt based on the transaction type.

Examples include:

- Deposit summary (old balance, deposit amount, new balance)
- Withdrawal summary
- Balance inquiry
- Credit payments
- Account open/close confirmations

If the transaction fails, the receipt clearly displays:

- Pre-transaction balance
- Attempted transaction amount
- Reason for failure

This ensures transparency and traceability in the event of failure. These error messages come directly from the exception classes that we implemented earlier.

```
string TransactionReceipt::printTransactionReceipt(){
    // Transaction Receipt print transaction receipt. Prints a receipt for a transaction
    ostream transactionReceipt;
    if (transactionSuccess){
        switch(transactionTicket.getType()){
            case TransactionType::Balance:
                transactionReceipt << "Current Balance: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::Deposit:
                transactionReceipt << "Account Balance: "
                    << printTransactionBalance(
                        preTransactionBalance
                    )
                    << "\t\t| Amount to Deposit: "
                    << printTransactionBalance(
                        transactionTicket.getTransactionAmount()
                    )
                    << "\nNew Balance: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::Withdrawl:
                transactionReceipt << "Account Balance: "
                    << printTransactionBalance(
                        preTransactionBalance
                    )
                    << "\t\t| Amount to Withdraw: "
                    << printTransactionBalance(
                        transactionTicket.getTransactionAmount()
                    )
                    << "\nNew Balance: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::Open:
                transactionReceipt << "Account Open";
                break;
            case TransactionType::Close:
                transactionReceipt << "Account Closed";
                break;
            case TransactionType::BalanceDue:
                transactionReceipt << "Balance Due: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::ClearCheck:
                transactionReceipt << "Account Balance: "
                    << printTransactionBalance(
                        preTransactionBalance // pre balance
                    )
                break;
        }
    }
}
```

```

        << "\t\t| Amount of Check: "
        << printTransactionBalance(
            transactionTicket.getTransactionAmount()
        )
        << "\nNew Balance: "
        << printTransactionBalance(
            postTransactionBalance // post balance
        );

        break;
    case TransactionType::CreditTransaction:
        transactionReceipt << "Credit Amount: "
        << printTransactionBalance(
            transactionTicket.getTransactionAmount()
        )
        << "\t\tBalance Due: "
        << printTransactionBalance(
            postTransactionBalance
        );

        break;
    case TransactionType::PayBalance:
        transactionReceipt << "Balance Due: "
        << printTransactionBalance(
            preTransactionBalance
        )
        << "\t\tNew Balance: "
        << printTransactionBalance(
            postTransactionBalance
        );

        break;
    case TransactionType::Portfolio:
        // print portfolio
        transactionReceipt << reasonForFailure;
        break;
    case TransactionType::TransactionHistory:
        // Print receipts
        transactionReceipt << reasonForFailure;
        break;
    case TransactionType::ClientInfo:
        transactionReceipt << "\n" + reasonForFailure;
        break;
    }
} else{
    // write reason for failure
    transactionReceipt << "Current Balance: "
        << printTransactionBalance(
            // pre balance
            preTransactionBalance
        )
        << "\t\t\tTransaction Amount: "
        << printTransactionBalance(
            // Transaction Amount
            transactionTicket.getTransactionAmount()
        ) << "\n";
    transactionReceipt << reasonForFailure;
}

// Return receipt string

```

```
    return transactionReceipt.str();
}
```

9.7 Unit Testing

9.7.1 Testing Strategy

The TransactionReceipt unit tests are here to verify:

- Proper storage of transaction ticket
- Correct account type storage
- Accurate pre and post balances
- Success flag correctness
- Failure reason storage

9.7.2 Test Implementation

```
static void testReceipt(){
    TransactionTicket transactionTicket = TransactionTicket();

    TransactionReceipt transactionReceipt =
        TransactionReceipt(
            transactionTicket,
            "Account Type",
            0.0,
            0.0,
            true,
            " "
        );

    ASSERT(transactionReceipt.getAccountType() == "Account Type");
    ASSERT(transactionReceipt.getPreTransactionBalance() == 0.0);
    ASSERT(transactionReceipt.getPostTransactionBalance() == 0.0);
    ASSERT(transactionReceipt.getReasonForFailure() == " ");
}
```

9.7.3 Test Runner

```
void testTransactionReceipt(){
    TEST(testReceipt);
}
```

9.8 Implementation

9.8.1 TransactionReceipt.hpp

The entire transaction receipt class should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "TransactionTicket.hpp"

class TransactionReceipt{
    /*
        Transaction Receipt Class
        contains a transaction ticket, success Indicator flag
    */
};
```



```

        reason for failure, account type, preTransaction Balance,
        post transaction balance, post transaction maturity date
    */
public:
    // Transaction Receipt constructor series
    TransactionReceipt(); // No args constructor
    TransactionReceipt(
        // Transaction Receipt Key work args constructor
        const TransactionTicket& transactionTicket,
        std::string accountType, // account type
        double preTransactionBalance, // pre transaction balance
        double postTransactionBalance, // post Transaction balance
        bool transactionSuccess, // transaction success indicator
        std::string reasonForFailure // reason for failure
    );
    TransactionReceipt(
        // Transaction Receipt Key work args constructor
        const TransactionTicket& transactionTicket,
        std::string accountType, // account type
        double preTransactionBalance, // pre transaction balance
        double postTransactionBalance, // post Transaction balance
        bool transactionSuccess, // transaction success indicator
        std::string reasonForFailure, // reason for failure
        Calendar newMaturityDate // new maturity date
    );
    TransactionReceipt(
        // Transaction Receipt copy constructor
        const TransactionReceipt& otherTransactionReceipt
    );

    // Transaction Receipt getter series
    TransactionTicket getTransactionTicket(); // return ticket
    std::string getAccountType(); // return account type
    double getPreTransactionBalance(); // return pre balance
    double getPostTransactionBalance(); // return post balance
    bool getTransactionSuccess(); // return transaction success
    Calendar getMaturityDate(); // return maturity date
    std::string getReasonForFailure(); // return failure reason

    // Transaction Receipt Formatter
    std::string printReceipt(); // prints the receipt
    std::string printSuccess(); // prints success
    std::string printTransactionBalance(double amount); // print balance
    std::string printTransactionReceipt();

private:
    TransactionTicket transactionTicket; // transaction ticket
    std::string accountType; // account type
    double preTransactionBalance; // pre transaction balance
    double postTransactionBalance; // post transaction balance
    std::string reasonForFailure; // reason for failure
    bool transactionSuccess; // success indicator flag
    Calendar newMaturityDate; // new Maturity date
};

// Unit testing
void testTransactionReceipt();

```

9.8.2 TransactionReceipt.cpp

The entire transaction receipt source file should look like this:

```
#include "TransactionReceipt.hpp"

using namespace std;

// Transaction receipt constructor series
TransactionReceipt::TransactionReceipt():
    // Transaction Receipt no args constructor
    transactionTicket(TransactionTicket()), // transaction ticket
    accountType(" "), // account type
    preTransactionBalance(0.0), // pre transaction balance
    postTransactionBalance(0.0), // post transaction balance
    transactionSuccess(false), // transaction success
    reasonForFailure(" ") // reason for failure
{}

TransactionReceipt::TransactionReceipt(
    // Transaction Receipt key word args constructor
    const TransactionTicket& transactionTicket, // transaction ticket
    string accountType, // account type
    double preTransactionBalance, // pre transaction balance
    double postTransactionBalance, // post transaction balance
    bool transactionSuccess, // success indicator
    std::string reasonForFailure // reason for failure
) :
    transactionTicket(transactionTicket), // set transaction ticket
    accountType(accountType), // set account type
    preTransactionBalance(preTransactionBalance), // set pre balance
    postTransactionBalance(postTransactionBalance), // set post balance
    transactionSuccess(transactionSuccess), // set success
    reasonForFailure(reasonForFailure) // set reason for failure
{}

TransactionReceipt::TransactionReceipt(
    // Transaction Receipt key word args constructor
    const TransactionTicket& transactionTicket, // transaction ticket
    string accountType, // account type
    double preTransactionBalance, // pre transaction balance
    double postTransactionBalance, // post transaction balance
    bool transactionSuccess, // success indicator
    std::string reasonForFailure, // reason for failure
    Calendar newMaturityDate // new maturity date
) :
    transactionTicket(transactionTicket), // set transaction ticket
    accountType(accountType), // set account type
    preTransactionBalance(preTransactionBalance), // set pre balance
    postTransactionBalance(postTransactionBalance), // set post balance
    transactionSuccess(transactionSuccess), // set success
    reasonForFailure(reasonForFailure), // set reason for failure
    newMaturityDate(newMaturityDate) // new maturity date
{}

TransactionReceipt::TransactionReceipt(
    const TransactionReceipt& otherTransactionReceipt
):
```

```

        transactionTicket(otherTransactionReceipt.transactionTicket), // copy ticket
        accountType(otherTransactionReceipt.accountType), // copy type
        preTransactionBalance(otherTransactionReceipt.preTransactionBalance), // pre
        postTransactionBalance(otherTransactionReceipt.postTransactionBalance), //post
        transactionSuccess(otherTransactionReceipt.transactionSuccess), // success
        reasonForFailure(otherTransactionReceipt.reasonForFailure) // failure reason
    {}

// Transaction Receipt getter Series
TransactionTicket TransactionReceipt::getTransactionTicket(){
    // Transaction receipt get transaction ticket. return transaction ticket
    return TransactionTicket(transactionTicket); // return ticket
}

string TransactionReceipt::getAccountType(){
    // Transaction Receipt get account type. return account type
    return accountType; // return account type
}

double TransactionReceipt::getPreTransactionBalance(){
    // get transaction balance. return pre transaction balance
    return preTransactionBalance; // return pre balance
}

double TransactionReceipt::getPostTransactionBalance(){
    // transaction receipt post transaction balance. return transaction
    // balance
    return postTransactionBalance; // return post balance
}

bool TransactionReceipt::getTransactionSuccess(){
    // transaction receipt get transaction success
    return transactionSuccess; // return success
}

string TransactionReceipt::getReasonForFailure(){
    // transaction receipt get reason for failure. returns reason
    // failure
    return reasonForFailure; // return failure reason
}

// Formatter series
string TransactionReceipt::printSuccess(){
    // Transaction Receipt print success method. returns done or
    // failed
    if (transactionSuccess){
        // if transaction successful
        return "Done"; // return done
    } else {
        // if transaction not successful
        return "Failed";
    }
}

string TransactionReceipt::printReceipt(){
    // Transaction Receipt print receipt method. prints
    // a formatted string of the receipt
    ostringstream receipt; // write string stream

```

```

    receipt << left << setw(13)
        << transactionTicket.getTransactionDate().printDate()
        << setw(20) << transactionTicket.getTransactionType()
        << setw(12) << printTransactionBalance(transactionTicket.getTransactionAmount())
        << setw(10) << printSuccess()
        << setw(10) << printTransactionBalance(postTransactionBalance);
    if (transactionTicket.getType() != TransactionType::Portfolio ||
        transactionTicket.getType() != TransactionType::TransactionHistory ||
        transactionTicket.getType() != TransactionType::ClientInfo){
        receipt << setw(20) << reasonForFailure;
    }

    return receipt.str(); // return receipt
}

string TransactionReceipt::printTransactionBalance(double amount){
    // Bank account print account balance method. Prints a fomatted
    // balance
    ostringstream transactionBalance; // write string stream
    transactionBalance << "$" << fixed << setprecision(2) << amount;
    // convert to string
    string formattedBalance = transactionBalance.str();

    // Insert commas manually
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance; // return balance
}

string TransactionReceipt::printTransactionReceipt(){
    // Transaction Receipt print transaction receipt. Prints a receipt for a transaction
    ostringstream transactionReceipt;
    if (transactionSuccess){
        switch(transactionTicket.getType()){
            case TransactionType::Balance:
                transactionReceipt << "Current Balance: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::Deposit:
                transactionReceipt << "Account Balance: "
                    << printTransactionBalance(
                        preTransactionBalance
                    )
                    << "\t\t| Amount to Deposit: "
                    << printTransactionBalance(
                        transactionTicket.getTransactionAmount()
                    )
                    << "\nNew Balance: "
                    << printTransactionBalance(
                        postTransactionBalance
                    );
                break;
            case TransactionType::Withdrawl:

```

```

        transactionReceipt << "Account Balance: "
        << printTransactionBalance(
            preTransactionBalance
        )
        << "\t\t| Amount to Withdraw: "
        << printTransactionBalance(
            transactionTicket.getTransactionAmount()
        )
        << "\nNew Balance: "
        << printTransactionBalance(
            postTransactionBalance
        );

        break;
    case TransactionType::Open:
        transactionReceipt << "Account Open";
        break;
    case TransactionType::Close:
        transactionReceipt << "Account Closed";
        break;
    case TransactionType::BalanceDue:
        transactionReceipt << "Balance Due: "
        << printTransactionBalance(
            postTransactionBalance
        );

        break;
    case TransactionType::ClearCheck:
        transactionReceipt << "Account Balance: "
        << printTransactionBalance(
            preTransactionBalance // pre balance
        )
        << "\t\t| Amount of Check: "
        << printTransactionBalance(
            transactionTicket.getTransactionAmount()
        )
        << "\nNew Balance: "
        << printTransactionBalance(
            postTransactionBalance // post balance
        );

        break;
    case TransactionType::CreditTransaction:
        transactionReceipt << "Credit Amount: "
        << printTransactionBalance(
            transactionTicket.getTransactionAmount()
        )
        << "\t\tBalance Due: "
        << printTransactionBalance(
            postTransactionBalance
        );

        break;
    case TransactionType::PayBalance:
        transactionReceipt << "Balance Due: "
        << printTransactionBalance(
            preTransactionBalance
        )
        << "\t\tNew Balance: "
        << printTransactionBalance(
            postTransactionBalance
        );

```

```

        break;
    case TransactionType::Portfolio:
        // print portfolio
        transactionReceipt << reasonForFailure;
        break;
    case TransactionType::TransactionHistory:
        // Print receipts
        transactionReceipt << reasonForFailure;
        break;
    case TransactionType::ClientInfo:
        transactionReceipt << "\n" + reasonForFailure;
        break;
    }
} else{
    // write reason for failure
    transactionReceipt << "Current Balance: "
        << printTransactionBalance(
            // pre balance
            preTransactionBalance
        )
        << "\t\t\tTransaction Amount: "
        << printTransactionBalance(
            // Transaction Amount
            transactionTicket.getTransactionAmount()
        ) << "\n";
    transactionReceipt << reasonForFailure;
}

// Return receipt string
return transactionReceipt.str();
}

static void testReceipt(){
    // initialize null transaction ticket
    TransactionTicket transactionTicket = TransactionTicket();
    // Initialize transaction receipt
    TransactionReceipt transactionReceipt = TransactionReceipt(
        transactionTicket, // null ticket
        "Account Type", // account type
        0.0, // pre balance
        0.0, // post balance
        true, // success
        " " // no reason
    );
    // Run Assertions
    ASSERT(transactionReceipt.getTransactionTicket() != nullptr);
    ASSERT(transactionReceipt.getAccountType() == "Account Type");
    ASSERT(transactionReceipt.getPreTransactionBalance() == 0.0);
    ASSERT(transactionReceipt.getPostTransactionBalance() == 0.0);
    ASSERT(transactionReceipt.getReasonForFailure() == " ");
}

void testTransactionReceipt(){
    // Unit test for transaction receipt class
    TEST(testReceipt); // test receipt
}

```

9.9 Summary

The `TransactionReceipt` class finalizes the transaction lifecycle. It transforms a request into a documented outcome.

By encapsulating transaction metadata, balance changes, and status information, it provides:

- Accountability
- Auditability
- Clear error reporting
- Standardized financial formatting

Together, `TransactionTicket` and `TransactionReceipt` form a clean and scalable transaction-processing architecture that closely models real-world banking systems.

10 CreditCard Class

10.1 Overview

Next we have the `CreditCard` class. This class represents a credit card used to perform transactions on a `CreditAccount`. All credit transaction actions occur using a credit card from the `CreditCard` class. A client can use their credit cards to make a credit transaction that will increase their balance due. They can also perform a transaction to pay their balance. Finally, they can view their transaction history.

A credit card contains:

- Card number (linked to account number)
- Expiration date
- Card level (spending tier)
- Credit charge amount

The card level determines the maximum spending limit allowed for transactions.

10.2 CardLevel Enumeration

We have another enumeration for the credit card level. This enumeration is here to help with the context switching on card type. Certainly, we could specify exactly what that level is with an integer, but we will see here why using an enumeration is beneficial.

```
enum class CardLevel : int {
    BASIC = 500,
    GOLD = 2000,
    PLATINUM = 5000
};
```

10.2.1 Purpose

The `CardLevel` enum assigns predefined credit limits. We could extend this enumeration to include multiple tiers of card level. As stated before that would be unadvisable due to the warnings and unpredictable behaviour if not implemented correctly. Again, know all your elements before developing enumerations.

- BASIC → \$500 limit
- GOLD → \$2000 limit
- PLATINUM → \$5000 limit

Stated more clearly using an enum ensures:

- Strong type safety
- Clear tier definitions
- Easy limit retrieval via casting

10.3 Class Declaration

This is the class declaration for the `CreditCard` class. Notice that we have a method for printing a formatted balance for the reasons stated before.

```
class CreditCard{
public:
    CreditCard();
    CreditCard(
        int cardNumber,
        Calendar expirationDate,
        CardLevel limit,
        double creditCharge
    );

    int getCardNumber();
    Calendar getExpirationDate();
    double getCardLimit();
    std::string getCardLevel();
    double getCreditCharge();

    std::string printCreditChargeAmount();

private:
    int cardNumber;
    Calendar expirationDate;
    CardLevel limit;
    double creditCharge;
};
```

10.4 Constructors

10.4.1 Default Constructor

Here we initialize a basic credit card with zero charge. Every null credit card is of type basic and the expiration date is the date at the time of initialization.

```
CreditCard::CreditCard()
: cardNumber(11111),
  expirationDate(Calendar()),
  limit(CardLevel::BASIC),
  creditCharge(0.0) {}
```

10.4.2 Parameterized Constructor

Here we initialize a credit card with specific values.

```
CreditCard::CreditCard(
    int accountNumber,
    Calendar expirationDate,
    CardLevel limit,
    double creditCharge
):
    cardNumber(accountNumber),
    expirationDate(expirationDate),
    limit(limit),
    creditCharge(creditCharge) {}
```


10.5 Getter Functions

These are the `CreditCard` class getter functions.

```
int CreditCard::getCardNumber();
Calendar CreditCard::getExpirationDate();
double CreditCard::getCardLimit();
std::string CreditCard::getCardLevel();
double CreditCard::getCreditCharge();
```

10.5.1 getCardLimit()

This member function returns the numeric limit associated with the card level. Notice it has to be casted.

```
return static_cast<int>(limit);
```

10.5.2 getCardLevel()

This method returns a string representation of the tier. Recall, when we said that there where follow on actions needed for handling enumerations types, well here is yet another example. Getting the card limit returns the value of the enumeration but not the label of the enumeration.

```
"Basic"
"Gold"
"Platinum"
```

10.6 Formatting Function

10.6.1 printCreditChargeAmount()

Here is our formatting function for the charge amount:

\$1,234.56

This is very similar to our other formatted print methods. Recall that the compiler can fetch the instructions for printing a formatted balance number faster if the instructions are a part of the class and not in some other class. Here are the implementation details:

- Uses `ostringstream`
- Applies fixed two-decimal precision
- Inserts commas manually

```
string CreditCard::printCreditChargeAmount(){
    ostringstream charge;
    charge << "$" << fixed << setprecision(2) << creditCharge;

    string formattedBalance = charge.str();

    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance;
}
```

10.7 Design Rationale

The purposes of the `CreditCard` class are outlined here:

- Encapsulates card metadata separately from the account
- Enforces spending tiers via `CardLevel`
- Provides formatted output for financial clarity
- Integrates directly with `CreditAccount`

By separating card logic from account logic, the system achieves:

- Better modularity
- Clear separation of responsibilities
- Reusability across account types

10.8 Unit Testing

10.8.1 Testing Strategy

The purpose of the `CreditCard` class unit tests are here to verify:

- Proper constructor initialization
- Correct expiration date storage
- Correct tier-to-limit mapping
- Correct charge amount storage
- Proper string formatting

10.8.2 Test Implementation

```
static void testCard(){
    CreditCard testCard(
        111111,
        Calendar(2,1,2026),
        CardLevel::BASIC,
        200.00
    );

    ASSERT(testCard.getCardNumber() == 111111);
    ASSERT(testCard.getCardLevel() == "Basic");
    ASSERT(testCard.getCardLimit() == 500);
    ASSERT(testCard.getCreditCharge() == 200.00);
}
```

10.8.3 Test Runner

```
void testCreditCard(){
    TEST(testCard);
}
```

10.9 Implementation

10.9.1 CreditCard.hpp

The entire CreditCard.hpp file should look like this:

```
#pragma once

#include "Stdafx.hpp"

/*
    Card Level Enum
    Determines the card limit on a credit car
*/
enum class CardLevel : int {
    BASIC = 500, // 500.00
    GOLD = 2000, // 2000.00
    PLATINUM = 5000 // 5000.00
};

/*
    Credit Card class
    Used to perform credit transactions on credit accounts.
    contains an account number, date of expiration, and card limit.
*/
class CreditCard{
public:
    /* Credit Card constructor series */
    CreditCard(); // no args constructor
    CreditCard(
        // Credit card keyword args constructor
        int cardNumber, // account number
        Calendar expirationDate, // expiration date
        CardLevel limit, // card limit
        double creditCharge // credit charge amount
    );

    /* Credit card getter series */
    int getCardNumber(); // returns account number
    Calendar getExpirationDate(); // return expiration date
    double getCardLimit(); // returns card limit
    std::string getCardLevel(); // get card level
    double getCreditCharge(); // get credit charge

    /* Credit Card Formatter Series */
    std::string printCreditChargeAmount();

private:
    int cardNumber; // account number
    Calendar expirationDate; // expiration date
    CardLevel limit; // card limit
    double creditCharge; // credit charge
};

// Unit test for dredit card class
void testCreditCard();
```

10.9.2 CreditCard.cpp

The entire source file should look like this:

```

#include "CreditCard.hpp"

using namespace std;

/* Credit Card Constructor series */
CreditCard::CreditCard()
    // Credit card no args constructor
    : cardNumber(11111), // account number
      expirationDate(Calendar()), // expiration date
      limit(CardLevel::BASIC), // card limit
      creditCharge(0.0) // credit charge
{}

CreditCard::CreditCard(
    int accountNumber, // account number
    Calendar expirationDate, // expiration date
    CardLevel limit, // limit
    double creditCharge // charge amount
) : // Set variables
    cardNumber(accountNumber), // account number
    expirationDate(expirationDate), // expiration date
    limit(limit), // card limit
    creditCharge(creditCharge) // credit charge
{}

/* Credit Card getter Series */
int CreditCard::getCardNumber() {
    // Credit card class get account number. returns account
    // number
    return cardNumber; // return card number
}

Calendar CreditCard::getExpirationDate() {
    // credit card class get expiration date
    return Calendar(expirationDate); // return expiration date
}

double CreditCard::getCardLimit() {
    // credit card get card limit. returns card limit
    return static_cast<int>(limit); // return card limit
}

double CreditCard::getCreditCharge(){
    // Credit card get credit charge method. returns that amount charged
    return creditCharge; // return credit charge
}

string CreditCard::printCreditChargeAmount(){
    // Credit card print charge amount. prints a formatted string of the
    // charge amount
    ostringstream charge; // write string stream
    charge << "$" << fixed << setprecision(2) << creditCharge;
    // convert to string
    string formattedBalance = charge.str();

    // Insert commas manually
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){

```

```

        formattedBalance.insert(i, ",");
    }

    return formattedBalance; // return balance
}

string CreditCard::getCardLevel(){
    // Credit card get card level. Returns card level
    switch (limit) {
        // Basic = $500
        case CardLevel::BASIC:
            return "Basic"; // return basic
        // Gold = $ 2000
        case CardLevel::GOLD:
            return "Gold"; // return gold
        // Platinum = $5000
        case CardLevel::PLATINUM:
            return "Platinum"; // return platinum
        default:
            return "Unknown"; // error handling
    }
}

/* Credit Card unit testing series */
static void testCard(){
    CreditCard testCard(
        111111, // account number
        Calendar(2,1,2026), // 2/2006
        CardLevel::BASIC, // cardlevel
        200.00 // charge amount
    );
    // Assert
    ASSERT(testCard.getAccountNumber() == 111111);
    ASSERT(testCard.getExpirationDate().getMonth() == 1);
    ASSERT(testCard.getExpirationDate().getDay() == 31);
    ASSERT(testCard.getExpirationDate().getYear() == 2026);
    ASSERT(testCard.getCardLevel() == "Basic");
    ASSERT(testCard.getCardLimit() == 500);
    ASSERT(testCard.getChargeAmount() == 200.00);
}

void testCreditCard(){
    // Unit test for credit card class
    TEST(testCard);
}

```

10.10 Summary

Transaction requests will still be completed with transaction tickets, but to perform a transaction on a credit account the client must have a credit card. The **CreditCard** class provides:

- Card-level tier enforcement
- Expiration tracking
- Charge encapsulation
- Currency formatting support

Together with `CreditAccount`, this class forms the transactional backbone of the credit subsystem within the banking architecture.

11 Check Class

11.1 Overview

This header file contains the `Check` class, which represents a bank check transaction. A check is a financial instrument tied to a specific account and written for a specific monetary amount on a specific date. Rather than passing these three pieces of information independently throughout the system, we encapsulate them inside a single `Check` object.

This approach improves readability, maintainability, and reduces the likelihood of mismatched parameters when processing transactions.

11.2 Design Rationale

The `Check` class is intentionally minimal and focused. Its purpose is to represent a single check transaction in a clear and controlled way.

- Encapsulation ensures that account number, amount, and date are accessed only through defined member functions.
- Constructors guarantee that a `Check` object is always initialized into a valid state.
- A copy constructor ensures safe duplication of check objects without exposing internal data.
- Getter-only access prevents unintended mutation of financial data.

Financial data should not be arbitrarily mutable. For that reason, this class does not provide setter functions.

11.3 Class Declaration

Below is the header declaration of the `Check` class.

Header Definitions

```
(Check.hpp)
#pragma once

#include "Stdafx.hpp"

class Check{
public:
    Check();
    Check(int accountNumber,
          double checkAmount,
          Calendar dateOfCheck);
    Check(const Check& otherCheck);

    int getAccountNumber();
    double getCheckAmount();
    Calendar getDateOfCheck();

private:
    int accountNumber;
    double checkAmount;
    Calendar dateOfCheck;
};
```

Unit Testing

```
// Unit testing
void testCheck();
```

Notice again the consistent naming convention. Getter methods follow the pattern `get<DataMemberName>()`. This eliminates ambiguity when reading or writing code that uses this class.

11.4 Constructors

Constructors ensure that every `Check` object is created in a valid and predictable state.

11.4.1 Default Constructor

The default constructor initializes all data members to safe values.

```
Check::Check()
: accountNumber(000000),
  checkAmount(0.0),
  dateOfCheck(Calendar()) {}
```

- `accountNumber` is set to 000000
- `checkAmount` is set to 0.0
- `dateOfCheck` is initialized using the default `Calendar()` constructor

This ensures that even an empty check object contains defined values, preventing undefined behavior. We have discussed why this is essential in several sections.

11.4.2 Parameterized Constructor

This constructor allows full initialization at the time of object creation.

```
Check::Check(int accountNumber,
             double checkAmount,
             Calendar dateOfCheck)
: accountNumber(accountNumber),
  checkAmount(checkAmount),
  dateOfCheck(dateOfCheck) {}
```

- `accountNumber` — The account tied to the check
- `checkAmount` — The monetary value written
- `dateOfCheck` — The date the check was issued

Using an initializer list ensures efficient and direct member initialization.

11.4.3 Copy Constructor

The copy constructor creates a duplicate of an existing `Check` object.

```
Check::Check(const Check& otherCheck)
: accountNumber(otherCheck.accountNumber),
  checkAmount(otherCheck.checkAmount),
  dateOfCheck(otherCheck.dateOfCheck) {}
```

This provides safe copying semantics and ensures each object maintains its own independent state.

11.5 Member Functions

The `Check` class provides getter functions for controlled access to its private data members.

11.5.1 getAccountNumber()

Returns the account number associated with the check.

```
int Check::getAccountNumber() {  
    return accountNumber;  
}
```

11.5.2 getCheckAmount()

Returns the monetary value written on the check.

```
double Check::getCheckAmount() {  
    return checkAmount;  
}
```

11.5.3 getDateOfCheck()

Returns the `Calendar` object representing the check date.

```
Calendar Check::getDateOfCheck() {  
    return dateOfCheck;  
}
```

Returning by value protects the internal state of the object.

11.6 Unit Testing

Here are the Unit tests to ensure the reliability and correctness of the `Check` class.

11.6.1 Testing Strategy

The unit test are designed to confirm that the following behaviors are verified:

- Correct storage of account number
- Correct storage of check amount
- Correct storage of check date (month, day, year)

11.6.2 Test Implementation

Test: Check Creation

```
static void testchk(){  
    Check testCheck(123456, 100.0,  
                    Calendar(01,04,2025));  
  
    ASSERT(testCheck.getAccountNumber() == 123456);  
    ASSERT(testCheck.getCheckAmount() == 100.0);  
    ASSERT(testCheck.getDateOfCheck().getMonth() == 01);  
    ASSERT(testCheck.getDateOfCheck().getDay() == 04);  
    ASSERT(testCheck.getDateOfCheck().getYear() == 2025);  
}
```

11.6.3 Test Runner

```
void testCheck(){  
    TEST(testchk);  
}
```

This test runner executes all unit tests related to the `Check` class in a single call.

11.7 Implementation

11.7.1 Check.hpp

The complete header file should appear as:

```
#pragma once

#include "Stdafx.hpp"

class Check{
public:
    Check();
    Check(int accountNumber,
          double checkAmount,
          Calendar dateOfCheck);
    Check(const Check& otherCheck);

    int getAccountNumber();
    double getCheckAmount();
    Calendar getDateOfCheck();

private:
    int accountNumber;
    double checkAmount;
    Calendar dateOfCheck;
};

void testCheck();
```

11.7.2 Check.cpp

The full implementation file should appear as:

```
#include "Check.hpp"

using namespace std;

Check::Check()
    : accountNumber(000000),
      checkAmount(0.0),
      dateOfCheck(Calendar()) {}

Check::Check(int accountNumber,
             double checkAmount,
             Calendar dateOfCheck)
    : accountNumber(accountNumber),
      checkAmount(checkAmount),
      dateOfCheck(dateOfCheck) {}

Check::Check(const Check& otherCheck)
    : accountNumber(otherCheck.accountNumber),
      checkAmount(otherCheck.checkAmount),
      dateOfCheck(otherCheck.dateOfCheck) {}

int Check::getAccountNumber(){return accountNumber;}
double Check::getCheckAmount(){return checkAmount;}
Calendar Check::getDateOfCheck(){return dateOfCheck;}

static void testchk(){
```

```

    Check testCheck(123456, 100.0,
                    Calendar(01,04,2025));

    ASSERT(testCheck.getAccountNumber() == 123456);
    ASSERT(testCheck.getCheckAmount() == 100.0);
    ASSERT(testCheck.getDateOfCheck().getMonth() == 01);
    ASSERT(testCheck.getDateOfCheck().getDay() == 04);
    ASSERT(testCheck.getDateOfCheck().getYear() == 2025);
}

void testCheck(){
    TEST(testchk);
}

```

11.8 Summary

The **Check** class provides a clean and focused abstraction for representing check transactions within the banking system. Its design emphasizes encapsulation, safe initialization, and controlled data access. With unit testing integrated, it ensures reliability and correctness as part of the broader financial application.

12 Investment Class

12.1 Overview

This header file defines the **Investment** interface. The purpose of this class is to describe the required behavior of all investment objects within the financial system.

Rather than implementing a specific investment type (such as stock, bond, or mutual fund), this class defines a contract. Any class that inherits from **Investment** must provide implementations for:

- Retrieving the number of shares
- Calculating the total value
- Printing a formatted representation of the investment

This design enforces polymorphism and ensures consistent behavior across all investment types.

12.2 Design Rationale

The **Investment** class is designed as an abstract base class. It is an interface for the different types of investment instruments that a Brokerage Account can purchase.

- It uses pure virtual functions to define required behavior.
- It includes a virtual destructor to ensure proper cleanup when derived objects are deleted through base-class pointers.
- It provides a shared formatting utility function to avoid duplicated balance-formatting logic in derived classes.

This approach ensures extensibility. New investment types can be added without modifying existing system logic. We can include swaps, futures, and forwards as long as their logic implements this class.

12.3 Class Declaration

Header Definitions

```
(Investment.hpp)
#pragma once

#include "Stdafx.hpp"

class Investment {
public:
    virtual ~Investment() = default;

    virtual int getNumberOfShares() const = 0;
    virtual double getTotalValue() const = 0;
    virtual std::string print() const = 0;

    std::string printFormattedBalance(double amount) const;
};
```

12.4 Destructor

12.4.1 virtual ~Investment() = default;

The destructor is declared virtual to ensure proper destruction of derived classes when an **Investment** pointer references a derived object.

This is critical for preventing memory leaks in polymorphic systems.

12.5 Pure Virtual Functions

These functions define the required interface for all derived investment types.

12.5.1 getNumberOfShares()

```
virtual int getNumberOfShares() const = 0;
```

Returns the number of shares associated with the investment.

- Must be implemented by derived classes
- Marked **const** to guarantee no modification of state

12.5.2 getTotalValue()

```
virtual double getTotalValue() const = 0;
```

Returns the computed total value of the investment.

Typically calculated as:

$$\text{numberOfShares} * \text{pricePerShare}$$

12.5.3 print()

```
virtual std::string print() const = 0;
```

Returns a formatted string describing the investment. Each derived class determines how the investment should be displayed.

12.6 Shared Utility Function

12.6.1 printFormattedBalance(double amount)

This function provides shared formatting logic for monetary values. We already know why we have it.

```

string Investment::printFormattedBalance(double amount) const {
    ostringstream value;
    value << "$" << fixed << setprecision(2) << amount;

    string formattedBalance = value.str();

    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }
    return formattedBalance;
}

```

Function Behavior

- Formats the number to two decimal places
- Prefixes the amount with a dollar sign
- Inserts comma separators for thousands
- Returns the formatted monetary string

Example Output If `amount = 12345.5`, the function returns:

\$12,345.50

Providing this function in the base class prevents repetition across derived investment types, so we do not need to implement a formatted print method in the individual investment instrument classes.

12.7 Implementation

12.7.1 Investment.hpp

The entire header file for `Investment` class should look like this:

```

#pragma once

#include "Stdafx.hpp"

/*
    Investment Interface
    describes the behavior of investment objects
    has a number of shares and total value
*/
class Investment {
public:
    // Investment Class destructor
    virtual ~Investment() = default;

    // Returns the number of shares
    virtual int getNumberOfShares() const = 0;
    // returns the total value
    virtual double getTotalValue() const = 0;

    // Formatted print
    virtual std::string print() const = 0;

    /* Stock formatter series */
    std::string printFormattedBalance(double amount) const;
};

```

12.7.2 Investment.cpp

The implementation file contains only the formatting function:

```
#include "Investment.hpp"

using namespace std;

string Investment::printFormattedBalance(double amount) const {
    ostringstream value;
    value << "$" << fixed << setprecision(2) << amount;

    string formattedBalance = value.str();

    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }
    return formattedBalance;
}
```

12.8 Summary

The **Investment** class defines a strict behavioral contract for all investment types in the banking system. By combining pure virtual functions with a shared formatting utility, it enforces polymorphism, promotes code reuse, and ensures consistent financial representation across the application.

This interface serves as the foundation for extensible and scalable investment modeling within the system.

13 Bond Class

13.1 Overview

This header file defines the **Bond** class, which represents a bond investment within the financial system. This is the first of our investment instruments. The **Bond** class inherits from the **Investment** interface and provides concrete implementations for all required investment behaviors.

A bond contains:

- Quantity (number of bonds owned)
- Face value (value per bond)
- Coupon rate (annual interest rate)
- Maturity date

Because **Bond** inherits from **Investment**, it must implement all pure virtual functions defined in the base class.

13.2 Design Rationale

The **Bond** class demonstrates polymorphism through inheritance.

- It implements all required investment interface functions.
- It calculates total value and annual yield using financial formulas.
- It computes maturity date dynamically based on the current date.
- It uses the shared **printFormattedBalance()** utility from the base class to ensure consistent monetary formatting.

This design ensures that all bond objects behave uniformly while still allowing them to be treated generically as **Investment** objects.

13.3 Class Declaration

Header Definitions

```
(Bond.hpp)
#pragma once

#include "Investment.hpp"

class Bond : public Investment {
public:
    Bond();
    Bond(int quantity,
         double faceValue,
         double couponRate,
         int yearsToMaturity);
    Bond(const Bond& otherBond);

    int getNumberOfShares() const override;
    double getTotalValue() const override;
    double getAnnualYield() const;
    Calendar getMaturityDate() const;

    std::string print() const override;

private:
    int quantity;
    double faceValue;
    double couponRate;
    Calendar maturityDate;
};
```

Unit Testing

```
void testBond();
```

13.4 Constructors

13.4.1 Default Constructor

Initializes all values to safe defaults.

```
Bond::Bond()
: quantity(0),
  faceValue(0.0),
  couponRate(0.0),
  maturityDate(Calendar()) {}
```

- Quantity = 0
- Face value = 0.0
- Coupon rate = 0.0
- Maturity date = default `Calendar()`

This guarantees a valid object state.

13.4.2 Parameterized Constructor

Allows full initialization and calculates maturity date dynamically. When we initialize a bond it takes the present date and adds the years to maturity using the overloaded + operator.

```
Bond::Bond(int quantity,
           double faceValue,
           double couponRate,
           int yearsToMaturity)
: quantity(quantity),
  faceValue(faceValue),
  couponRate(couponRate) {

    Calendar today = Calendar();
    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + yearsToMaturity
    );
}
```

- Maturity date is computed as current year + yearsToMaturity
- Month and day remain the same as today

This ensures future maturity calculation is automatic.

13.4.3 Copy Constructor

We have our copy constructor to create a deep copy of an existing Bond object.

```
Bond::Bond(const Bond& otherBond)
: quantity(otherBond.quantity),
  faceValue(otherBond.faceValue),
  couponRate(otherBond.couponRate),
  maturityDate(otherBond.maturityDate) {}
```

Each bond maintains independent data after copying.

13.5 Investment Interface Implementation

13.5.1 getNumberOfShares()

Returns the total quantity of bonds owned.

```
int Bond::getNumberOfShares() const {
    return quantity;
}
```

13.5.2 getTotalValue()

Returns total bond value.

```
double Bond::getTotalValue() const {
    return quantity * faceValue;
}
```

13.5.3 getAnnualYield()

Returns annual interest yield.

```
double Bond::getAnnualYield() const {
    return quantity * faceValue * couponRate;
}
```

This follows the financial formula:

$$\text{Annual Yield} = \text{Quantity} \times \text{Face Value} \times \text{Coupon Rate}$$

13.5.4 getMaturityDate()

Returns the bond's maturity date, a `Calendar` object.

```
Calendar Bond::getMaturityDate() const {
    return maturityDate;
}
```

13.6 Formatted Output

13.6.1 print()

Here is our Bond class print method. It produces a formatted string describing the bond.

```
string Bond::print() const {
    stringstream bondString;

    bondString << "Bond\n"
               << "Quantity: " << quantity
               << "\nFace Value: "
               << printFormattedBalance(faceValue)
               << "\nCoupon: "
               << (couponRate * 100) << "%"
               << "\nMaturity Date: "
               << getMaturityDate().printDate()
               << "\tTotal Value: "
               << printFormattedBalance(getTotalValue())
               << "\n";

    return bondString.str();
}
```

Example Output Here is an example output. Notice that the maturity date is sometime in the future.

```
Bond
Quantity: 1
Face Value: $100.00
Coupon: 9%
Maturity Date: 04/01/2028
Total Value: $100.00
```

13.7 Unit Testing

13.7.1 Testing Strategy

We have out unit tests to verify:

- Correct maturity year calculation

- Correct quantity storage
- Correct annual yield calculation
- Correct total value calculation

13.7.2 Test Implementation

```
static void testBondInvestment(){
    Bond testBond(1, 100, .09, 2);

    ASSERT(testBond.getMaturityDate().getYear() == 2028);
    ASSERT(testBond.getNumberOfShares() == 1);

    ASSERT_APPROX_EQUAL(
        testBond.getAnnualYield(),
        9,
        0.001
    );

    ASSERT_APPROX_EQUAL(
        testBond.getTotalValue(),
        100,
        0.001
    );
}
```

13.7.3 Test Runner

```
void testBond(){
    TEST(testBondInvestment);
}
```

13.8 Implementation

13.8.1 Bond.hpp

This is the complete header file for the Bond class:

```
#pragma once

#include "Investment.hpp"

class Bond : public Investment {
public:
    Bond();
    Bond(int quantity,
        double faceValue,
        double couponRate,
        int yearsToMaturity);
    Bond(const Bond& otherBond);

    int getNumberOfShares() const override;
    double getTotalValue() const override;
    double getAnnualYield() const;
    Calendar getMaturityDate() const;

    std::string print() const override;

private:
```

```
    int quantity;
    double faceValue;
    double couponRate;
    Calendar maturityDate;
};
```

```
void testBond();
```

13.8.2 Bond.cpp

Here is the complete implementation of the `Bond.cpp` file:

```
#include "Bond.hpp"

using namespace std;

Bond::Bond()
    : quantity(0),
      faceValue(0.0),
      couponRate(0.0),
      maturityDate(Calendar()) {}

Bond::Bond(int quantity,
           double faceValue,
           double couponRate,
           int yearsToMaturity)
    : quantity(quantity),
      faceValue(faceValue),
      couponRate(couponRate) {

    Calendar today = Calendar();
    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + yearsToMaturity
    );
}

Bond::Bond(const Bond& otherBond)
    : quantity(otherBond.quantity),
      faceValue(otherBond.faceValue),
      couponRate(otherBond.couponRate),
      maturityDate(otherBond.maturityDate) {}

int Bond::getNumberOfShares() const {
    return quantity;
}

double Bond::getTotalValue() const {
    return quantity * faceValue;
}

double Bond::getAnnualYield() const {
    return quantity * faceValue * couponRate;
}

Calendar Bond::getMaturityDate() const {
    return maturityDate;
}
```

```

string Bond::print() const {
    stringstream bondString;

    bondString << "Bond\n"
        << "Quantity: " << quantity
        << "\nFace Value: "
        << printFormattedBalance(faceValue)
        << "\nCoupon: "
        << (couponRate * 100) << "%"
        << "\nMaturity Date: "
        << getMaturityDate().printDate()
        << "\tTotal Value: "
        << printFormattedBalance(getTotalValue())
        << "\n";

    return bondString.str();
}

static void testBondInvestment(){
    Bond testBond(1, 100, .09, 2);

    ASSERT(testBond.getMaturityDate().getYear() == 2028);
    ASSERT(testBond.getNumberOfShares() == 1);

    ASSERT_APPROX_EQUAL(
        testBond.getAnnualYield(),
        9,
        0.001
    );

    ASSERT_APPROX_EQUAL(
        testBond.getTotalValue(),
        100,
        0.001
    );
}

void testBond(){
    TEST(testBondInvestment);
}

```

13.9 Summary

The `Bond` class is a concrete implementation of the `Investment` interface. It encapsulates bond-specific financial data while adhering to a polymorphic investment contract.

Its design emphasizes:

- Proper initialization
- Financial calculation accuracy
- Code reuse through inheritance
- Clean formatted output
- Thorough unit testing

This makes the `Bond` class a reliable and extensible component within the broader investment system.

14 Option Class

14.1 Overview

Next we have the `Option` class. This is the second of our three investment instruments. This class represents a financial derivative contract and extends the `Investment` base class. While other investments such as stocks or bonds represent direct ownership or debt instruments, an option represents the right to buy or sell an underlying asset.

An `Option` contains:

- Option type (`Call` or `Put`)
- Number of contracts
- Premium (price per share)
- Contract size (shares per contract)

This mirrors real-world financial markets where each option contract controls a fixed number of shares and has a defined premium cost.

14.2 Design Rationale

The key ideas here are:

- Inheritance from `Investment` to enable polymorphic behavior.
- Use of a strongly-typed `enum class` for option type safety.
- Clear mathematical modeling of contract value.
- Separation of share calculation from total value calculation.
- Centralized formatting logic through the `print()` function.

This design allows options to integrate seamlessly with other investment types while preserving their unique financial behavior.

14.3 Class Declaration

Header Definitions

```
#pragma once

#include "Investment.hpp"

enum class OptionType {
    Call,
    Put
};

class Option: public Investment {
public:
    Option();
    Option(
        OptionType optionType,
        int contracts,
        double premium,
        int contractSize
    );
    Option(const Option& otherOption);

    int getNumberOfShares() const override;
```

```

        double getTotalValue() const override;
        OptionType getOptionType() const;

        std::string print() const override;

    private:
        OptionType optionType;
        int contract;
        double premium;
        int contractSize;
};

```

Unit Testing

```
void testOption();
```

14.4 Constructors

14.4.1 Default Constructor

Initializes an empty option position.

```

Option::Option():
    optionType(OptionType::Call),
    contract(0),
    premium(0.0),
    contractSize(0) {}

```

This constructor is useful for temporary placeholders or default initialization before assignment.

14.4.2 Parameterized Constructor

This constructor is used to create a fully defined option position.

```

Option::Option(
    OptionType optionType,
    int contracts,
    double premium,
    int contractSize
):
    optionType(optionType),
    contract(contracts),
    premium(premium),
    contractSize(contractSize) {}

```

This constructor initializes all member variables using an initializer list, which is both efficient and idiomatic C++.

14.4.3 Copy Constructor

We can create a deep copy of an existing option.

```

Option::Option(const Option& otherOption):
    optionType(otherOption.optionType),
    contract(otherOption.contract),
    premium(otherOption.premium),
    contractSize(otherOption.contractSize) {}

```

This ensures that duplicated option objects maintain identical state.

14.5 Core Investment Functions

14.5.1 getNumberOfShares()

Returns the total number of underlying shares controlled by the option.

```
int Option::getNumberOfShares() const {
    return contract * contractSize;
}
```

Mathematically:

$$TotalShares = Contracts \times ContractSize$$

14.5.2 getTotalValue()

Returns the total premium paid for all contracts.

```
double Option::getTotalValue() const {
    return contract * premium * contractSize;
}
```

Mathematically:

$$TotalValue = Contracts \times Premium \times ContractSize$$

14.5.3 getOptionType()

Returns the enum representing the option type.

```
OptionType Option::getOptionType() const {
    return optionType;
}
```

This preserves strong type safety rather than returning a string.

14.6 Formatting Function

14.6.1 print()

Generates a formatted string representation of the option.

```
string Option::print() const{
    stringstream optionString;

    optionString << "Option: "
        << (optionType == OptionType::Call ? "Call" : "Put")
        << "\nContracts: " << contract
        << "\nPremium: " << printFormattedBalance(premium)
        << "\t\tTotal Value: "
        << printFormattedBalance(getTotalValue())
        << "\n";

    return optionString.str();
}
```

The output includes:

- Option type (Call or Put)
- Number of contracts
- Premium (formatted currency)
- Total contract value (formatted currency)

This ensures standardized financial formatting across all investment types.

Example Output Here is an example output for put options.

```
Option: Put
Contracts: 2
Premium: $13.50 Total Value: $2,700.48
```

or for call options:

```
Option: Call
Contracts: 19
Premium: $8.43 Total Value: $16,024.88
```

14.7 Unit Testing

14.7.1 Testing Strategy

We have our unit tests to verify:

- Proper storage of option type
- Correct share calculation
- Accurate total value computation

14.7.2 Test Implementation

```
static void testOptionInvestment(){
    Option testOption = Option(
        OptionType::Call,
        1,
        100.00,
        10
    );

    ASSERT(testOption.getOptionType() == OptionType::Call);

    ASSERT_APPROX_EQUAL(
        testOption.getNumberOfShares(),
        10,
        0.001
    );

    ASSERT_APPROX_EQUAL(
        testOption.getTotalValue(),
        1000,
        0.001
    );
}
```

14.7.3 Test Runner

```
void testOption(){
    TEST(testOptionInvestment);
}
```

14.8 Implementation

14.8.1 Option.hpp

The entire header file should look like this:

```

#pragma once

#include "Investment.hpp"

/*
    Option Type enum
    options can be either put or call in this app
*/
enum class OptionType {
    Call, // Call Option
    Put // Put option
};

/*
    Option Class
    extends investment. contains a type enum,
    number of contracts, premium, and contract size
*/
class Option: public Investment {
public:
    // Option Class Constructor Series
    Option(); // no args constructor
    Option(
        OptionType optionType, // option type
        int contracts, // contracts
        double premium, // premium
        int contractSize // contract size
    );
    // Copy constructor
    Option(const Option& otherOption);

    // Option required methods
    int getNumberOfShares() const override;
    double getTotalValue() const override;
    OptionType getOptionType() const;

    // Option Print method
    std::string print() const override;

private:
    OptionType optionType; // option type
    int contract; // number of contract
    double premium; // cost
    int contractSize; // contract size
};

// Unit test
void testOption();

```

14.8.2 Option.cpp

The entire source file should look like this:

```

#include "Option.hpp"

using namespace std;

/* Option class constructor series */
Option::Option():

```



```

    // Option no args constructor
    optionType(OptionType::Call), // call
    contract(0), // contracts
    premium(0.0), // premium
    contractSize(0) // contract size
{}

Option::Option(
    // Option Key word args constructor
    OptionType optionType, // option type
    int contracts, // contracts
    double premium, // premium
    int contractSize // size
): optionType(optionType),
    contract(contracts),
    premium(premium), // premium
    contractSize(contractSize)
{}

Option::Option(const Option& otherOption):
    // Option Copy constructor
    optionType(otherOption.optionType), // copy type
    contract(otherOption.contract), // copy contract
    premium(otherOption.premium), // copy premium
    contractSize(otherOption.contractSize) // copy size
{}

// Option class investment functions
int Option::getNumberOfShares() const {
    // Option get number of shares method. returns the number of
    // shares
    return contract * contractSize; // num of shares
}

double Option::getTotalValue() const {
    // Option class get total value method. returns the total
    // value of the contract
    return contract * premium * contractSize;
}

OptionType Option::getOptionType() const{
    // Option class get option type method. returns the option type
    return optionType; // return type
}

// Option formatted
string Option::print() const{
    // Option class print method. Prints the option
    stringstream optionString; // intalize string
    // Build String
    optionString << "Option: "
        << (optionType == OptionType::Call ? "Call" : "Put")
        << "\nContracts: " << contract // contract
        << "\nPremium: " << printFormattedBalance(
            premium // price per share
        ) << "\t\tTotal Value: " << printFormattedBalance(
            getTotalValue()
        ) << "\n"; // total value
}

```

```

    return optionString.str(); // return string
}

/* Unit Testing Series */
static void testOptionInvestment(){
    // Unit test for option creation
    Option testOption = Option(
        OptionType::Call,
        1, // contracts
        100.00, // premium
        10 // size
    );
    // Assert
    ASSERT(testOption.getOptionType() == "Call");
    ASSERT_APPROX_EQUAL(
        // test number of shares
        testOption.getNumberOfShares(),
        1 * 10,
        0.001
    );
    ASSERT_APPROX_EQUAL(
        // Test total value
        testOption.getTotalValue(),
        1 * 100 * 10,
        0.001
    );
}

void testOption(){
    // Unit test for the option class
    TEST(testOptionInvestment);
}

```

14.9 Summary

The `Option` class models real-world derivative contracts within the investment framework. By encapsulating contract count, premium pricing, and share calculation, it provides:

- Accurate financial modeling
- Clean polymorphic integration
- Strong type safety through enums
- Standardized output formatting

Together with other derived investment classes, `Option` contributes to a scalable and extensible portfolio management architecture.

15 Stock Class

15.1 Overview

Next we have the `Stock` class, the last of our three investment. This class represents an equity investment and extends the `Investment` base class. Unlike bonds or options, a stock represents ownership in a company through shares.

A `Stock` contains:

- Ticker symbol

- Number of shares owned
- Price per share

This mirrors real-world equity markets where investors purchase shares of publicly traded companies identified by ticker symbols.

15.2 Design Rationale

The key ideas here are:

- Inheritance from `Investment` to enable polymorphism.
- Clear separation between share count and price per share.
- Direct computation of total market value.
- Encapsulation of ticker identity within the class.
- Centralized formatted output for portfolio reporting.

This design allows stocks to integrate seamlessly with other investment types such as bonds and options while preserving their unique properties.

15.3 Class Declaration

Header Definitions

```
#pragma once

#include "Investment.hpp"

class Stock : public Investment {
public:
    Stock();
    Stock(
        std::string tickerSymbol,
        int shares,
        double pricePerShare
    );
    Stock(const Stock& otherStock);

    int getNumberOfShares() const override;
    double getTotalValue() const override;
    std::string getTickerSymbol() const;

    virtual std::string print() const override;

private:
    std::string tickerSymbol;
    int shares;
    double pricePerShare;
};
```

Unit Testing

```
void testStock();
```

15.4 Constructors

15.4.1 Default Constructor

Initializes an empty stock position. Key things to not here is that every null stock does not have a ticker symbols.

```
Stock::Stock():  
    tickerSymbol("#-"),  
    shares(0),  
    pricePerShare(0.0) {}
```

This constructor is useful for placeholder objects before assignment.

15.4.2 Parameterized Constructor

Used to create a fully defined stock position.

```
Stock::Stock(  
    string tickerSymbol,  
    int shares,  
    double pricePerShare  
) :  
    tickerSymbol(tickerSymbol),  
    shares(shares),  
    pricePerShare(pricePerShare) {}
```

This constructor initializes all member variables using an initializer list for efficiency and clarity.

15.4.3 Copy Constructor

Creates a deep copy of an existing stock object.

```
Stock::Stock(const Stock& otherStock):  
    tickerSymbol(otherStock.tickerSymbol),  
    shares(otherStock.shares),  
    pricePerShare(otherStock.pricePerShare) {}
```

This ensures duplicated objects maintain identical state.

15.5 Core Investment Functions

15.5.1 getNumberOfShares()

Returns the total number of shares owned.

```
int Stock::getNumberOfShares() const {  
    return shares;  
}
```

15.5.2 getTotalValue()

Returns the total market value of the stock position.

```
double Stock::getTotalValue() const {  
    return shares * pricePerShare;  
}
```

Mathematically:

$$TotalValue = Shares \times PricePerShare$$

15.5.3 getTickerSymbol()

Returns the stock's ticker symbol.

```
string Stock::getTickerSymbol() const {  
    return tickerSymbol;  
}
```

This allows controlled access while preserving encapsulation.

15.6 Formatting Function

15.6.1 print()

Generates a formatted string representation of the stock position.

```
string Stock::print() const{  
    ostringstream stockString;  
  
    stockString << "Ticker: #" << tickerSymbol << "\n"  
                << "Shares: " << shares  
                << "\nPrice: "  
                << printFormattedBalance(pricePerShare)  
                << "\t\t\tTotal Value: "  
                << printFormattedBalance(getTotalValue())  
                << "\n";  
  
    return stockString.str();  
}
```

The output includes:

- Ticker symbol
- Number of shares
- Price per share (formatted currency)
- Total market value (formatted currency)

This ensures consistent formatting across all investment types.

Example Output Here is a sample output for a `Stock` class print that we might see.

```
Ticker: #JPM  
Shares: 432  
Price: $428.55    Total Value: $185,133.32
```

15.7 Unit Testing

15.7.1 Testing Strategy

We have out unit tests to verify:

- Proper storage of ticker symbol
- Correct share count
- Accurate total value calculation

15.7.2 Test Implementation

```
static void testStockInvestment(){
    Stock testStock(
        "AAPL",
        10,
        100.0
    );

    ASSERT(testStock.getTickerSymbol() == "AAPL");
    ASSERT(testStock.getNumberOfShares() == 10);

    ASSERT_APPROX_EQUAL(
        testStock.getTotalValue(),
        1000.0,
        0.001
    );
}
```

15.7.3 Test Runner

```
void testStock(){
    TEST(testStockInvestment);
}
```

15.8 Implementation

15.8.1 Stock.hpp

The header file should appear as followed:

```
#pragma once

#include "Investment.hpp"

/*
    Stock Class
    extends investment contains a ticker smbolic, shares,
    and price per share
*/
class Stock : public Investment {
public:
    // Stock constructor series
    Stock(); // no args constructor
    Stock(
        // Stock key work args constructor
        std::string ticketSymbol, // ticket
        int shares, // shares
        double pricePerShare // price per share
    );
    Stock(const Stock& otherStock);

    // Stock Investment getter series
    int getNumberOfShares() const override; // number of shares
    double getTotalValue() const override; // total value
    std::string getTickerSymbol() const; // ticker

    /* Printer series */
    virtual std::string print() const override;
```

```

    private:
        std::string tickerSymbol; // ticker symbol
        int shares; // shares
        double pricePerShare; // price pershare
};

// Unit test
void testStock();

```

15.8.2 Stock.cpp

The entire source file should appear as followed:

```

#include "Stock.hpp"

using namespace std;

Stock::Stock():
    // Stock no args constructor
    tickerSymbol("#-"), shares(0), pricePerShare(0.0){}

Stock::Stock(
    // Stock key word args constructor
    string tickerSymbol, // ticket symbol
    int shares, // shares
    double pricePerShare // price per share
) : tickerSymbol(tickerSymbol), // set symbol
    shares(shares), // set shares
    pricePerShare(pricePerShare) // set price per share
{}

Stock::Stock(const Stock& otherStock):
    // Stock copy constructor
    tickerSymbol(otherStock.tickerSymbol), // copy ticker
    shares(otherStock.shares), // copy shares
    pricePerShare(otherStock.pricePerShare) // copy price per share
{}

/* Stock getter series */
int Stock::getNumberOfShares() const {
    // Get number of shares returns the number of shares
    return shares; // return shares
}

double Stock::getTotalValue() const {
    // Get Total value. returns the # of shares * price
    return shares * pricePerShare; // total value
}

string Stock::getTickerSymbol() const {
    // Get ticker symbol. return the ticker
    return tickerSymbol; // return symbol
}

string Stock::print() const{
    ostringstream stockString; // intalize string
    // Build String
    stockString << "Ticker: #" << tickerSymbol << "\n" // Ticker
        << "Shares: " << shares // shares

```

```

        << "\nPrice: " << printFormattedBalance(
            pricePerShare
        ) // price per share
        << "\t\t\tTotal Value: " << printFormattedBalance(
            // Formated balance
            getTotalValue()
        ) << "\n"; // total value
    return stockString.str(); // return string
}

/* Unit Testing Series */
static void testStockInvestment(){
    // unit test for stock class
    Stock testStock(
        // test stock
        "AAPL", // ticker
        10, // number of shares
        100.0 // price per share
    );
    // ASSERT
    ASSERT(testStock.getTickerSymbol() == "AAPL");
    ASSERT(testStock.getNumberOfShares() == 10);
    ASSERT_APPROX_EQUAL(
        testStock.getTotalValue(), // 10 * 1000
        1000.0, // 100
        0.001
    );
}

void testStock(){
    // Unit test for stock class
    TEST(testStockInvestment);
}

```

15.9 Summary

The **Stock** class models equity ownership within the investment framework. By encapsulating ticker identity, share quantity, and pricing logic, it provides:

- Accurate valuation of stock positions
- Clean polymorphic integration with other investments
- Standardized currency formatting
- Clear and maintainable object design

Together with **Bond** and **Option**, the **Stock** class completes a flexible and extensible portfolio management architecture.

16 CreditAccount Class

16.1 Overview

Next we have the **CreditAccount** class. This class models a credit-based financial account. Unlike a traditional bank account, a credit account represents money owed rather than money owned.

Purchases increase the balance due, while payments decrease it. The class also maintains full transaction history using **TransactionReceipt** objects.

A **CreditAccount** contains:

- A Depositor (creditor)
- Account number
- Account type
- Balance due
- Account status (open/closed)
- Interest rate
- Transaction receipt history

16.2 Design Rationale

The design of this class emphasizes:

- Reverse accounting logic (debt-based model)
- Strict validation for payments and spending limits
- Lifecycle management (open/close account)
- Automatic receipt generation for every transaction
- Encapsulation of account state

The class mirrors real-world credit card systems and integrates directly with `CreditCard`, `TransactionTicket`, and `TransactionReceipt`.

16.3 Class Declaration

Here is the class declaration for the `CreditAccount` class:

```
class CreditAccount{
public:
    CreditAccount();
    CreditAccount(
        Depositor creditor,
        int accountNumber,
        double balanceDue,
        bool accountStatus
    );
    CreditAccount(BankAccount bankAccount);
    CreditAccount(const CreditAccount& otherCreditAccount);

    Depositor getDepositor();
    int getAccountNumber();
    std::string getAccountType();
    AccountType getType();
    double getBalanceDue();
    bool getAccountStatus();
    std::vector<TransactionReceipt> getTransactionReceipts();

    std::string printAccount();
    std::string printAccountBalance();
    std::string printAccountStatus();

    TransactionReceipt getBalanceDue(CreditCard creditCard);
    TransactionReceipt payBalanceDue(CreditCard creditCard);
    TransactionReceipt creditTransaction(CreditCard creditCard);
    TransactionReceipt reOpenAccount(TransactionTicket transactionTicket);
```

```

    TransactionReceipt closeAccount(TransactionTicket transactionTicket);

    void addTransactionReceipt(TransactionReceipt transactionReceipt);

private:
    Depositor creditor;
    int accountNumber;
    AccountType accountType;
    double balanceDue;
    bool accountStatus;
    double interestRate = 0.07;
    std::vector<TransactionReceipt> transactionReceipts;
};

```

A key thing to not here that in our bank simulation we charge a 7% interest rate on all credit transactions for simplicity.

16.4 Constructors

16.4.1 Default Constructor

Initializes a new open credit account with zero balance due. We will define what an `AccountType` is later.

```

CreditAccount::CreditAccount():
    creditor(Depositor()),
    accountNumber(111111),
    accountType(AccountType::Credit),
    balanceDue(0.0),
    accountStatus(true) {}

```

16.4.2 Parameterized Constructor

Creates a credit account with user-defined values.

```

CreditAccount::CreditAccount(
    Depositor creditor,
    int accountNumber,
    double balanceDue,
    bool accountStatus
):
    creditor(creditor),
    accountNumber(accountNumber),
    accountType(AccountType::Credit),
    balanceDue(balanceDue),
    accountStatus(accountStatus) {}

```

16.4.3 BankAccount Conversion Constructor

Constructs a credit account from an existing bank account. This gives us the ability to convert an initialized `BankAccount` into a `CreditAccount`.

```

CreditAccount::CreditAccount(BankAccount bankAccount)
    : creditor(bankAccount.getDepositor()),
      accountNumber(bankAccount.getAccountNumber()),
      accountType(AccountType::Credit),
      balanceDue(0.0),
      accountStatus(true) {}

```

16.4.4 Copy Constructor

Creates a deep copy of another credit account including receipts.

```
CreditAccount::CreditAccount(
    const CreditAccount& otherCreditAccount
):
    creditor(otherCreditAccount.creditor),
    accountNumber(otherCreditAccount.accountNumber),
    accountType(otherCreditAccount.accountType),
    balanceDue(otherCreditAccount.balanceDue),
    accountStatus(otherCreditAccount.accountStatus),
    transactionReceipts(otherCreditAccount.transactionReceipts)
{}
```

16.5 Getter Functions

Here are the `CreditAccount` class getter functions.

```
Depositor CreditAccount::getDepositor();
int CreditAccount::getAccountNumber();
string CreditAccount::getAccountType();
AccountType CreditAccount::getType();
double CreditAccount::getBalanceDue();
bool CreditAccount::getAccountStatus();
vector<TransactionReceipt> CreditAccount::getTransactionReceipts();
```

These functions provide controlled access to internal account data.

16.6 Formatting Functions

16.6.1 printAccount()

Returns a formatted string containing:

- Depositor name
- SSN
- Account number
- Account type
- Account status
- Balance due

16.6.2 printAccountBalance()

Formats the balance due with currency formatting:

\$1,234.56

Includes manual comma insertion and two decimal precision.

16.6.3 printAccountStatus()

This function returns:

```
"Open"    // if accountStatus == true
"Closed"  // otherwise
```

16.7 Transaction Operations

Each transaction generates and logs a `TransactionReceipt`.

16.7.1 `getBalanceDue()`

returns current balance due without modifying account state.

`TransactionType::BalanceDue`

16.7.2 `payBalanceDue()`

As stated before a client with `CreditAccount` can pay down their balance. Whenever a balance is paid an interest rate is charged to the amount. Reduces balance due using the formula:

$$paymentAmount = creditCharge - (interestRate \times balanceDue)$$

Throws:

- `InvalidAmountException` if payment < 0

16.7.3 `creditTransaction()`

Increases balance due for purchases.

Validation rules:

- Amount must be positive
- Cannot exceed card limit

Throws:

- `InvalidAmountException`
- `SpendingLimitException`

16.7.4 `reOpenAccount()`

Reopens a closed account.

Throws:

- `AccountOpenException` if already open

16.7.5 `closeAccount()`

Closes account only if balance due equals zero.

Throws:

- `AccountClosedException`
- `InvalidAccountException` if balance due $\neq$ 0

16.8 Transaction Logging

All successful and unsuccessful transactions are stored:

```
transactionReceipts.emplace_back(transactionReceipt);
```

This enables full transaction history retrieval.

16.9 Unit Testing

16.9.1 Testing Strategy

We have our unit tests to verify:

- Proper construction
- Balance updates
- Spending limits
- Account lifecycle changes
- Receipt storage

16.9.2 Test Runner

```
void testCreditAccount(){
    TEST(testCreditAcct);
    TEST(testGetBalanceDue);
    TEST(testCreditAccountGetTransactionReceipts);
    TEST(testPayBalanceDue);
    TEST(testCreditTransaction);
    TEST(testCreditAccountReOpenAccount);
    TEST(testCreditAccountCloseAccount);
}
```

16.10 Implementation

16.10.1 CreditAccount.hpp

The entire header file should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"
#include "Check.hpp"

/*
    Checking account class
    contains a clear check method. Clients can make deposits and
    withdrawals when they want to. Charges an over draft fee if account
    is over drawn
*/
class CheckingAccount: public BankAccount {
public:
    // Checking Account constructor series
    CheckingAccount(); // no args constructor
    CheckingAccount(
        // Checking account key word args constructor
        Depositor depositor, // depositor
        int accountNumber, // account number
        double accountBalance, // account balance
        bool accountStatus // account Status
    );
    CheckingAccount(const CheckingAccount& otherCheckingAccount);

    // Checking Account Transaction Series
    TransactionReceipt clearCheck(Check checkToClear);
};
```

```
// Unit Test
void testCheckingAccount();
```

16.10.2 CreditAccount.cpp

The entire source file should appear as this:

```
#include "CheckingAccount.hpp"

using namespace std;

/* Checking account class constructor series */
CheckingAccount::CheckingAccount(): BankAccount(
    // Checking Account no args constructor
    Depositor(), // null depositor
    000000, // account number
    AccountType::Checking, // checking account
    0.0, // balance
    true // status
){}

CheckingAccount::CheckingAccount(
    // Checking account key word args constructor
    Depositor depositor, // depositor
    int accountNumber, // account number
    double accountBalance, // account balance
    bool accountStatus // account status
) : BankAccount(
    // Bank Account constructor
    depositor, // set depositor
    accountNumber, // set account number
    AccountType::Checking, // set checking
    accountBalance, // set account balance
    accountStatus // set account Status
){}

CheckingAccount::CheckingAccount(
    // Checking account copy constructor
    const CheckingAccount& otherCheckingAccount
) : BankAccount(otherCheckingAccount){}

/* Checking Account Transaction Series */
TransactionReceipt CheckingAccount::clearCheck(Check checkToClear){
    // Checking Account Clear Check Method. Withdraws from an account
    double preTransactionBalance = accountBalance; // copy balance
    // Get six months ago
    Calendar today = Calendar(); // today's date
    int presentMonth = today.getMonth();
    int presentYear = today.getYear();

    presentMonth -= 6; // subtract 6
    if(presentMonth <= 0) {
        presentMonth += 12; // add 12 to month
        presentYear -= 1; // subtract 1 from year
    }

    Calendar sixMonthsAgo = Calendar(
        // Six months ago object
```

```

        presentMonth, // month
        today.getDay(), // same day
        presentYear // year
    );

    if (checkToClear.getDateOfCheck().before(sixMonthsAgo)){
        // if older than 6 months ago
        throw CheckTooOldException(
            checkToClear.getDateOfCheck().printDate()
        );
    } else if (checkToClear.getCheckAmount() > accountBalance){
        accountBalance -= 2.50; // apply fee
        // Throw exception
        throw InsufficientFundsException(to_string(preTransactionBalance));
    } else if (checkToClear.getDateOfCheck().after(today)) {
        // if check too old
        throw PostDatedCheckException();
    } else {
        // Valid check clear
        accountBalance -= checkToClear.getCheckAmount();
        // Generate transaction Receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            TransactionTicket( // transaction ticket
                TransactionType::ClearCheck, // transaction type
                checkToClear.getAccountNumber(), // account number
                checkToClear.getCheckAmount(), // transaction amount
                today, // time of transaction
                0 // no term
            ),
            getAccountType(), // account type
            preTransactionBalance, // pre balance
            accountBalance, // post balance
            true, // success
            " " // no failure
        );
        // append receipt and return
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    }
}

/* Unit Testing Series */
static void testCheckingAcct(){
    // Unit test for checking account creation
    CheckingAccount testCheckingAccount = CheckingAccount(
        // test checking account
        Depositor(), // null depositor
        000000, // account number
        100.00, // account balance
        true // account open
    );
    // Assert
    ASSERT(testCheckingAccount.getDepositor() != nullptr);
    ASSERT(testCheckingAccount.getAccountNumber() == 000000);
    ASSERT(testCheckingAccount.getAccountType() == "Checking");
    ASSERT(testCheckingAccount.getAccountBalance() == 100.0);
    ASSERT(testCheckingAccount.getAccountStatus() == true);
}

```

```

static void testClearCheck(){
    // Unit test for clear check
    Check testCheck = Check(
        // Initialize test check
        000000, // account number
        50.00, // amount
        Calendar() // time now
    );
    // Initialize test Checking account
    CheckingAccount testCheckingAccount = CheckingAccount(
        Depositor(), // null depositor
        testCheck.getAccountNumber(), // account number
        100.00, // balance
        true // open
    );
    // Generate Transaction Receipt
    TransactionReceipt transactionReceipt = testCheckingAccount.clearCheck(
        testCheck // test check
    );
    // Assert
    ASSERT_APPROX_EQUAL(
        transactionReceipt.getPostTransactionBalance(), // transaction
        50.00, // expected
        0.001 // tolerance
    );
}

void testCheckingAccount(){
    // Unit test for checking account class
    TEST(testCheckingAcct);
    TEST(testClearCheck);
}

```

16.11 Summary

The `CreditAccount` class models a realistic credit card system within the banking architecture. It provides:

- Debt-based accounting logic
- Spending limit enforcement
- Interest-aware payments
- Account lifecycle management
- Complete transaction logging

Together with `CreditCard`, `TransactionTicket`, and `TransactionReceipt`, it completes the credit processing layer of the system.

17 BankAccount Class

17.1 Overview

Next we have the `BankAccount` class. This class serves as the core foundation for all account types in the banking system. It models a real-world bank account by storing customer information, account metadata, financial state, and transaction history.

A `BankAccount` contains:

- A `Depositor` (client name and SSN)
- Account number
- Account type (Checking, Savings, CD, etc.)
- Account balance
- Account status (Open or Closed)
- A record of `TransactionReceipt` objects

This mirrors real-world banking systems where each account maintains both financial state and a permanent transaction history.

17.2 Design Rationale

The key ideas here are:

- Encapsulation of account state and behavior.
- Use of an `AccountType` enum for strong type safety.
- Separation between transaction request (`TransactionTicket`) and transaction result (`TransactionReceipt`).
- Automatic transaction logging after every operation.
- Support for inheritance through virtual transaction methods.

This design allows specialized account types (such as CD or Credit) to extend the base functionality while preserving core account behavior.

17.3 AccountType Enumeration

```
enum class AccountType {  
    Checking,  
    Savings,  
    CD,  
    Commercial,  
    Brokerage,  
    Credit  
};
```

This enum ensures only valid account categories can be assigned.

17.4 Class Declaration

Header Definitions Here we have the header definitions for the `BankAccount`. Several things to note here are the virtual methods and non virtual methods.

```
class BankAccount{  
public:  
    BankAccount();  
    BankAccount(  
        Depositor depositor,  
        int accountNumber,  
        AccountType accountType,  
        double accountBalance,  
        bool accountStatus  
    );  
    BankAccount(const BankAccount& otherBankAccount);
```

```

    Depositor getDepositor();
    int getAccountNumber();
    std::string getAccountType();
    AccountType getType();
    double getAccountBalance();
    bool getAccountStatus();
    std::vector<TransactionReceipt> getTransactionReceipts();

    virtual TransactionReceipt getBalance(TransactionTicket transactionTicket);
    virtual TransactionReceipt makeDeposit(TransactionTicket transactionTicket);
    virtual TransactionReceipt makeWithdrawal(TransactionTicket transactionTicket);
    TransactionReceipt reOpenAccount(TransactionTicket transactionTicket);
    TransactionReceipt closeAccount(TransactionTicket transactionTicket);

    virtual void addTransactionReceipt(
        const TransactionReceipt& transactionReceipt
    );

    virtual std::string printAccount();
    std::string printAccountBalance();
    std::string printAccountStatus();
    std::string printTransactionHistory();
    virtual std::string printAccountReceipt();

protected:
    Depositor depositor;
    int accountNumber;
    AccountType accountType;
    double accountBalance;
    bool accountStatus;
    std::vector<TransactionReceipt> transactionReceipts;
};

```

17.5 Constructors

17.5.1 Default Constructor

This initializes a blank account with default values. By default every null account is a `CheckingAccount` and any account number with 000000 we know is a null account.

```

BankAccount::BankAccount():
    depositor(Depositor()),
    accountNumber(000000),
    accountType(AccountType::Checking),
    accountBalance(0.0),
    accountStatus(false) {}

```

Useful for placeholder objects or deferred initialization.

17.5.2 Parameterized Constructor

This creates a fully initialized account.

```

BankAccount::BankAccount(
    Depositor depositor,
    int accountNumber,
    AccountType accountType,
    double accountBalance,
    bool accountStatus

```

```

):
    depositor(depositor),
    accountNumber(accountNumber),
    accountType(accountType),
    accountBalance(accountBalance),
    accountStatus(accountStatus) {}

```

17.5.3 Copy Constructor

Creates a deep copy of an existing account.

```

BankAccount::BankAccount(
    const BankAccount& otherBankAccount
):
    depositor(otherBankAccount.depositor),
    accountNumber(otherBankAccount.accountNumber),
    accountType(otherBankAccount.accountType),
    accountBalance(otherBankAccount.accountBalance),
    accountStatus(otherBankAccount.accountStatus),
    transactionReceipts(otherBankAccount.transactionReceipts) {}

```

17.6 Getter Functions

```

Depositor getDepositor();
int getAccountNumber();
std::string getAccountType();
AccountType getType();
double getAccountBalance();
bool getAccountStatus();
std::vector<TransactionReceipt> getTransactionReceipts();

```

These functions provide controlled access to account state while preserving encapsulation.

17.7 Transaction Functions

17.7.1 getBalance()

Returns a receipt reflecting the current balance without modifying account state.

```

/* Bank Account Transaction Series */
TransactionReceipt BankAccount::getBalance(TransactionTicket transactionTicket){
    // Bank Account get balance. returns the account balance
    TransactionReceipt transactionReceipt = TransactionReceipt(
        // create transaction receipt
        transactionTicket, // transaction ticket
        getAccountType(), // account type
        accountBalance, // account balance
        accountBalance, // account balance
        true, // success
        " "
    );

    // add receipt to record
    addTransactionReceipt(transactionReceipt);
    return transactionReceipt; // return receipt
}

```

17.7.2 makeDeposit()

- Validates that the transaction amount is positive.
- Updates account balance.
- Generates and logs a TransactionReceipt.

Throws InvalidAmountException if the amount is negative.

```
TransactionReceipt BankAccount::makeDeposit(TransactionTicket transactionTicket){
    // Bank Account make deposit. deposits funds into an account
    if (transactionTicket.getTransactionAmount() < 0){
        // if transaction amount is less than 0
        throw InvalidAmountException(
            // throw invalid amount exception
            transactionTicket.printTransactionAmount()
        );
    } else {
        // Copy balance and perform valid deposit
        double preTransactionBalance = accountBalance;
        accountBalance += transactionTicket.getTransactionAmount();

        // Creat transaction Receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account type
            preTransactionBalance, // pre balance
            accountBalance, // account balance
            true, // succes
            " "
        );

        // Add transaction to record and return receipt
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    }
}
```

17.7.3 makeWithdrawl()

- Validates that the amount is positive.
- Ensures sufficient funds.
- Deducts from balance.
- Generates and logs a receipt.

Throws:

- InvalidAmountException
- InsufficientFundsException

```
TransactionReceipt BankAccount::makeWithdrawl(TransactionTicket transactionTicket){
    // Bank Account make withdrawl method. withdraws funds from an account
    if (transactionTicket.getTransactionAmount() < 0){
        // If transaction amount is less than 0
        throw InvalidAmountException(
            // Throw invalid amount exception

```

```

        to_string(transactionTicket.getTransactionAmount())
    );
} else if (transactionTicket.getTransactionAmount() > accountBalance){
    // if transaction amount is greater than the balance
    throw InsufficientFundsException(
        // throw insufficient funds exception
        printAccountBalance()
    );
} else {
    // copy balance and perform transaction
    double preTransactionBalance = accountBalance; // copy balance
    accountBalance -= transactionTicket.getTransactionAmount();

    // Create Transaction Receipt
    TransactionReceipt transactionReceipt = TransactionReceipt(
        transactionTicket, // transaction ticket
        getAccountType(), // account type
        preTransactionBalance, // pre transaction balance
        accountBalance, // post transaction balance
        true, // success
        " " // no reason
    );

    // Add receipt to record and return
    addTransactionReceipt(transactionReceipt);
    return transactionReceipt; // return receipt
}
}

```

17.7.4 closeAccount()

- Sets account status to closed.
- Resets balance to zero.
- Logs receipt.

Throws `AccountClosedException` if already closed.

```

TransactionReceipt BankAccount::closeAccount(TransactionTicket transactionTicket){
    // Bank Account close account function. closes an account and withdraws the
    // balance
    if (accountStatus){
        accountStatus = false; // close account
        accountBalance = 0.0; // set balance to 0

        // generate receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account type
            accountBalance, // account balance
            accountBalance, // account balance
            true, // success
            " " // no reason
        );

        // add receipt
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    } else {

```

```

        throw AccountClosedException();
    }
}

```

17.7.5 reOpenAccount()

- Reopens a closed account.
- Logs receipt.

Throws `AccountOpenException` if already open.

```

TransactionReceipt BankAccount::reOpenAccount(TransactionTicket transactionTicket){
    // ReOpenAccount function. re-opens a closed account
    if(!accountStatus) {
        accountStatus = true; // re open account

        // Generate receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account Type
            accountBalance, // pre balance
            accountBalance, // post balance
            true, // success
            " " // no reason
        );

        // Add transaction receipt to record
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    } else {
        // Throw account open exception
        throw AccountOpenException();
    }
}

```

17.8 Formatting Functions

17.8.1 printAccount()

Generates a formatted single-line account summary including:

- Client name
- SSN
- Account number
- Account type
- Status
- Formatted balance

17.8.2 printAccountBalance()

Here is our formatted balance function. it formats the balance with:

\$1,234,567.89

Using fixed precision and manual comma insertion.

17.8.3 printAccountStatus()

Returns:

- "Open" if active
- "Closed" otherwise

17.8.4 printTransactionHistory()

This method return the list of transaction receipts in a printed format.

- Account header information
- A formatted transaction history table
- All stored TransactionReceipt entries

17.9 Unit Testing

17.9.1 Testing Strategy

Our BankAccount unit tests verify:

- Proper account initialization
- Correct balance updates
- Deposit and withdrawal behavior
- Close and reopen functionality
- Receipt logging
- Proper formatted output

17.9.2 Test Runner

```
void testBankAccount(){
    TEST(testAccount);
    TEST(testPrintAccount);
    TEST(testGetBalance);
    TEST(testMakeDeposit);
    TEST(testMakeWithdrawl);
    TEST(testCloseOpenAccount);
    TEST(testBankAccountGetTransactionReceipts);
    TEST(testAddTransactionReceipt);
}
```

17.10 Implementation

17.10.1 BankAccount.hpp

The header file should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "Depositor.hpp"
#include "TransactionReceipt.hpp"

/*
    AccountType Enum
    returns the account type
```

```

*/
enum class AccountType {
    Checking, // checking account
    Savings, // savings account
    CD, // CD account
    Commercial, // Commercial account
    Brokerage, // brokerage account
    Credit // credit account
};

class BankAccount{
    /*
        Bank Account Class
        serves as the base for all accounts. contains a Depositor, account number,
        account type, account balance, account status, and a record of transaction
        receipts
    */
public:
    // Bank Account constructor series
    BankAccount(); // No args constructor
    BankAccount(
        // Bank Account Key word args constructor
        Depositor depositor, // depositor
        int accountNumber, // account number
        AccountType accountType, // account type
        double accountBalance, // account balance
        bool accountStatus // account status
    );
    // Bank Account copy constructor
    BankAccount(const BankAccount& otherBankAccount);

    // Bank Account Getter Series
    Depositor getDepositor(); // return depositor
    int getAccountNumber(); // return account number
    std::string getAccountType(); // return account type string
    AccountType getType(); // returns account type enum
    double getAccountBalance(); // return account balance
    bool getAccountStatus(); // return account status
    // return all transaction receipts
    std::vector<TransactionReceipt> getTransactionReceipts();

    // Bank Account Transaction Series
    virtual TransactionReceipt getBalance(TransactionTicket transactionTicket);
    virtual TransactionReceipt makeDeposit(TransactionTicket transactionTicket);
    virtual TransactionReceipt makeWithdrawal(TransactionTicket transactionTicket);
    TransactionReceipt reOpenAccount(TransactionTicket transactionTicket);
    TransactionReceipt closeAccount(TransactionTicket transactionTicket);

    // Bank Account helper methods
    virtual void addTransactionReceipt(const TransactionReceipt& transactionReceipt);

    // Bank Account formatted Series
    virtual std::string printAccount(); // formatted print
    std::string printAccountBalance(); // format balance
    std::string printAccountStatus(); // format account status
    std::string printTransactionHistory(); // print receipts
    virtual std::string printAccountReceipt(); // format for receipts

```



```

protected:
    Depositor depositor; // client name and ssn
    int accountNumber; // account number
    AccountType accountType; // account Type
    double accountBalance; // account balance
    bool accountStatus; // account Status
    std::vector<TransactionReceipt> transactionReceipts; // transaction receipts
    BankAccount* receiptMemoryAddress = nullptr; // Bank Account memory address

};

// Unit Testing
void testBankAccount();

```

17.10.2 BankAccount.cpp

The entire source file should look like this:

```

#include "BankAccount.hpp"

using namespace std;

// Bank Account constructor series
BankAccount::BankAccount():
    // Bank Account No args constructor
    depositor(Depositor()), // depositor
    accountNumber(000000), // account Number
    accountType(AccountType::Checking), // account type
    accountBalance(0.0), // account balance
    accountStatus(false) // account status
{}

BankAccount::BankAccount(
    // Bank Account Key word args constructor
    Depositor depositor, // depositor
    int accountNumber, // account number
    AccountType accountType, // account type
    double accountBalance, // account Balance
    bool accountStatus // account status
):
    depositor(depositor), // set depositor
    accountNumber(accountNumber), // set account number
    accountType(accountType), // set account type
    accountBalance(accountBalance), // set account balance
    accountStatus(accountStatus) // set account status
{}

BankAccount::BankAccount(
    // Bank Account copy constructor
    const BankAccount& otherBankAccount // other bank account
):
    depositor(otherBankAccount.depositor), // copy depositor
    accountNumber(otherBankAccount.accountNumber), // copy number
    accountType(otherBankAccount.accountType), // copy type
    accountBalance(otherBankAccount.accountBalance), // copy balance

```

```
        accountStatus(otherBankAccount.accountStatus), // copy status
        transactionReceipts(otherBankAccount.transactionReceipts) // copy receipts
    {}

    // Bank account Getter Series
    Depositor BankAccount::getDepositor(){
        // Bank Account Get depositor function. returns depositor copy
        return Depositor(depositor); // return depositor
    }

    int BankAccount::getAccountNumber(){
        // Bank account get account number. returns account number
        return accountNumber; // return account number
    }

    string BankAccount::getAccountType(){
        // Bank Account get account Type. returns the account Type
        switch(accountType){
            // Checking account
            case AccountType::Checking:
                return "Checking";
            case AccountType::Savings:
                // Savings account
                return "Savings"; // return savings
            case AccountType::CD:
                // CD account
                return "CD"; // return CD
            case AccountType::Commercial:
                // Commercial Account
                return "Commercial"; // return commercial
            case AccountType::Brokerage:
                // Brokerage Account
                return "Brokerage"; // return brokerage
            default:
                return "Unknown";
        }
    }

    AccountType BankAccount::getType(){
        // Bank Account return account type. returns just the type
        return accountType; // return account type
    }

    double BankAccount::getAccountBalance(){
        // bank account get account balance. returns the account balance
        return accountBalance; // return account balance
    }

    bool BankAccount::getAccountStatus(){
        // bank account get account Status. returns account status
        return accountStatus; // return status
    }

    vector<TransactionReceipt> BankAccount::getTransactionReceipts(){
        // bank Account get transaction receipt. returns record of
        // receipts
        return transactionReceipts; // return all receipts
    }
```

```

// BankAccount Helper Series
void BankAccount::addTransactionReceipt(const TransactionReceipt& transactionReceipt){
    // Bank Account add transaction receipt method. adds a transaction
    // receipt to the record
    transactionReceipts.emplace_back(transactionReceipt); // add receipt
}

/* Bank Account Transaction Series */
TransactionReceipt BankAccount::getBalance(TransactionTicket transactionTicket){
    // Bank Account get balance. returns the account balance
    TransactionReceipt transactionReceipt = TransactionReceipt(
        // create transaction receipt
        transactionTicket, // transaction ticket
        getAccountType(), // account type
        accountBalance, // account balance
        accountBalance, // account balance
        true, // success
        " "
    );

    // add receipt to record
    addTransactionReceipt(transactionReceipt);
    return transactionReceipt; // return receipt
}

TransactionReceipt BankAccount::makeDeposit(TransactionTicket transactionTicket){
    // Bank Account make deposit. deposits funds into an account
    if (transactionTicket.getTransactionAmount() < 0){
        // if transaction amount is less than 0
        throw InvalidAmountException(
            // throw invalid amount exception
            transactionTicket.printTransactionAmount()
        );
    } else {
        // Copy balance and perform valid deposit
        double preTransactionBalance = accountBalance;
        accountBalance += transactionTicket.getTransactionAmount();

        // Creat transaction Receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account type
            preTransactionBalance, // pre balance
            accountBalance, // account balance
            true, // succes
            " "
        );

        // Add transaction to record and return receipt
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    }
}

TransactionReceipt BankAccount::makeWithdrawl(TransactionTicket transactionTicket){
    // Bank Account make withdrawl method. withdraws funds from an account
    if (transactionTicket.getTransactionAmount() < 0){

```

```

        // If transaction amount is less than 0
        throw InvalidAmountException(
            // Throw invalid amount exception
            to_string(transactionTicket.getTransactionAmount())
        );
    } else if (transactionTicket.getTransactionAmount() > accountBalance){
        // if transaction amount is greater than the balance
        throw InsufficientFundsException(
            // throw insufficient funds exception
            printAccountBalance()
        );
    } else {
        // copy balance and perform transaction
        double preTransactionBalance = accountBalance; // copy balance
        accountBalance -= transactionTicket.getTransactionAmount();

        // Create Transaction Receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account type
            preTransactionBalance, // pre transaction balance
            accountBalance, // post transaction balance
            true, // success
            " " // no reason
        );

        // Add receipt to record and return
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    }
}

TransactionReceipt BankAccount::closeAccount(TransactionTicket transactionTicket){
    // Bank Account close account function. closes an account and withdraws the
    // balance
    if (accountStatus){
        accountStatus = false; // close account
        accountBalance = 0.0; // set balance to 0

        // generate receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account type
            accountBalance, // account balance
            accountBalance, // account balance
            true, // success
            " " // no reason
        );

        // add receipt
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    } else {
        throw AccountClosedException();
    }
}

TransactionReceipt BankAccount::reOpenAccount(TransactionTicket transactionTicket){
    // ReOpenAccount function. re-opens a closed account

```

```

    if(!accountStatus) {
        accountStatus = true; // re open account

        // Generate receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction ticket
            getAccountType(), // account Type
            accountBalance, // pre balance
            accountBalance, // post balance
            true, // success
            " " // no reason
        );

        // Add transaction receipt to record
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    } else {
        // Throw account open exception
        throw AccountOpenException();
    }
}

/* Bank Account Formmetter Series */
string BankAccount::printAccount(){
    // Bank Account print Account. returns a formatted string of account
    ostringstream accountString; // initialize stream string
    // Build string
    accountString << left << setw(20) <<
        depositor.getDepositorName().printName()
        << setw(12) << depositor.getDepositorSSN()
        << setw(10) << accountNumber
        << setw(15) << getAccountType()
        << setw(7) << printAccountStatus()
        << right << setw(15) <<
        printAccountBalance();
    return accountString.str(); // return string
}

string BankAccount::printAccountBalance(){
    // Bank account print account balance method. Prints a fomatted
    // balance
    ostringstream balance; // write string stream
    balance << "$" << fixed << setprecision(2) << accountBalance;
    // convert to string
    string formattedBalance = balance.str();

    // Insert commas manually
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }

    return formattedBalance; // return balance
}

string BankAccount::printAccountStatus(){
    // Print account Status. Returns a formatted string of the
    // account status

```

```

    if (accountStatus){
        // return open if true
        return "Open";
    } else {
        // return closed if flase
        return "Closed";
    }
}

string BankAccount::printTransactionHistory(){
    // Bank Account print Transaction history. Returns the long of
    // receipts
    ostringstream transactionHistory;
    transactionHistory << left
        << setw(21) << "\nClient Name" // client name
        << setw(12) << "SSN" // ssn
        << setw(10) << "Acct Num" // account number
        << setw(15) << "Acct Type" // account type
        << setw(7) << "Status" // account status
        << right
        << setw(12) << "Balance"; // balance
    if (accountType == AccountType::CD){
        // If maturity date write maturity date headline
        transactionHistory << setw(15) << "Maturity Date";
    }
    transactionHistory << "\n"
        << printAccount() << "\n"
        << "\n\n\t\t"
        << string(20, '*') << " Transaction History "
        << string(20, '*') << "\n"
        << setw(5) << "Date" // transaction date
        << setw(20) << "Transaction" // transaction type
        << setw(15) << "Amount" // transaction amount
        << setw(10) << "Status" // status
        << setw(12) << "Balance" // post balance
        << setw(25) << "Reason for Failure\n"; // error
    for (TransactionReceipt receipt: transactionReceipts){
        // print receipt
        transactionHistory << receipt.printReceipt() << "\n";
    }
    return transactionHistory.str(); // return string
}

string BankAccount::printAccountReceipt(){
    // Print Account prints accounts for receipts
    ostringstream receiptString;
    receiptString << left
        << "Client Name: "
        << depositor.getDepositorName().printName()
        << "\t\t\tAccount Number: "
        << to_string(accountNumber)
        << "\nAccount Type: "
        << getAccountType();
    // return receipt string
    return receiptString.str();
}

// Unit testing series

```

```
static BankAccount generateTestAccount(){
    // Generate test account. generates a test account
    BankAccount testBankAccount = BankAccount(
        // Initialize test account
        Depositor(
            Name("Nickolas", "Black"),
            "123456789"
        ), 000000, AccountType::Savings, 100.00, true
    );
    return testBankAccount; // return test account
}

static void testAccount(){
    // Unit test for bank account creation
    BankAccount testBankAccount = generateTestAccount();
    // Run Assertions
    ASSERT(testBankAccount
        .getDepositor().getDepositorName().printName() ==
        "Nickolas Black");
    ASSERT(testBankAccount
        .getDepositor().getDepositorSSN() ==
        "123456789");
    ASSERT(testBankAccount.getAccountNumber() == 000000);
    ASSERT(testBankAccount.getAccountType() == "Account Type");
    ASSERT(testBankAccount.getAccountBalance() == 100.00);
    ASSERT(testBankAccount.getAccountStatus() == true);
}

static void testGetBalance(){
    // Unit Test for Bank Account get balance function
    BankAccount testBankAccount = generateTestAccount();
    // Generate test transaction ticket
    TransactionTicket testTransactionTicket = TransactionTicket(
        TransactionType::Balance, // transaction type
        testBankAccount.getAccountNumber(), // account Number
        0.0, // transaction amount
        Calendar(01,01,2001), // transaction date
        0 // cd term
    );
    // Generate Transaction receipt
    TransactionReceipt testTransactionReceipt = testBankAccount.getBalance(
        testTransactionTicket // Test ticket
    );
    // Run Assertions
    ASSERT(testTransactionReceipt.getPostTransactionBalance() == 100.00);
}

static void testMakeDeposit(){
    // Unit Test for Bank Account make deposit function
    BankAccount testBankAccount = generateTestAccount();
    // Generat test Transaction ticket
    TransactionTicket testTransactionTicket = TransactionTicket(
        TransactionType::Deposit, // transaction type
        testBankAccount.getAccountNumber(), // account number
        20.0, // transaction amount
        Calendar(01,01,2001), // transaction date
        0 // cd term
    );
}
```

```
// Generate test receipt
TransactionReceipt testTransactionReceipt = testBankAccount.makeDeposit(
    testTransactionTicket // test ticket
);
// Run Assertion
ASSERT(testTransactionReceipt.getPostTransactionBalance() == 120.0);
}

static void testMakeWithdrawl(){
    // Unit test for bank Account make withdrawl function
    BankAccount testBankAccount = generateTestAccount();
    // Generate test transaction Ticket
    TransactionTicket testTransactionTicket = TransactionTicket(
        TransactionType::Withdrawl, // transaction type
        testBankAccount.getAccountNumber(), // account number
        50.0, // transaction amount
        Calendar(01,01,2001), // transaction date
        0 // cd term
    );
    // Generate transaction Receipt
    TransactionReceipt testTransactionReceipt = testBankAccount.makeWithdrawl(
        testTransactionTicket // test ticket
    );
    // Run Assertions
    ASSERT(testTransactionReceipt.getPostTransactionBalance() == 50.0);
}

static void testCloseOpenAccount(){
    // Unit test for reopen account
    BankAccount testBankAccount = generateTestAccount();
    // Generate transaction ticket
    TransactionTicket testTransactionTicket = TransactionTicket(
        TransactionType::Close, // transaction type
        testBankAccount.getAccountNumber(), // test account number
        0.0, // transaction amount
        Calendar(01,01,2001), // transaction date
        0 // cd term
    );
    // Generate transaction receipt
    TransactionReceipt testTransactionReceipt = testBankAccount.closeAccount(
        testTransactionTicket // test ticket
    );
    // run assertion
    ASSERT(testBankAccount.getAccountStatus() == false);

    // Generate ticket to re open account
    testTransactionTicket = TransactionTicket(
        TransactionType::Open, // transaction type
        testBankAccount.getAccountNumber(), // account number
        0.0, // transaction amount
        Calendar(01,01,2001), // transaction date
        0 // cd term
    );
    // Generate transaction Receipt
    testTransactionReceipt = testBankAccount.reOpenAccount(
        testTransactionTicket // test ticket
    );
    // run assertion
```



```

    ASSERT(testBankAccount.getAccountStatus() == true);
}

static void testBankAccountGetTransactionReceipts(){
    // get transaction receipt unit test
    BankAccount testBankAccount = generateTestAccount();
    // Add a transaction receipt
    testBankAccount.addTransactionReceipt(
        TransactionReceipt() // test receipt
    );
    // assert
    ASSERT(testBankAccount.getTransactionReceipts() != nullptr);
}

static void testAddTransactionReceipt(){
    // Test add transaction Receipt unit test
    BankAccount testBankAccount = generateTestAccount();
    // Add receipts
    testBankAccount.addTransactionReceipt(
        TransactionReceipt() // null receipt
    );
    testBankAccount.addTransactionReceipt(
        TransactionReceipt() // null receipt
    );
    // Run Assertion
    ASSERT(testBankAccount.getTransactionReceipts().size() == 2);
}

static void testPrintAccount(){
    // Unit test for print account
    BankAccount account = BankAccount(
        // Initialize test account
        Depositor(
            // Initialize depositor
            Name(
                // Initialize name
                "Nickolas", "Black"),
                "123456789"
            ),
        123456, // Account Number
        AccountType::Checking, // Account Type
        100000000, // account Balance
        true // account status
    );
    // Run Assertion
    ASSERT(account.printAccount() ==
        "Nickolas Black      123456789   123456   Checking   Open      $100,000,000.00");
}

void testBankAccount(){
    // Test account unit test
    TEST(testAccount); // test account creation
    TEST(testPrintAccount); // test print
    TEST(testGetBalance); // test get balance
    TEST(testMakeDeposit); // test make deposit
    TEST(testMakeWithdrawl); // test make withdrawl
    TEST(testCloseOpenAccount); // test close & open account
    TEST(testBankAccountGetTransactionReceipts); // test get receipts
}

```

```
    TEST(testAddTransactionReceipt); // test add receipt  
}
```

17.11 Summary

The **BankAccount** class forms the backbone of the banking system. By encapsulating depositor identity, financial state, transaction processing, and receipt logging, it provides:

- Accurate financial state management
- Automatic transaction documentation
- Strong type-safe account categorization
- Extensibility for specialized account types

It models real-world banking behavior while maintaining clean, object-oriented design principles.

18 SavingsAccount Class

18.1 Overview

Next we have the **SavingsAccount** class. This class represents a savings account within the banking system and extends the **BankAccount** base class.

Unlike checking or certificate accounts, a savings account allows clients to perform deposits and withdrawals at any time while maintaining standard account rules defined in the base class.

A **SavingsAccount** contains (inherited from **BankAccount**):

- Depositor information
- Account number
- Account type
- Account balance
- Account status (open/closed)

The account type is automatically set to **Savings**, ensuring type consistency throughout the banking system.

18.2 Design Rationale

The key ideas here are:

- Inheritance from **BankAccount** to enable polymorphism.
- Automatic assignment of **AccountType::Savings**.
- Reuse of deposit and withdrawal logic defined in the base class.
- Constructor delegation to centralize account initialization.
- Clean separation between account type and account behavior.

This design ensures savings accounts integrate seamlessly with other account types (e.g., checking or CD accounts) while preserving type-specific identity.

18.3 Class Declaration

Header Definitions Here are the header definitions for `SavingsAccount`. Key things to note here is that a `SavingsAccount` extends the `BankAccount` class.

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"

class SavingsAccount: public BankAccount{
public:
    SavingsAccount();

    SavingsAccount(
        Depositor depositor,
        int accountNumber,
        double accountBalance,
        bool accountStatus
    );

    SavingsAccount(const SavingsAccount& otherSavingsAccount);
};

// Unit Test
void testSavingsAccount();
```

18.4 Constructors

18.4.1 Default Constructor

Initializes a savings account with default values.

```
SavingsAccount::SavingsAccount(): BankAccount(
    Depositor(),
    000000,
    AccountType::Savings,
    0.0,
    true
){}
```

This constructor:

- Creates an empty depositor
- Sets account number to 0
- Sets account type to Savings
- Initializes balance to 0.0
- Opens the account by default

It is useful for placeholder objects or delayed assignment.

18.4.2 Parameterized Constructor

Creates a fully defined savings account using parameters.

```
SavingsAccount::SavingsAccount(
    Depositor depositor,
    int accountNumber,
```

```
        double accountBalance,  
        bool accountStatus  
    ) : BankAccount(  
        depositor,  
        accountNumber,  
        AccountType::Savings,  
        accountBalance,  
        accountStatus  
    ) {}
```

This constructor delegates initialization to the `BankAccount` constructor while automatically setting the account type to `Savings`.

This guarantees that the account cannot accidentally be misclassified.

18.4.3 Copy Constructor

Creates a copy of an existing savings account.

```
SavingsAccount::SavingsAccount(  
    const SavingsAccount& otherSavingsAccount  
) : BankAccount(otherSavingsAccount) {}
```

Since all member data is managed in the base class, the copy constructor simply invokes the base class copy constructor.

18.5 Inherited Functionality

The `SavingsAccount` class inherits all public member functions from `BankAccount`, including:

- `getDepositor()`
- `getAccountNumber()`
- `getAccountType()`
- `getAccountBalance()`
- `getAccountStatus()`
- Deposit and withdrawal operations

No additional behavior is required because savings accounts follow standard banking rules defined in the base class.

18.6 Unit Testing

18.6.1 Testing Strategy

We have our unit tests to verify:

- Proper inheritance from `BankAccount`
- Correct assignment of account number
- Automatic setting of account type to "Savings"
- Accurate balance initialization
- Proper account status storage

18.6.2 Test Implementation

```
static void testSavingsAcct(){
    SavingsAccount testSavingsAccount(
        Depositor(),
        123456,
        100.0,
        true
    );

    ASSERT(testSavingsAccount.getDepositor() != nullptr);
    ASSERT(testSavingsAccount.getAccountNumber() == 123456);
    ASSERT(testSavingsAccount.getAccountType() == "Savings");
    ASSERT(testSavingsAccount.getAccountBalance() == 100.0);
    ASSERT(testSavingsAccount.getAccountStatus() == true);
}
```

18.6.3 Test Runner

```
void testSavingsAccount(){
    TEST(testSavingsAcct);
}
```

18.7 Implementation

18.7.1 SavingsAccount.hpp

The header file should appear as followed:

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"

/*
    Savings Account Class
    this class extends bank account. A client can perform deposits
    and withdraws at any time
*/
class SavingsAccount: public BankAccount{
public:
    // Savings Account constructor series
    SavingsAccount(); // no args constructor
    SavingsAccount(
        // Savings Account no args constructir
        Depositor depositor, // depositor
        int accountNumber, // account number
        double accountBalance, // account balance
        bool accountStatus // account status
    );
    SavingsAccount(const SavingsAccount& otherSavingsAccount);
};

// Unit Test
void testSavingsAccount();
```

18.7.2 SavingsAccount.cpp

The entire source file should appear as followed:

```
#include "SavingsAccount.hpp"

using namespace std;

// Savings Account constructor series
SavingsAccount::SavingsAccount(): BankAccount(
    // Savings Account No Args Constructor
    Depositor(), // null depositor
    000000, // account number
    AccountType::Savings, // account type
    0.0, // no balance
    true // account open
){}

SavingsAccount::SavingsAccount(
    // Savings Account key word args constructor
    Depositor depositor, // depositor
    int accountNumber, // account number
    double accountBalance, // account balance
    bool accountStatus // account Status
) : BankAccount(
    // Bank Account constructor
    depositor, // set depositor
    accountNumber, // set number
    AccountType::Savings, // set account type
    accountBalance, // set account balance
    accountStatus // set account status
){}

SavingsAccount::SavingsAccount(
    // Savings Account copy constructor
    const SavingsAccount& otherSavingsAccount
) : BankAccount(otherSavingsAccount){}

/* Unit Testing Series */
static void testSavingsAcct(){
    // Unit test for savings account construction
    SavingsAccount testSavingsAccount = SavingsAccount(
        Depositor(), // initialize depositor
        123456, // account number
        100.0, // account balance
        true // account open
    );
    // Assert
    ASSERT(testSavingsAccount.getDepositor() != nullptr);
    ASSERT(testSavingsAccount.getAccountNumber() == 123456);
    ASSERT(testSavingsAccount.getAccountType() == "Savings");
    ASSERT(testSavingsAccount.getAccountBalance() == 100.0);
    ASSERT(testSavingsAccount.getAccountStatus() == true);
}

void testSavingsAccount(){
    // Unit test for savings account class
    TEST(testSavingsAcct);
}
```

18.8 Summary

The **SavingsAccount** class models a standard savings account within the banking framework. By delegating core functionality to **BankAccount** while automatically enforcing the savings account type, it provides:

- Clean inheritance-based design
- Strong type safety
- Centralized account logic
- Seamless integration with other account types

The result is a simple, maintainable, and extensible banking architecture.

19 CheckingAccount Class

19.1 Overview

Next we have the **CheckingAccount** class. This class represents a checking account within the banking system and extends the **BankAccount** base class. Unlike savings accounts, a checking account introduces the ability to clear written checks. It also applies an overdraft fee if a check is processed with insufficient funds.

A **CheckingAccount** contains (inherited from **BankAccount**):

- Depositor information
- Account number
- Account type
- Account balance
- Account status (open/closed)
- Transaction history

Additionally, it introduces:

- A `clearCheck()` transaction method

19.2 Design Rationale

The key ideas here are:

- Inheritance from **BankAccount** to enable polymorphism.
- Automatic assignment of `AccountType::Checking`.
- Encapsulation of check-clearing rules within the class.
- Enforcement of banking constraints (six-month validity, overdraft fee).
- Automatic transaction receipt generation.

This design ensures checking accounts integrate seamlessly with other account types while preserving behavior unique to check processing.

19.3 Class Declaration

Header Definitions Here are the header definitions for a `CheckingAccount`:

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"
#include "Check.hpp"

class CheckingAccount: public BankAccount {
public:
    CheckingAccount();

    CheckingAccount(
        Depositor depositor,
        int accountNumber,
        double accountBalance,
        bool accountStatus
    );

    CheckingAccount(const CheckingAccount& otherCheckingAccount);

    TransactionReceipt clearCheck(Check checkToClear);
};
```

Unit Testing

```
void testCheckingAccount();
```

19.4 Constructors

19.4.1 Default Constructor

Initializes a checking account with default values.

```
CheckingAccount::CheckingAccount(): BankAccount(
    Depositor(),
    000000,
    AccountType::Checking,
    0.0,
    true
){}
```

This constructor:

- Creates an empty depositor
- Sets account number to 0
- Sets account type to Checking
- Initializes balance to 0.0
- Opens the account by default

19.4.2 Parameterized Constructor

This creates a fully defined checking account.

```
CheckingAccount::CheckingAccount(
    Depositor depositor,
    int accountNumber,
    double accountBalance,
    bool accountStatus
) : BankAccount(
    depositor,
    accountNumber,
    AccountType::Checking,
    accountBalance,
    accountStatus
){}
```

This guarantees that the account type is always correctly set to `Checking`.

19.4.3 Copy Constructor

We can create a copy of an existing checking account.

```
CheckingAccount::CheckingAccount(
    const CheckingAccount& otherCheckingAccount
) : BankAccount(otherCheckingAccount){}
```

All state is managed by the base class, so delegation ensures consistent duplication.

19.5 Core Checking Function

19.5.1 clearCheck()

Processes a check and enforces banking rules.

```
TransactionReceipt CheckingAccount::clearCheck(Check checkToClear){
    double preTransactionBalance = accountBalance;

    Calendar today = Calendar();
    int presentMonth = today.getMonth();
    int presentYear = today.getYear();

    presentMonth -= 6;
    if(presentMonth <= 0) {
        presentMonth += 12;
        presentYear -= 1;
    }

    Calendar sixMonthsAgo = Calendar(
        presentMonth,
        today.getDay(),
        presentYear
    );

    if (checkToClear.getDateOfCheck().before(sixMonthsAgo)){
        throw CheckTooOldException(
            checkToClear.getDateOfCheck().printDate()
        );
    }
    else if (checkToClear.getCheckAmount() > accountBalance){
        accountBalance -= 2.50;
    }
}
```

```

        throw InsufficientFundsException(
            to_string(preTransactionBalance)
        );
    }
    else if (checkToClear.getDateOfCheck().after(today)) {
        throw PostDatedCheckException();
    }
    else {
        accountBalance -= checkToClear.getCheckAmount();

        TransactionReceipt transactionReceipt =
            TransactionReceipt(
                TransactionTicket(
                    TransactionType::ClearCheck,
                    checkToClear.getAccountNumber(),
                    checkToClear.getCheckAmount(),
                    today,
                    0
                ),
                getAccountType(),
                preTransactionBalance,
                accountBalance,
                true,
                " "
            );

        addTransactionReceipt(transactionReceipt);
        return transactionReceipt;
    }
}

```

Behavior Summary The method enforces:

- Checks older than six months are rejected.
- Post-dated checks are rejected.
- Insufficient funds triggers a \$2.50 overdraft fee.
- Valid checks reduce account balance.
- Every successful transaction generates a receipt.

19.6 Unit Testing

19.6.1 Testing Strategy

The unit tests verify:

- Proper account construction
- Correct type assignment
- Successful check clearing
- Accurate post-transaction balance

19.6.2 Test Implementation

```
static void testCheckingAcct(){
    CheckingAccount testCheckingAccount(
        Depositor(),
        000000,
        100.00,
        true
    );

    ASSERT(testCheckingAccount.getDepositor() != nullptr);
    ASSERT(testCheckingAccount.getAccountNumber() == 000000);
    ASSERT(testCheckingAccount.getAccountType() == "Checking");
    ASSERT(testCheckingAccount.getAccountBalance() == 100.0);
    ASSERT(testCheckingAccount.getAccountStatus() == true);
}

static void testClearCheck(){
    Check testCheck(
        000000,
        50.00,
        Calendar()
    );

    CheckingAccount testCheckingAccount(
        Depositor(),
        testCheck.getAccountNumber(),
        100.00,
        true
    );

    TransactionReceipt transactionReceipt =
        testCheckingAccount.clearCheck(testCheck);

    ASSERT_APPROX_EQUAL(
        transactionReceipt.getPostTransactionBalance(),
        50.00,
        0.001
    );
}
```

19.6.3 Test Runner

```
void testCheckingAccount(){
    TEST(testCheckingAcct);
    TEST(testClearCheck);
}
```

19.7 Implementation

19.7.1 CheckingAccount.hpp

The entire header file should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"
#include "Check.hpp"
```

```

/*
    Checking account class
    contains a clear check method. Clients can make deposits and
    withdrawals when they want to. Charges an over draft free if account
    is over drawn
*/
class CheckingAccount: public BankAccount {
public:
    // Checking Account constructor series
    CheckingAccount(); // no args constructor
    CheckingAccount(
        // Checking account key word args constructor
        Depositor depositor, // depositor
        int accountNumber, // account number
        double accountBalance, // account balance
        bool accountStatus // account Status
    );
    CheckingAccount(const CheckingAccount& otherCheckingAccount);

    // Checking Account Transaction Series
    TransactionReceipt clearCheck(Check checkToClear);
};

// Unit Test
void testCheckingAccount();

```

19.7.2 CheckingAccount.cpp

The header file to look like this:

```

#include "CheckingAccount.hpp"

using namespace std;

/* Checking account class constructor series */
CheckingAccount::CheckingAccount(): BankAccount(
    // Checking Account no args constructor
    Depositor(), // null depositor
    000000, // account number
    AccountType::Checking, // checking account
    0.0, // balance
    true // status
){}

CheckingAccount::CheckingAccount(
    // Checking account key word args constructor
    Depositor depositor, // depositor
    int accountNumber, // account number
    double accountBalance, // account balance
    bool accountStatus // account status
) : BankAccount(
    // Bank Account constructor
    depositor, // set depositor
    accountNumber, // set account number
    AccountType::Checking, // set checking
    accountBalance, // set account balance
    accountStatus // set account Status
){}

```

```

CheckingAccount::CheckingAccount(
    // Checking account copy constructor
    const CheckingAccount& otherCheckingAccount
) : BankAccount(otherCheckingAccount){

/* Checking Account Transaction Series */
TransactionReceipt CheckingAccount::clearCheck(Check checkToClear){
    // Checking Account Clear Check Method. Withdraws from an account
    double preTransactionBalance = accountBalance; // copy balance
    // Get six months ago
    Calendar today = Calendar(); // today's date
    int presentMonth = today.getMonth();
    int presentYear = today.getYear();

    presentMonth -= 6; // subtract 6
    if(presentMonth <= 0) {
        presentMonth += 12; // add 12 to month
        presentYear -= 1; // subtract 1 from year
    }

    Calendar sixMonthsAgo = Calendar(
        // Six months ago object
        presentMonth, // month
        today.getDay(), // same day
        presentYear // year
    );

    if (checkToClear.getDateOfCheck().before(sixMonthsAgo)){
        // if older than 6 months ago
        throw CheckTooOldException(
            checkToClear.getDateOfCheck().printDate()
        );
    } else if (checkToClear.getCheckAmount() > accountBalance){
        accountBalance -= 2.50; // apply fee
        // Throw exception
        throw InsufficientFundsException(to_string(preTransactionBalance));
    } else if (checkToClear.getDateOfCheck().after(today)) {
        // if check too old
        throw PostDatedCheckException();
    } else {
        // Valid check clear
        accountBalance -= checkToClear.getCheckAmount();
        // Generate transaction Receipt
        TransactionReceipt transactionReceipt = TransactionReceipt(
            TransactionTicket( // transaction ticket
                TransactionType::ClearCheck, // transaction type
                checkToClear.getAccountNumber(), // account number
                checkToClear.getCheckAmount(), // transaction amount
                today, // time of transaction
                0 // no term
            ),
            getAccountType(), // account type
            preTransactionBalance, // pre balance
            accountBalance, // post balance
            true, // success
            " " // no failure
        );
        // append receipt and return
    }
}

```

```
        addTransactionReceipt(transactionReceipt);
        return transactionReceipt; // return receipt
    }
}

/* Unit Testing Series */
static void testCheckingAcct(){
    // Unit test for checking account creation
    CheckingAccount testCheckingAccount = CheckingAccount(
        // test checking account
        Depositor(), // null depositor
        000000, // account number
        100.00, // account balance
        true // account open
    );
    // Assert
    ASSERT(testCheckingAccount.getDepositor() != nullptr);
    ASSERT(testCheckingAccount.getAccountNumber() == 000000);
    ASSERT(testCheckingAccount.getAccountType() == "Checking");
    ASSERT(testCheckingAccount.getAccountBalance() == 100.0);
    ASSERT(testCheckingAccount.getAccountStatus() == true);
}

static void testClearCheck(){
    // Unit test for clear check
    Check testCheck = Check(
        // Initialize test check
        000000, // account number
        50.00, // amount
        Calendar() // time now
    );
    // Initialize test Checking account
    CheckingAccount testCheckingAccount = CheckingAccount(
        Depositor(), // null depositor
        testCheck.getAccountNumber(), // account number
        100.00, // balance
        true // open
    );
    // Generate Transaction Receipt
    TransactionReceipt transactionReceipt = testCheckingAccount.clearCheck(
        testCheck // test check
    );
    // Assert
    ASSERT_APPROX_EQUAL(
        transactionReceipt.getPostTransactionBalance(), // transaction
        50.00, // expected
        0.001 // tolerance
    );
}

void testCheckingAccount(){
    // Unit test for checking account class
    TEST(testCheckingAcct);
    TEST(testClearCheck);
}
```

19.8 Summary

The `CheckingAccount` class models real-world checking behavior within the banking framework. By extending `BankAccount` and introducing structured check processing logic, it provides:

- Enforcement of check validity rules
- Overdraft fee handling
- Automatic transaction receipt generation
- Clean inheritance-based architecture

Together with `SavingsAccount` and other account types, `CheckingAccount` completes a robust and extensible banking system design.

20 CDAccount Class

20.1 Overview

Next we have the `CDAccount` class. This class represents a Certificate of Deposit (CD) account and extends the `SavingsAccount` base class.

Unlike a savings account, a CD account:

- Requires an initial deposit
- Does not allow withdrawals or deposits until the maturity date
- Tracks a fixed-term maturity date

A `CDAccount` contains (inherited from `SavingsAccount`):

- Depositor information
- Account number
- Account type
- Account balance
- Account status (open/closed)

Additionally, it introduces:

- A `maturityDate` member
- Overridden deposit and withdrawal methods
- Custom account printing methods

20.2 Design Rationale

The key ideas here are:

- Inheritance from `SavingsAccount` to reuse common functionality.
- Enforcement of CD maturity rules for deposits and withdrawals.
- Encapsulation of the maturity date logic within the class.
- Automatic computation of new maturity date after transactions.
- Standardized formatted output for account and receipt printing.

This ensures that CD accounts behave according to real-world banking rules while integrating cleanly with the account framework.

20.3 Class Declaration

Header Definitions

```
#pragma once

#include "Stdafx.hpp"
#include "SavingsAccount.hpp"

class CDAccount : public SavingsAccount {
public:
    CDAccount();
    CDAccount(
        Depositor depositor,
        int accountNumber,
        double accountBalance,
        bool accountStatus,
        int cdTerm
    );
    CDAccount(
        Depositor depositor,
        int accountNumber,
        double accountBalance,
        bool accountStatus,
        Calendar maturityDate
    );
    CDAccount(const CDAccount& otherCDAccount);

    Calendar getMaturityDate();

    TransactionReceipt makeWithdrawal(
        TransactionTicket transactionTicket
    ) override;

    TransactionReceipt makeDeposit(
        TransactionTicket transactionTicket
    ) override;

    std::string printAccount() override;
    std::string printAccountReceipt() override;

private:
    Calendar maturityDate;
};
```

Unit Testing

```
void testCDAccount();
```

20.4 Constructors

20.4.1 Default Constructor

Initializes a CD account with default values.

```
CDAccount::CDAccount() : SavingsAccount(
    Depositor(),
    000000,
    0.0,
    true
```



```

), maturityDate(Calendar()) {
    accountType = AccountType::CD;
}

```

This constructor:

- Creates an empty depositor
- Sets account number to 0
- Sets account type to CD
- Initializes balance to 0.0
- Opens the account
- Initializes maturity date to today

20.4.2 Parameterized Constructor (CD Term)

This creates a CD account with a specific term length in years.

```

CDAccount::CDAccount(
    Depositor depositor,
    int accountNumber,
    double accountBalance,
    bool accountStatus,
    int cdTerm
) : SavingsAccount(depositor, accountNumber, accountBalance, accountStatus) {
    accountType = AccountType::CD;

    Calendar today;
    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + cdTerm
    );
}

```

20.4.3 Parameterized Constructor (Maturity Date)

This creates a CD account with an explicit maturity date.

```

CDAccount::CDAccount(
    Depositor depositor,
    int accountNumber,
    double accountBalance,
    bool accountStatus,
    Calendar maturityDate
) : SavingsAccount(depositor, accountNumber, accountBalance, accountStatus),
    maturityDate(maturityDate) {
    accountType = AccountType::CD;
}

```

20.4.4 Copy Constructor

This creates a copy of an existing CD account.

```

CDAccount::CDAccount(const CDAccount& otherCDAccount)
    : SavingsAccount(otherCDAccount){}

```

20.5 Core CD Functions

20.5.1 getMaturityDate()

This returns a copy of the CD account's maturity date.

```
Calendar CDAccount::getMaturityDate(){
    return Calendar(maturityDate);
}
```

20.5.2 makeWithdrawl()

This performs a withdrawal only if the CD has matured.

```
TransactionReceipt CDAccount::makeWithdrawl(
    TransactionTicket transactionTicket
){
    Calendar today;
    double preTransactionBalance = accountBalance;

    if (today.before(maturityDate)){
        throw CDMaturityDateException(maturityDate.printDate());
    }

    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + transactionTicket.getCDTerm()
    );

    SavingsAccount::makeWithdrawl(transactionTicket);

    TransactionReceipt transactionReceipt(
        transactionTicket,
        getAccountType(),
        preTransactionBalance,
        accountBalance,
        true,
        " ",
        Calendar(maturityDate)
    );

    return transactionReceipt;
}
```

20.5.3 makeDeposit()

This performs a deposit only after maturity; otherwise throws an exception.

```
TransactionReceipt CDAccount::makeDeposit(
    TransactionTicket transactionTicket
){
    Calendar today;
    double preTransactionBalance = accountBalance;

    if (today.before(maturityDate)){
        throw InvalidAccountException("Cannot deposit into a CD after opening");
    }

    maturityDate = Calendar(
```

```

        today.getMonth(),
        today.getDay(),
        today.getYear() + transactionTicket.getCDTerm()
    );

    SavingsAccount::makeDeposit(transactionTicket);

    TransactionReceipt transactionReceipt(
        transactionTicket,
        getAccountType(),
        preTransactionBalance,
        accountBalance,
        true,
        " ",
        Calendar(maturityDate)
    );

    return transactionReceipt;
}

```

20.6 Formatting Functions

20.6.1 printAccount()

This prints the account details with maturity date.

```

string CDAccount::printAccount(){
    ostreamstream cdAccountString;
    cdAccountString << SavingsAccount::printAccount()
                    << right << setw(15)
                    << maturityDate.printDate();
    return cdAccountString.str();
}

```

20.6.2 printAccountReceipt()

This prints a transaction receipt including maturity date.

```

string CDAccount::printAccountReceipt(){
    ostreamstream cdAccountReceipt;
    cdAccountReceipt << BankAccount::printAccountReceipt()
                    << "\t\t\tMaturity Date: "
                    << maturityDate.printDate();
    return cdAccountReceipt.str();
}

```

20.7 Unit Testing

20.7.1 Testing Strategy

We have our unit tests to verify:

- Proper CD account creation
- Accurate account details
- Correct maturity date handling

20.7.2 Test Implementation

```
static void testCDAcct(){
    CDAccount testCDAccount(
        Depositor(),
        000000,
        100.0,
        true,
        Calendar(1,9,2026)
    );

    ASSERT(testCDAccount.getDepositor() != nullptr);
    ASSERT(testCDAccount.getAccountNumber() == 000000);
    ASSERT(testCDAccount.getAccountBalance() == 100.0);
    ASSERT(testCDAccount.getAccountStatus() == true);
    ASSERT(testCDAccount.getMaturityDate().getDay() == 1);
    ASSERT(testCDAccount.getMaturityDate().getMonth() == 9);
    ASSERT(testCDAccount.getMaturityDate().getYear() == 2026);
}
```

20.7.3 Test Runner

```
void testCDAccount(){
    TEST(testCDAcct);
}
```

20.8 Implementation

20.8.1 CDAccount.hpp

The header file should appear as follows:

```
#pragma once

#include "Stdafx.hpp"
#include "SavingsAccount.hpp"

/*
    CD Account Class
    Extends Savings Account. Clients make an intial deposit
    but cannot withdrawl or deposit until maturity date
*/
class CDAccount : public SavingsAccount{
public:
    // CD Account class constructor series
    CDAccount(); // No args constructor
    CDAccount(
        // CD Account key args constructor
        Depositor depositor, // depositor
        int accountNumber, // account number
        double accountBalance, // account balance
        bool accountStatus, // account status
        int cdTerm
    );
    CDAccount(
        // CD Account key args constructor
        Depositor depositor, // depositor
        int accountNumber, // account number
        double accountBalance, // account balance
        bool accountStatus, // account status

```

```

        Calendar maturityDate
    );
    // CD Account copy constructor
    CDAccount(const CDAccount& otherCDAccount);

    /* CDAccount getter series */
    Calendar getMaturityDate(); // return maturity date

    /* CD Account Transaction Series */
    TransactionReceipt makeWithdrawl(
        // Make withdrawl method overrides withdraw
        TransactionTicket transactionTicket // transaction ticket
    ) override;

    TransactionReceipt makeDeposit(
        // Make deposit method overrides deposit
        TransactionTicket transactionTicket // transaction ticket
    ) override;

    // CD Account print series
    std::string printAccount() override;
    std::string printAccountReceipt() override;

private:
    Calendar maturityDate; // maturity date
};

// Unit test
void testCDAccount();

```

20.8.2 CDAccount.cpp

The entire source file should appear as follows:

```

#include "CDAccount.hpp"

using namespace std;

/* CD Account constructor series */
CDAccount::CDAccount() : SavingsAccount (
    // CD Account No args constructor
    Depositor(), // null depositor
    000000, // account number
    0.0, // balance
    true // open
), maturityDate(Calendar()) {
    accountType = AccountType::CD;
}

CDAccount::CDAccount(
    // CD Account Key word args constructor
    Depositor depositor, // depositor
    int accountNumber, // account number
    double accountBalance, // account Balance
    bool accountStatus, // account status
    int cdTerm // cd term
) : SavingsAccount (
    depositor, // set depositor
    accountNumber, // set account number

```

```

        accountBalance, // set account balance
        accountStatus // set status
    ) {
        // Set account type
        accountType = AccountType::CD;
        // Compute maturity date
        Calendar today;
        maturityDate = Calendar(
            today.getMonth(),
            today.getDay(),
            today.getYear() + cdTerm
        );
    }

CDAccount::CDAccount(
    // CD Account Key word args constructor with maturity date
    Depositor depositor, // depositor
    int accountNumber, // account number
    double accountBalance, // account Balance
    bool accountStatus, // account status
    Calendar maturityDate
) : SavingsAccount (
    depositor, // set depositor
    accountNumber, // set account number
    accountBalance, // set account balance
    accountStatus // set status
), maturityDate(maturityDate) {
    // Set account type
    accountType = AccountType::CD;
}

CDAccount::CDAccount(
    // CD Account copy constructor
    const CDAccount& otherCDAccount
) : SavingsAccount(otherCDAccount){}

/* CD Account getter series */
Calendar CDAccount::getMaturityDate(){
    // CD account get maturity date. returns a calendar
    // object of the maturity date
    return Calendar(maturityDate); // return copy
}

/* CD Account Transaction Series */
TransactionReceipt CDAccount::makeWithdrawl(
    // CD Account Make Withdrawl method. check for maturity date
    // returns a transaction receipt
    TransactionTicket transactionTicket
) {
    // Get todays date
    Calendar today;
    double preTransactionBalance = accountBalance;

    // check for maturity
    if (today.before(maturityDate)){
        // throw maturity date exception
        throw CDMaturityDateException(maturityDate.printDate());
    }
}

```

```

    // Compute new Maturity Date
    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + transactionTicket.getCDTerm()
    );
    // Perform withdrawl and append receipt
    SavingsAccount::makeWithdrawl(transactionTicket);
    TransactionReceipt transactionReceipt = TransactionReceipt(
        // Cd Transaction receipt
        transactionTicket, // transaction Ticket
        getAccountType(),
        preTransactionBalance, // pre balance
        accountBalance, // post balance
        true, // succes
        " ",
        Calendar(maturityDate) // new maturity date
    );
    return transactionReceipt; // return receipt
}

TransactionReceipt CDAccount::makeDeposit(
    // CD Make deposit method. Returns an Invalid Account Exception
    TransactionTicket transactionTicket
){
    Calendar today; // todays date
    double preTransactionBalance = accountBalance; // copy balance

    if (today.before(maturityDate)){
        // Throw invalid account error
        throw InvalidAccountException("Cannot deposit into a CD after opening");
    }

    // Compute new Maturity Date
    maturityDate = Calendar(
        today.getMonth(),
        today.getDay(),
        today.getYear() + transactionTicket.getCDTerm()
    );
    // Perform withdrawl and append receipt
    SavingsAccount::makeDeposit(transactionTicket);
    TransactionReceipt transactionReceipt = TransactionReceipt(
        // Cd Transaction receipt
        transactionTicket, // transaction Ticket
        getAccountType(),
        preTransactionBalance, // pre balance
        accountBalance, // post balance
        true, // succes
        " ",
        Calendar(maturityDate) // new maturity date
    );
    return transactionReceipt; // return receipt
}

/* CD Accoount Formmate Series */
string CDAccount::printAccount(){
    ostringstream cdAccountString; // initialize string stream

```

```

        cdAccountString << SavingsAccount::printAccount()
                        << right << setw(15)
                        << maturityDate.printDate();
        return cdAccountString.str();
    }

string CDAccount::printAccountReceipt(){
    // CD account print account Receipt
    ostringstream cdAccountReceipt;
    cdAccountReceipt << BankAccount::printAccountReceipt()
                    << "\t\t\tMaturity Date: "
                    << maturityDate.printDate();
    return cdAccountReceipt.str();
}

// CD Account Unit Test
static void testCDAcct(){
    // unit test for CD Account creation
    CDAccount testCDAccount = CDAccount(
        // Test CD account
        Depositor(), // null depositor
        000000, // account number
        100.0 , // balance
        true, // account open
        Calendar(1,9,2026)
    );
    // Assert
    ASSERT(testCDAccount.getDepositor() != nullptr);
    ASSERT(testCDAccount.getAccountNumber() == 000000);
    ASSERT(testCDAccount.getAccountBalance() == 100.0);
    ASSERT(testCDAccount.getAccountStatus() == true);
    ASSERT(testCDAccount.getMaturityDate().getDay() == 1);
    ASSERT(testCDAccount.getMaturityDate().getMonth() == 9);
    ASSERT(testCDAccount.getMaturityDate().getYear() == 2026);
}

void testCDAccount(){
    // Unit Test for CD Account Class
    TEST(testCDAcct);
}

```

20.9 Summary

The CDAccount class models Certificate of Deposit behavior. By extending SavingsAccount and introducing maturity-based transaction rules, it provides:

- Enforced maturity rules for withdrawals and deposits
- Automatic maturity date computation
- Transaction receipts that include updated maturity date
- Consistent integration with the account hierarchy

Together with SavingsAccount and CheckingAccount, the CDAccount class completes a flexible, robust, and realistic banking account system.

21 CommercialAccount Class

21.1 Overview

Next we have the `CommercialAccount` class. This class represents a commercial banking account and extends the `CheckingAccount` base class. A commercial account is a combination of a `BankAccount` and a `CreditAccount`. Clients with commercial account can perform checking and credit transactions.

A `CommercialAccount`:

- Inherits checking account functionality
- Contains an integrated `CreditAccount`
- Allows centralized transaction logging

A `CommercialAccount` contains:

- All properties of a `CheckingAccount` (depositor, account number, balance, status)
- A `CreditAccount` object

This structure models real-world commercial accounts which typically combine checking and credit capabilities under one client relationship.

21.2 Design Rationale

The key ideas here are:

- Inheritance from `CheckingAccount` to reuse transaction functionality.
- Composition of a `CreditAccount` for additional credit operations.
- Single location for transaction receipt logging.
- Clear encapsulation of checking and credit subaccounts.

This ensures commercial accounts behave consistently with both checking and credit rules while maintaining centralized bookkeeping.

21.3 Class Declaration

Header Definitions Here we have our header declarations for `CommercialAccount`:

```
#pragma once
```

```
#include "Stdafx.hpp"
#include "Depositor.hpp"
#include "CheckingAccount.hpp"
#include "CreditAccount.hpp"
```

```
class CommercialAccount : public CheckingAccount {
public:
    CommercialAccount();
    CommercialAccount(CheckingAccount checkingAccount);
    CommercialAccount(const CommercialAccount& otherCommercialAccount);

    CreditAccount getCreditAccount();

    void addTransactionReceipt(
        const TransactionReceipt& transactionReceipt
    ) override;

private:
    CreditAccount creditAccount;
};
```

Unit Testing

```
void testCommercialAccount();
```

21.4 Constructors

21.4.1 Default Constructor

This initializes a commercial account with default checking and credit accounts.

```
CommercialAccount::CommercialAccount():
    CheckingAccount(CheckingAccount()),
    creditAccount(CreditAccount(CheckingAccount())) {
    accountType = AccountType::Commercial;
}
```

21.4.2 Parameterized Constructor

This initializes a commercial account with a specific checking account.

```
CommercialAccount::CommercialAccount(
    CheckingAccount commercialCheckingAccount
) : CheckingAccount(commercialCheckingAccount),
    creditAccount(commercialCheckingAccount) {
    accountType = AccountType::Commercial;
}
```

21.4.3 Copy Constructor

This creates a deep copy of an existing commercial account.

```
CommercialAccount::CommercialAccount(
    const CommercialAccount& otherCommercialAccount
) : CheckingAccount(otherCommercialAccount),
    creditAccount(otherCommercialAccount.creditAccount) {}
```

21.5 Core Commercial Functions

21.5.1 getCreditAccount()

This returns a copy of the integrated CreditAccount.

```
CreditAccount CommercialAccount::getCreditAccount(){
    return CreditAccount(creditAccount);
}
```

21.5.2 addTransactionReceipt()

This overrides receipt logging to maintain a centralized location.

```
void CommercialAccount::addTransactionReceipt(
    const TransactionReceipt& transactionReceipt
){
    CheckingAccount::addTransactionReceipt(transactionReceipt);
}
```

21.6 Unit Testing

21.6.1 Testing Strategy

The unit tests verify:

- Proper creation of commercial accounts
- Correct initialization of checking and credit subaccounts
- Accurate transaction receipt logging

21.6.2 Test Implementation

```
static void testCommercialAcct(){
    CommercialAccount testCommercialAccount(
        CheckingAccount(
            Depositor(),
            000000,
            100.0,
            true
        )
    );

    testCommercialAccount.addTransactionReceipt(TransactionReceipt());
    testCommercialAccount.addTransactionReceipt(TransactionReceipt());

    ASSERT(testCommercialAccount.getDepositor() != nullptr);
    ASSERT(testCommercialAccount.getAccountStatus());
    ASSERT(testCommercialAccount.getAccountNumber() == 000000);
    ASSERT(testCommercialAccount.getAccountBalance() == 100.0);
    ASSERT(testCommercialAccount.getAccountType() == "Commercial");

    ASSERT(testCommercialAccount.creditAccount.getDepositor() != nullptr);
    ASSERT(testCommercialAccount.creditAccount.getAccountNumber() == 000000);
    ASSERT(testCommercialAccount.creditAccount.getAccountType() == "Commercial");
    ASSERT(testCommercialAccount.creditAccount.getAccountBalance() == 0.0);
    ASSERT(testCommercialAccount.creditAccount.getAccountStatus());

    ASSERT(testCommercialAccount.getTransactionReceipts() == 2);
}
```

21.6.3 Test Runner

```
void testCommercialAccount(){
    TEST(testCommercialAcct);
}
```

21.7 Implementation

21.7.1 CommercialAccount.hpp

The entire header file should appear as follows:

```
#pragma once

#include "Stdafx.hpp"
#include "Depositor.hpp"
#include "CheckingAccount.hpp"
#include "CreditAccount.hpp"
```

```

/*
    Commercial account class
    The extends checking account, can performing checking transactions
    and contains a credit account.
*/
class CommercialAccount : public CheckingAccount {
public:
    // Commercial Account constructor series
    CommercialAccount(); // no args constructor
    CommercialAccount(
        // Commercial Account key word args constructor
        CheckingAccount checkingAccount // checking account
    );
    // Commercial account copy constructor
    CommercialAccount(
        const CommercialAccount& otherCommercialAccount // copy
    );

    // Commercial Account getter Series
    CreditAccount getCreditAccount(); // return credit account

    /* Commercial Account Logging Series */
    void addTransactionReceipt(
        const TransactionReceipt& transactionReceipt // receipt
    ) override;

private:
    CreditAccount creditAccount; // credit account
};

// Unit Testing
void testCommercialAccount();

```

21.7.2 CommercialAccount.cpp

The entire source file should appear as followed:

```

#include "CommercialAccount.hpp"

using namespace std;

// Commercial Account Constructor Series
CommercialAccount::CommercialAccount():
    // Commercial Account no args constructor
    CheckingAccount(CheckingAccount()), // Null checking account
    creditAccount(CreditAccount(CheckingAccount())) {
    accountType = AccountType::Commercial; // set account type to commercial
} // Null credit account

CommercialAccount::CommercialAccount(
    // Commercial Account key word args constructor
    CheckingAccount commercialCheckingAccount // Checking account
) : CheckingAccount(commercialCheckingAccount), // set checking
    creditAccount(commercialCheckingAccount){
    accountType = AccountType::Commercial; // set account type to commercial
}

CommercialAccount::CommercialAccount(
    const CommercialAccount& otherCommercialAccount

```

```

) : CheckingAccount(otherCommercialAccount), // bank account copy constructor
    creditAccount(otherCommercialAccount.creditAccount){} // credit account

/* Commercial Account Getter Series */
CreditAccount CommercialAccount::getCreditAccount(){
    return CreditAccount(creditAccount); // return copy
}

/* Commercial Account Logging Series */
void CommercialAccount::addTransactionReceipt(
    const TransactionReceipt& transactionReceipt
){
    // Overriden add transaction receipts method adds transaction receipts to
    // one account as single location
    CheckingAccount::addTransactionReceipt(transactionReceipt); // add receipt to checking
}

/* Unit Testing */
static void testCommercialAcct(){
    // Commercial account unit test. tests commercial account creation
    CommercialAccount testCommercialAccount = CommercialAccount(
        CheckingAccount(
            Depositor(), // depositor
            000000, // account Number
            100.0, // account balance
            true // account open
        )
    );
    // test add transaction receipt
    testCommercialAccount.addTransactionReceipt(
        //Add transaction receipt
        TransactionReceipt()
    );
    testCommercialAccount.addTransactionReceipt(
        // Add transaction receipt
        TransactionReceipt()
    );
    // Assertions
    ASSERT(testCommercialAccount.getDepositor() != nullptr);
    ASSERT(testCommercialAccount.getAccountStatus());
    ASSERT(testCommercialAccount.getAccountNumber() == 000000);
    ASSERT(testCommercialAccount.getAccountBalance() == 100.0);
    ASSERT(testCommercialAccount.getAccountType() == "Commercial");
    ASSERT(testCommercialAccount.creditAccount.getDepositor() != nullptr);
    ASSERT(testCommercialAccount.creditAccount.getAccountNumber() == 000000);
    ASSERT(testCommercialAccount.creditAccount.getAccountType() == "Commercial");
    ASSERT(testCommercialAccount.creditAccount.getAccountBalance() == 0.0);
    ASSERT(testCommercialAccount.creditAccount.getAccountStatus());
    ASSERT(testCommercialAccount.getTransactionReceipts() == 2);
}

void testCommercialAccount(){
    // Unit Test for commercial account
    TEST(testCommercialAcct);
}

```

21.8 Summary

The `CommercialAccount` class models commercial banking accounts that combine checking and credit capabilities. By extending `CheckingAccount` and composing a `CreditAccount`, it provides:

- Integrated checking and credit functionality
- Centralized transaction receipt logging
- Safe copying and initialization of all subaccounts
- Realistic modeling of commercial account behavior

This completes a robust commercial account design within the banking system.

22 BrokerageAccount Class

22.1 Overview

Next we have the `BrokerageAccount` class. This class represents a brokerage account and extends the `CommercialAccount` base class. A `BrokerageAccount` is a `CommercialAccount` with a portfolio. Since it contains a `CreditAccount`, clients with a brokerage account can use their credit accounts to buy on margin.

A `BrokerageAccount` has:

- Inherits commercial account functionality
- Contains a portfolio of `Investment` objects (stocks, bonds, options)
- Provides aggregated views of total value and share counts
- Allows printing a detailed account and portfolio summary

This structure models real-world brokerage accounts where clients manage multiple types of investments under a single account.

22.2 Design Rationale

The key ideas here are:

- Inheritance from `CommercialAccount` for unified checking and credit functionality.
- Composition of a polymorphic portfolio of `Investment` objects.
- Aggregation methods (`getTotalValue`, `getTotalShares`) for portfolio analysis.
- Centralized formatted output for account and portfolio reporting.
- Prevention of deep copying to maintain unique pointers in the portfolio.

This design allows a brokerage account to manage diverse investments while ensuring accurate and safe memory management using smart pointers.

22.3 Class Declaration

Header Definitions

```
#pragma once

#include "Stdafx.hpp"
#include "CommercialAccount.hpp"
#include "Investment.hpp"
#include "Stock.hpp"
#include "Bond.hpp"
```

```
#include "Option.hpp"

class BrokerageAccount: public CommercialAccount {
public:
    BrokerageAccount();
    BrokerageAccount(CommercialAccount brokerageCommercialAccount);

    BrokerageAccount(const BrokerageAccount&) = delete;

    void addInvestment(std::unique_ptr<Investment> investment);

    double getTotalValue() const;
    int getTotalShares() const;

    std::string printPortfolio();
    std::string printTotalValue() const;

private:
    std::vector<std::unique_ptr<Investment>> investmentPortfolio;
};

// Unit Test
void testBrokerageAccount();
```

22.4 Constructors

22.4.1 Default Constructor

This initializes a brokerage account with default commercial account.

```
BrokerageAccount::BrokerageAccount():
    CommercialAccount(CommercialAccount())
{
    accountType = AccountType::Brokerage;
}
```

22.4.2 Parameterized Constructor

This initializes a brokerage account using an existing commercial account.

```
BrokerageAccount::BrokerageAccount(
    CommercialAccount brokerageCommercialAccount
): CommercialAccount(brokerageCommercialAccount)
{
    accountType = AccountType::Brokerage;
}
```

22.4.3 Deleted Copy Constructor

Deep copying is deleted to prevent issues with unique pointers in the portfolio.

```
BrokerageAccount(const BrokerageAccount&) = delete;
```

22.5 Core Brokerage Functions

22.5.1 addInvestment()

Adds an Investment object to the brokerage portfolio.

```
void BrokerageAccount::addInvestment(unique_ptr<Investment> investment) {
    investmentPortfolio.emplace_back(std::move(investment));
}
```

22.5.2 getTotalValue()

Computes the total market value of all investments.

```
double BrokerageAccount::getTotalValue() const {
    double total = 0.0;
    for (const auto& inv : investmentPortfolio) {
        total += inv->getTotalValue();
    }
    return total;
}
```

22.5.3 getTotalShares()

Computes the total number of shares across all investments.

```
int BrokerageAccount::getTotalShares() const {
    int total = 0;
    for (const auto& inv : investmentPortfolio) {
        total += inv->getNumberOfShares();
    }
    return total;
}
```

22.5.4 printPortfolio()

Generates a formatted summary of the account and its investment portfolio.

```
string BrokerageAccount::printPortfolio() {
    ostream portfolioString;
    portfolioString << left
        << setw(21) << "\nClient Name"
        << setw(12) << "SSN"
        << setw(10) << "Acct Num"
        << setw(15) << "Acct Type"
        << setw(10) << "Status"
        << right
        << setw(12) << "Balance"
        << setw(15) << "Balance Due"
        << "\n"
        << printAccount()
        << setw(15)
        << CommercialAccount::getCreditAccount().printAccountBalance()
        << "\n\n\t\t"
        << string(20, '*') << " Investment Portfolio "
        << string(20, '*') << "\n";

    for (const auto& inv: investmentPortfolio){
        portfolioString << inv->print() << "\n";
    }
    portfolioString << string(100, '*')
        << "\n\t\t\t\t\t\t\tTotal Value: "
        << printTotalValue()
        << "\n\t\t\t\t\t\t\tTotal Number of Shares: "
        << getTotalShares() << "\n"
        << string(100, '*') << "\n\n"
        << "\t\t\t\t\tTransaction History\n\n"
        << setw(10) << "Date"
        << setw(20) << "Transaction"
        << setw(15) << "Amount"
```



```

        << setw(10) << "Status"
        << setw(12) << "Balance"
        << setw(25) << "Reason for Failure\n";

    for (TransactionReceipt receipt: transactionReceipts){
        portfolioString << receipt.printReceipt();
    }
    return portfolioString.str();
}

```

22.5.5 printTotalValue()

This returns the formatted total value of all investments with commas as all our other print transaction balance functions.

```

string BrokerageAccount::printTotalValue() const {
    ostringstream totalValue;
    totalValue << "$" << fixed << setprecision(2) << getTotalValue();
    string formattedBalance = totalValue.str();
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){
        formattedBalance.insert(i, ",");
    }
    return formattedBalance;
}

```

22.6 Unit Testing

22.6.1 Testing Strategy

The unit tests verify:

- Polymorphic behavior of `Investment` objects
- Correct aggregation of total value and shares
- Portfolio handling with multiple investment types

22.6.2 Test Implementation

```

static void testInvestment(){
    unique_ptr<Investment> investment =
        make_unique<Stock>("MSFT", 5, 200);
    ASSERT(investment->getNumberOfShares() == 5);
    ASSERT_APPROX_EQUAL(investment->getTotalValue(), 1000.00, 0.001);
}

static void testBrokerageAccountTotals(){
    BrokerageAccount testBrokerageAccount;
    testBrokerageAccount.addInvestment(make_unique<Stock>("APPL", 10, 150.00));
    testBrokerageAccount.addInvestment(make_unique<Stock>("GOOG", 2, 1000.00));

    ASSERT(testBrokerageAccount.getTotalShares() == 12);
    ASSERT_APPROX_EQUAL(testBrokerageAccount.getTotalValue(), 3500.00, 0.001);
}

static void testMixedPortfolio(){
    BrokerageAccount testBrokerageAccount;
    testBrokerageAccount.addInvestment(make_unique<Stock>("AAPL", 10, 100.0));
    testBrokerageAccount.addInvestment(make_unique<Bond>(1, 1000.0, 0.05, 3));
}

```

```
    ASSERT_APPROX_EQUAL(testBrokerageAccount.getTotalValue(), 2000.0, 0.001);
}
```

22.6.3 Test Runner

```
void testBrokerageAccount(){
    TEST(testInvestment);
    TEST(testBrokerageAccountTotals);
    TEST(testMixedPortfolio);
}
```

22.7 Implementation

22.7.1 BrokerageAccount.hpp

The entire header file should appear as followed:

```
#pragma once

#include "Stdafx.hpp"
#include "CommercialAccount.hpp"
#include "Investment.hpp"
#include "Stock.hpp"
#include "Bond.hpp"
#include "Option.hpp"

/*
    Brokerage Account Class
    extends Commercial account. also contains a portfolio vector of
    stock, bonds, and options investment interface objects
*/
class BrokerageAccount: public CommercialAccount{
public:
    // Brokerage account constructor series
    BrokerageAccount();
    BrokerageAccount(
        // Brokerage Key word args constructor
        CommercialAccount brokerageCommercialAccount
    );
    // Deep clone
    BrokerageAccount(
        const BrokerageAccount&
    ) = delete;

    // Brokerage Helper functions
    void addInvestment(std::unique_ptr<Investment> investment);

    /* Brokerage account transaction series */
    double getTotalValue() const; // returns total value
    int getTotalShares() const; // return total shares

    // Print Account
    std::string printPortfolio(); // returns account string
    std::string printTotalValue() const; // returns formatted value
private:
    std::vector<std::unique_ptr<Investment>> investmentPortfolio; // portfolio
};
```

```
// Unit Test
void testBrokerageAccount();
```

22.7.2 BrokerageAccount.cpp

The source file should appear as followed:

```
#include "BrokerageAccount.hpp"
#include "RandomGenerator.hpp"

using namespace std;

/* Brokerage Account Constructor */
BrokerageAccount::BrokerageAccount():
    // Brokerage Account no args constructor
    CommercialAccount(CommercialAccount())
{
    // Set Account type
    accountType = AccountType::Brokerage;
}

BrokerageAccount::BrokerageAccount(
    // Brokerage account Key word args constructor
    CommercialAccount brokerageCommercialAccount
): CommercialAccount(brokerageCommercialAccount)
{
    // Set Account Type
    accountType = AccountType::Brokerage;
}

/* Brokerage account getter series */
void BrokerageAccount::addInvestment(
    unique_ptr<Investment> investment
) {
    // Brokerage account add investment. add an investment to the portfolio
    investmentPortfolio.emplace_back(std::move(investment));
}

double BrokerageAccount::getTotalValue() const {
    // Brokerage account get total value. Aggregates all the
    // investment values
    double total = 0.0; // total counter
    for (const auto& inv : investmentPortfolio) {
        // loop through pointers and add total value
        total += inv->getTotalValue(); // add value
    }
    return total; // return total
}

int BrokerageAccount::getTotalShares() const {
    // Brokerage account get total shares. aggregates the total
    // number of shares
    int total = 0; // counter
    for (const auto& inv : investmentPortfolio) {
        // Loop through pointers and add shares
        total += inv->getNumberOfShares(); // add shares
    }
    return total; // return total shares
}
```

```

/* Brokerage account print Series */
string BrokerageAccount::printPortfolio() {
    // Brokerage account print account method. returns the
    // account string
    ostringstream portfolioString; // initialize string stream
    // Build string with boiler plate
    portfolioString << left
        << setw(21) << "\nClient Name"
        << setw(12) << "SSN"
        << setw(10) << "Acct Num"
        << setw(15) << "Acct Type"
        << setw(10) << "Status"
        << right
        << setw(12) << "Balance"
        << setw(15) << "Balance Due"
        << "\n"
        << printAccount()
        << setw(15)
        << CommercialAccount::getCreditAccount().printAccountBalance()
        << "\n\n\t\t"
        << string(20, '*') << " Investment Portfolio "
        << string(20, '*') << "\n";
    for (const auto& inv: investmentPortfolio){
        // loop through portfolio and print
        portfolioString << inv->print() << "\n";
    }
    portfolioString << string(100, '*')
        << "\n\t\t\t\t\t\t\t\t\t\tTotal Value: "
        << printTotalValue() // print total value
        << "\n\t\t\t\t\t\t\t\t\t\tTotal Number of Shares: "
        << getTotalShares() << "\n" // print total shares
        << string(100, '*') << "\n\n"
        // Print transaction histor
        << "\t\t\t\t\t\t\tTransaction History\n\n"
        << setw(10) << "Date"
        << setw(20) << "Transaction"
        << setw(15) << "Amount"
        << setw(10) << "Status"
        << setw(12) << "Balance"
        << setw(25) << "Reason for Failure\n";
    for (TransactionReceipt receipt: transactionReceipts){
        // print receipt
        portfolioString << receipt.printReceipt();
    }
    return portfolioString.str(); // return string
}

string BrokerageAccount::printTotalValue() const {
    ostringstream totalValue; // write string stream
    totalValue << "$" << fixed << setprecision(2) << getTotalValue();
    // convert to string
    string formattedBalance = totalValue.str();

    // Insert commas manually
    int dot = formattedBalance.find('.');
    for (int i = dot - 3; i > 1; i -= 3){

```

```

        formattedBalance.insert(i, ",");
    }
    return formattedBalance; // return balance
}

/* Unit Testing Series */
static void testInvestment(){
    // Unit test for investment polymorphism
    unique_ptr<Investment> investment =
        make_unique<Stock>(
            "MSFT", // Microsoft
            5, // number of share
            200 // price per share
        );
    // ASSERT
    ASSERT(investment->getNumberOfShares() == 5);
    ASSERT_APPROX_EQUAL(
        investment->getTotalValue(), // total value
        1000.00,
        0.001
    );
}

static void testBrokerageAccountTotals(){
    // Unit test for the aggregation of investments
    BrokerageAccount testBrokerageAccount; // test account
    // ADD investments
    testBrokerageAccount.addInvestment(
        // Add apple share
        make_unique<Stock>(
            "APPL", // Apple
            10, // 10 shares
            150.00 // price per share
        )
    );
    testBrokerageAccount.addInvestment(
        // Add google share
        make_unique<Stock>(
            "GOOG", // google
            2, // 2 shares
            1000.00 // per share
        )
    );
    // ASSERT
    ASSERT(testBrokerageAccount.getTotalShares() == 12);
    ASSERT_APPROX_EQUAL(
        testBrokerageAccount.getTotalValue(),
        3500.00,
        0.001
    );
}

static void testMixedPortfolio(){
    // Unit test for portfolio with different investments
    BrokerageAccount testBrokerageAccount; // test account

    // Add test investments
    testBrokerageAccount.addInvestment(

```

```

        // Test stock
        make_unique<Stock>(
            // Test stock
            "AAPL", // apple
            10, // number of shares
            100.0 // price per share
        )
    );
    testBrokerageAccount.addInvestment(
        // Test bond
        make_unique<Bond>(
            1, // one bond
            1000.0, // face value
            0.05, // coupon
            3 // years to maturity
        )
    );
    // Assert
    ASSERT_APPROX_EQUAL(
        testBrokerageAccount.getTotalValue(),
        2000.0, // expected
        0.001 // tolerance
    );
}

void testBrokerageAccount(){
    // Unit test for brokerage account class
    TEST(testInvestment);
    TEST(testBrokerageAccountTotals);
    TEST(testMixedPortfolio);
}

```

22.8 Summary

The `BrokerageAccount` class models a brokerage account that manages a diversified investment portfolio. By extending `CommercialAccount` and containing a polymorphic investment vector, it provides:

- Aggregated total value and share calculations
- Centralized printing of portfolio and transaction history
- Safe memory management using `unique_ptr`
- Realistic modeling of brokerage account behavior

This completes the account hierarchy, supporting savings, checking, CD, commercial, and brokerage accounts within the banking system.

23 RandomGenerator Class

23.1 Overview

The `RandomGenerator` class provides utilities to generate random numbers, bank accounts, investment objects, and transactions. It is designed for simulation and testing purposes within the banking system. This is the core functionality needed to randomize our data and the transaction processes.

Key responsibilities:

- Generate random integers and doubles

- Generate account numbers
- Generate random accounts of various types (Checking, Savings, CD, Commercial, Brokerage)
- Generate random investment objects (Stock, Bond, Option)
- Generate random portfolios
- Generate random bank transactions and credit card transactions
- Support unit testing and simulation of banking operations

A key thing to note here is how we select the ticker symbols for our stocks. Notices that the tickers are listed here and these are the only securities could appear. The allows us to not have to specify which stock to put in the portfolio because the system does it for us.

23.2 Design Rationale

The class centralizes all random generation logic:

- Uses `std::mt19937` for reliable pseudo-random number generation
- Encapsulates account and transaction creation logic for simulation
- Returns `unique_ptr` to avoid memory leaks and enforce ownership semantics
- Supports polymorphic behavior when generating accounts and investments

23.3 Class Declaration

```
#pragma once

#include <random>
#include "Investment.hpp"
#include "CheckingAccount.hpp"
#include "SavingsAccount.hpp"
#include "CommercialAccount.hpp"
#include "BrokerageAccount.hpp"
#include "CreditAccount.hpp"
#include "CDAccount.hpp"
#include "TransactionTicket.hpp"

class RandomGenerator{
public:
    static std::mt19937& randomNumberGenerator();
    double randomDouble(double min, double max);
    int randomInteger(int min, int max);
    int generateAccountNumber();
    std::unique_ptr<BankAccount> generateAccount(const Depositor& depositor);
    std::unique_ptr<Investment> generateRandomInvestment();
    void generateRandomPortfolio(BrokerageAccount& brokerageAccount, int size);
    CardLevel generateCardLevel();
    TransactionTicket generateBankAccountTransaction(int accountNumber);
    Check generateCheckingTransaction(int accountNumber);
    CreditCard generateCreditAccountTransaction(int accountNumber);
};

// Unit test
void testRandomGenerator();
```

23.4 Random Number Generators

- `randomNumberGenerator()`: returns a static `mt19937` generator
- `randomDouble(min, max)`: returns a double in $[min, max]$
- `randomInteger(min, max)`: returns an integer in $[min, max]$

```
std::mt19937& RandomGenerator::randomNumberGenerator() { ... }
double RandomGenerator::randomDouble(double min, double max) { ... }
int RandomGenerator::randomInteger(int min, int max) { ... }
```

23.5 Account Generation

- `generateAccountNumber()`: sequential account number generator
- `generateAccount(const Depositor&)`: generates a random account type, initializing:
 - `CheckingAccount`, `SavingsAccount`, `CDAccount`, `CommercialAccount`, `BrokerageAccount`
- `generateRandomPortfolio(BrokerageAccount&, size)`: adds `size` random investments to a brokerage account

23.6 Investment Generation

- `generateRandomInvestment()`: generates a random `Stock`, `Bond`, or `Option`
- `generateCardLevel()`: returns a random `CardLevel` (`BASIC`, `GOLD`, `PLATINUM`)

23.7 Transaction Generation

- `generateBankAccountTransaction(accountNumber)`: returns a random bank transaction ticket
- `generateCheckingTransaction(accountNumber)`: returns a random check for a checking account
- `generateCreditAccountTransaction(accountNumber)`: returns a random credit card transaction

23.8 Unit Testing

23.8.1 Random Values

```
static void testRandom(){
    RandomGenerator testRandomGenerator;
    double testRandomDouble = testRandomGenerator.randomDouble(5.0, 10.0);
    int testRandomInteger = testRandomGenerator.randomInteger(5, 10);
    ASSERT(testRandomDouble > 5.0 && testRandomDouble < 10.0);
    ASSERT(testRandomInteger > 5 && testRandomInteger < 10);
}
```

23.8.2 Random Account Generation

```
static void testGenerateAccount(){
    Depositor testDepositor(Name("Max", "Mustermann"), "123456789");
    RandomGenerator testRandomGenerator;
    unique_ptr<BankAccount> testRandomAccount = testRandomGenerator.generateAccount(testDepositor);
    ASSERT(testRandomAccount->getDepositor().getDepositorName().printName() == "Max Mustermann");
    ASSERT(testRandomAccount->getDepositor().getDepositorSSN() == "123456789");
    ASSERT(testRandomAccount->getAccountBalance() > 0.0);
    ASSERT(testRandomAccount->getAccountNumber() != nullptr);
    ASSERT(testRandomAccount->getAccountStatus() == true);
    ASSERT(testRandomAccount->getAccountType() != nullptr);
}
```


23.8.3 Random Portfolio Generation

```
static void testGenerateRandomPortfolio(){
    RandomGenerator randomGenerator;
    auto brokerageAccount = make_unique<BrokerageAccount>(CommercialAccount());
    randomGenerator.generateRandomPortfolio(*brokerageAccount, randomGenerator.randomInteger(3, 10));
    ASSERT(brokerageAccount->getTotalShares() > 0);
    ASSERT(brokerageAccount->getTotalValue() > 0);
}
```

23.8.4 Test Runner

```
void testRandomGenerator(){
    TEST(testRandom);
    TEST(testGenerateAccount);
    TEST(testGenerateRandomPortfolio);
}
```

23.9 Random Generator Implementation

23.9.1 RandomGenerator.hpp

This is the RandomGenerator.hpp in its entirety:

```
#pragma once

#include <random>
#include "Investment.hpp"
#include "CheckingAccount.hpp"
#include "SavingsAccount.hpp"
#include "CommercialAccount.hpp"
#include "BrokerageAccount.hpp"
#include "CreditAccount.hpp"
#include "CDAccount.hpp"
#include "TransactionTicket.hpp"

/*
    Random Generator class
    this class generate random numbers to be used on accounts an
    transactions. It also generates random accounts, and transactions
    for the simulation
*/
class RandomGenerator{
public:
    // Seeding function
    static std::mt19937& randomNumberGenerator();
    // Random Double
    double randomDouble(double min, double max);
    // random Integer
    int randomInteger(int min, int max);
    // generate account number
    int generateAccountNumber();
    // Generate random account
    std::unique_ptr<BankAccount> generateAccount(
        // Generate an account
        const Depositor& depositor
    );
    // Random Investment generator
    std::unique_ptr<Investment> generateRandomInvestment();
    // Generate random Portfolio
```

```

        void generateRandomPortfolio(
            // generate random portfolio
            BrokerageAccount& brokerageAccount, // account
            int size // size of portfolio
        );
        // Generate Card level
        CardLevel generateCardLevel();
        // Transaction generator series
        TransactionTicket generateBankAccountTransaction(int accountNumber);
        Check generateCheckingTransaction(int accountNumber);
        CreditCard generateCreditAccountTransaction(int accountNumber);
};

// Unit test
void testRandomGenerator();

```

23.9.2 RandomGenerator.hpp

This is the entire source file in it's entirety:

```

#include "RandomGenerator.hpp"

using namespace std;

/* Random Number generator Series */
std::mt19937& RandomGenerator::randomNumberGenerator(){
    // Random Generator random number sedding function
    // sets the seeding variable using the mt19937
    static std::mt19937 gen(random_device{}());
    return gen; // return gen
}

double RandomGenerator::randomDouble(
    // Random Generator random double generator
    // generates a random double number between a provided
    // random
    double min, // min number
    double max // max number
){
    return uniform_real_distribution<double>(
        // Uniform distribution
        min, // min
        max // max
    )(randomNumberGenerator()); // random number generator
}

int RandomGenerator::randomInteger(
    // Random Generator Random Integer generator. Generate a random
    // interger number between two numbers
    int min, // min number
    int max // max number
) {
    // Uniform distribution
    return uniform_int_distribution<int>(
        min, // minimum number
        max // maximum number
    )(randomNumberGenerator()); // random number generator
}

```

```

int RandomGenerator::generateAccountNumber(){
    // Generate Account Number. Lowers the probability
    static int nextAccountNumber = 10000; // initial number
    return nextAccountNumber++; // return initial number plus one
}

unique_ptr<BankAccount>
RandomGenerator::generateAccount(const Depositor& client){
    // Random Generator. Random Account generator
    // account
    Depositor depositor(client); // copy depositor

    int randomAccountNumber = generateAccountNumber();
    double accountBalance = randomDouble(100.00, 100000.00);
    bool open = true;
    int type = randomInteger(1,5);

    switch (type){
        // Context switch on account type
        case 1:{
            // 1 for checking Account
            string accountNumber = "1" + to_string(randomAccountNumber);
            return make_unique<CheckingAccount>(
                // generate Checking account
                depositor, // depositor
                stoi(accountNumber), // account number
                accountBalance, // account balance
                open // account open
            );
        }
        case 2: {
            // 2 for savings account
            string accountNumber = "2" + to_string(randomAccountNumber);
            return make_unique<SavingsAccount>(
                // generate savings account
                depositor, // depositor
                stoi(accountNumber), // account number
                accountBalance, // account balance
                open // account open
            );
        }
        case 3:{
            // 3 for CD account
            string accountNumber = "3" + to_string(randomAccountNumber);
            return make_unique<CDAccount>(
                // generate CD account
                depositor, //depositor
                stoi(accountNumber), // account number
                accountBalance, // account balance
                open, // account Open
                Calendar(
                    // Random date
                    randomInteger(1, 12), // random month
                    randomInteger(1, 29), // random day
                    randomInteger(2019, 2029) // random year
                )
            );
        }
    }
}

```

```

    case 4: {
        // 4 for Commercial account
        string accountNumber = "4" + to_string(randomAccountNumber);
        CheckingAccount checkingAccount(
            // Commercial Checking account
            depositor, // depositor
            stoi(accountNumber), // account number
            accountBalance, // account balance
            open // account open
        );
        // return commercial account
        return make_unique<CommercialAccount>(checkingAccount);
    }
    case 5: {
        // 5 for brokerage account
        string accountNumber = "5" + to_string(randomAccountNumber);
        CheckingAccount checkingAccount(
            // Checking Account
            depositor, // depositor
            stoi(accountNumber), // account number
            accountBalance, // account balance
            open // account open
        );
        // Initialize commercial account
        CommercialAccount commercialAccount(checkingAccount);
        auto brokerageAccount = make_unique<BrokerageAccount>(commercialAccount);

        // generate portfolio
        generateRandomPortfolio(
            *brokerageAccount, // brokerage account
            randomInteger(1, 10)
        );
        // Return brokerage account
        return brokerageAccount;
    }
    default:
        // Return null account
        return make_unique<BankAccount>(); // not nullptr
}
}

unique_ptr<Investment>
RandomGenerator::generateRandomInvestment(){
    // Random Generator generate random investment method. generates
    // a random investment transactions
    int type = randomInteger(1,3); // generate random int

    if (type == 1){
        // 1 for stock
        static const vector<string> tickers = {
            "AAPL", // apple
            "MSFT", // microsoft
            "GOOG", // google
            "AMZN", // amazon
            "CDNS", // cadence design
            "NFLX", // netflix
            "DIS", // disney
            "JPM", // jp morgan

```

```

        "AMD", // amd
        "BA", // boeing
        "BLK", // blackrock
        "MU", // micron technologies
        "VZ" // verizon
    };

    return make_unique<Stock>(
        // return stock ptr
        tickers[randomInteger(0, tickers.size() - 1)], // random ticker
        randomInteger(1, 500), // random quantity
        randomDouble(50.0, 1000.0) // random price
    );
}

if (type == 2){
    // 2 for bond
    return make_unique<Bond>(
        randomInteger(1, 50), // quantity
        1000.0, // face Value
        randomDouble(0.02, 0.09), // coupon rate
        randomInteger(1, 30) // random years
    );
}

return make_unique<Option>(
    // Return option by default
    // Option type
    randomInteger(0, 1) ? OptionType::Call : OptionType::Put,
    randomInteger(1, 20), // number of contracts
    randomDouble(1.0, 15.0), // premium
    100 // contract size
);
}

TransactionTicket RandomGenerator::generateBankAccountTransaction(int accountNumber){
    // Random Generates a random bank account transaction
    TransactionTicket transactionTicket; // initialize transaction ticket
    int transactionType = randomInteger(1,7);
    double transactionAmount = randomDouble(-50, 25000);

    switch(transactionType){
        // Balance
        case 1:
            return transactionTicket = TransactionTicket(
                TransactionType::Balance, // Balance
                accountNumber, // account Numbher
                0.0, // transaction amount
                Calendar(), // time not
                0
            );
        case 2:
            // Return Deposit transaction
            return transactionTicket = TransactionTicket(
                TransactionType::Deposit, // deposit
                accountNumber, // account number
                transactionAmount, // transaction amount
                Calendar(), // time now
            );
    }
}

```

```

        randomInteger(6, 24) // no term
    );
case 3:
    // Return Withdrawl Transaction
    return transactionTicket = TransactionTicket(
        TransactionType::Withdrawl, // withdrawl
        accountNumber, // account Number
        transactionAmount, // transaction amount
        Calendar(), // time now
        randomInteger(6, 24) // no new term
    );
case 4:
    // Return Close Account
    return transactionTicket = TransactionTicket(
        TransactionType::Close, // transaction
        accountNumber, // account Number
        0.0, // transaction amount
        Calendar(), // time now
        0 // no new term
    );
case 5:
    // Return Open Account
    return transactionTicket = TransactionTicket(
        TransactionType::Open, // transaction type
        accountNumber, // accountNumber
        0.0, // no amount
        Calendar(), // time now
        0 // no term
    );
case 6:
    // Return transaction history
    return transactionTicket = TransactionTicket(
        TransactionType::TransactionHistory, // transaction type
        accountNumber, // account number
        0.0, // no amounty
        Calendar(), // time now
        0
    );
case 7:
    // return Client Info
    return transactionTicket = TransactionTicket(
        TransactionType::ClientInfo, // transaction Type
        accountNumber,
        0.0, // // no amount
        Calendar(),
        0
    );
default:
    // return null ticket
    return TransactionTicket();
}
}

void RandomGenerator::generateRandomPortfolio(
    // Random generator generate random portforlio. Generates a random
    // portfolio
    BrokerageAccount& brokerageAccount, // brokerage account

```

```

    int size // size
){
    for (int i = 0; i < size; i++){
        // Generate Random Investment
        brokerageAccount.addInvestment(generateRandomInvestment());
    }
}

Check RandomGenerator::generateCheckingTransaction(int accountNumber){
    // Random Generate Checking transaction function. Returns a check for
    // a checking transaction
    Calendar today; // Get time now
    Check checkToClear = Check(
        accountNumber, // account Number
        randomDouble(-10, 30000), // check amount
        Calendar(
            randomInteger(1, 12),
            randomInteger(1, 28),
            randomInteger(
                today.getYear() - 1,
                2026
            )
        )
    );
    return checkToClear; // return check to clear
}

CreditCard RandomGenerator::generateCreditAccountTransaction(int accountNumber){
    // Random Generator generate credit account transaction
    CardLevel cardLevel; // initialize card level
    int level = randomInteger(1, 3);

    // Determine cardlevel
    switch(level){
        case 1:
            // Random 1
            cardLevel = CardLevel::BASIC; // set to basic
            break;
        case 2:
            // Random 2
            cardLevel = CardLevel::GOLD; // set to gold
            break;
        case 3:
            // random 3
            cardLevel = CardLevel::PLATINUM; // set platinum
            break;
    }

    CreditCard creditCard = CreditCard(
        accountNumber, // account Number
        Calendar(
            // Expiration Date
            randomInteger(1, 12),
            randomInteger(2024, 2030)
        ),
        cardLevel, // cardlevel
        randomDouble(-10.00, 5000)
    );
}

```

```

        // return credit card
        return creditCard;
    }

CardLevel RandomGenerator::generateCardLevel(){
    // Random generator generate card level method. Generate
    // a random card level
    int level = randomInteger(1, 3);
    if (level == 1){
        // Return platnum if one
        return CardLevel::PLATINUM;
    }
    if (level == 2){
        // Return gold if 2
        return CardLevel::GOLD;
    }
    // Return basic
    return CardLevel::BASIC;
}

/* Unit Test Random Generator Class*/
static void testRandom(){
    RandomGenerator testRandomGenerator = RandomGenerator();
    double testRandomDouble = testRandomGenerator.randomDouble(
        // test random double
        5.0, // min
        10.0 // max
    );
    int testRandomInteger = testRandomGenerator.randomInteger(
        // Test Random Integer
        5, // min
        10 // max
    );
    // Assert
    ASSERT(testRandomDouble > 5.0 && testRandomDouble < 10.0);
    ASSERT(testRandomInteger > 5 && testRandomInteger < 10);
}

static void testGenerateAccount(){
    // Unit test for test generate account
    Depositor testDepositor = Depositor(
        // Initialize test depositor
        Name(
            "Max", // first name
            "Mustermann" // last name
        ),
        "123456789" // social security number
    );
    // Initialize random generator
    RandomGenerator testRandomGenerator;
    unique_ptr<BankAccount> testRandomAccount =
        testRandomGenerator.generateAccount(
            // generate test account
            testDepositor
        );
    // ASSERT
    ASSERT(testRandomAccount->getDepositor())

```



```

        .getDepositorName()
        .printName() == "Max Mustermann");
ASSERT(testRandomAccount->getDepositor().getDepositorSSN()
        == "123456789");
ASSERT(testRandomAccount->getAccountBalance() > 0.0);
ASSERT(testRandomAccount->getAccountNumber() != nullptr);
ASSERT(testRandomAccount->getAccountStatus() == true);
ASSERT(testRandomAccount->getAccountType() != nullptr);
}

static void testGenerateRandomPortfolio(){
    // Unit test for generate random portfolio also tests
    // random investment
    RandomGenerator randomGenerator;
    auto brokerageAccount = make_unique<BrokerageAccount>(
        CommercialAccount()
    );
    // Generate random portfolio
    randomGenerator.generateRandomPortfolio(
        *brokerageAccount,
        randomGenerator.randomInteger(3, 10)
    );
    // Assert
    ASSERT(brokerageAccount->getTotalShares() > 0);
    ASSERT(brokerageAccount->getTotalValue() > 0);
}

void testRandomGenerator(){
    // Unit test for random Generator
    TEST(testRandom);
    TEST(testGenerateAccount);
    TEST(testGenerateRandomPortfolio);
}

```

23.10 Summary

The `RandomGenerator` class provides a comprehensive framework for:

- Simulating random account and investment data
- Testing account behavior under random transactions
- Supporting banking system simulations

By centralizing all random generation, it simplifies testing and reduces duplicated code across accounts and investments.

24 Bank Class

24.1 Overview

The `Bank` class represents a bank system that manages multiple types of accounts, including:

- Checking Accounts
- Savings Accounts
- CD Accounts
- Commercial Accounts

- Brokerage Accounts
- Credit Accounts

It is capable of performing the following operations:

- Deposits and Withdrawals
- Transfers
- Brokerage transactions
- Credit transactions
- Retrieving account information and transaction history
- Maintaining statistics of all accounts

The class uses a vector of `std::unique_ptr<BankAccount>` to store all accounts, enabling polymorphic behavior.

24.2 Member Variables

These are the static data members used to aggregate the bank statistics.

- `std::vector<std::unique_ptr<BankAccount>> accountDatabase`
Stores all accounts in the bank.
- `static int totalNumberOfAccounts`
Tracks total number of accounts.
- `static double valueOfAllAccounts`
Tracks total value of all accounts.
- `static int totalNumberOfCheckingAccounts`
Tracks number of checking accounts.
- `static double valueOfAllCheckingAccounts`
Tracks total value of checking accounts.
- `static int totalNumberOfSavingsAccounts`
Tracks number of savings accounts.
- `static double valueOfAllSavingsAccounts`
Tracks total value of savings accounts.
- `static int totalNumberOfCDAccounts`
Tracks number of CD accounts.
- `static double valueOfAllCDAccounts`
Tracks total value of CD accounts.
- `static int totalNumberOfCommercialAccounts`
Tracks number of commercial accounts.
- `static double valueOfAllCommercialAccounts`
Tracks total value of commercial accounts.
- `static int totalNumberOfBrokerageAccounts`
Tracks number of brokerage accounts.
- `static double valueOfAllBrokerageAccounts`
Tracks total value of brokerage accounts.
- `static int totalNumberOfCreditAccounts`
Tracks number of credit accounts.
- `static double valueOfAllCreditAccounts`
Tracks total credit balances.

24.3 Constructors

- `Bank(int numberOfTransactions)`
Initializes the bank by:
 1. Performing unit tests on all account-related classes.
 2. Loading accounts from `bankAccounts.txt`.
 3. Performing a sequence of randomly generated transactions.
 4. Writing all transactions and account information to an HTML file.

24.4 Public Member Functions

- `TransactionReceipt bankAccountTransaction(BankAccount* randomAccount)`
Performs a generic bank account transaction (deposit, withdrawal, open/close, balance inquiry).
- `TransactionReceipt checkingAccountTransaction(BankAccount* checkingAccount)`
Performs a checking account transaction including clearing checks and regular bank transactions.
- `TransactionReceipt commercialAccountTransaction(BankAccount* commercialAccount)`
Performs transactions for commercial accounts, including:
 - Checking account transactions
 - Bank transactions
 - Credit account transactions
- `TransactionReceipt brokerageAccountTransaction(BankAccount* brokerageAccount)`
Performs brokerage account transactions, including printing portfolio or commercial/credit transactions.
- `void loadAccounts(const std::string& fileName)`
Loads accounts from a text file, generating account objects with a random account type and initial balance.
- `void writeDataBase(std::ofstream& out)`
Writes the bank database and account statistics to an HTML file.
- `TransactionReceipt transactionHistory(BankAccount* bankAccount)`
Returns a transaction receipt containing the full history of the specified account.
- `TransactionReceipt clientInformation(BankAccount* bankAccount)`
Returns a transaction receipt containing information on all accounts owned by a particular client.
- `std::string printFormattedBalance(double balance)`
Returns a formatted string of the account balance with commas and two decimal places.

24.5 Private Member Functions

- `BankAccount* findAccountByNumber(std::string socialSecurityNumber)`
Searches for an account by the depositor's SSN and returns a pointer to the account.

24.6 Transaction Handling and Exceptions

The `Bank` class handles various transaction types with exception handling for:

- `InvalidAmountException`
- `InsufficientFundsException`
- `CDMaturityDateException`
- `InvalidAccountException`
- `AccountClosedException`

- AccountOpenException
- CheckTooOldException
- PostDatedCheckException
- SpendingLimitException
- CardExpiredException

These ensure safe transactions and accurate account statistics.

24.7 HTML Output

All account data, transactions, and statistics are output in a structured HTML format, including:

- Account table with columns for Name, SSN, Account Number, Type, Status, Balance, and Maturity Date.
- Bank statistics: total accounts and balances by account type.
- Transaction receipts for each simulated transaction.

24.8 Bank Implementation

24.8.1 Bank.hpp

The bank header file should look like this:

```
#pragma once

#include "Stdafx.hpp"
#include "BankAccount.hpp"

/*
Bank Class
Reads accounts from a txt file, contains an array of bank account,
commercial accounts and brokerage accounts. Can perform withdrawals,
deposits, transfers, brokerage transactions
*/
class Bank{
public:
    // Constructor Series
    Bank(int numberOfTransactions);

    /* Bank Class Transaction Series */
    TransactionReceipt bankAccountTransaction(BankAccount* randomAccount);
    TransactionReceipt checkingAccountTransaction(BankAccount* checkingAccount);
    TransactionReceipt commercialAccountTransaction(BankAccount* commercialAccount);
    TransactionReceipt brokerageAccountTransaction(BankAccount* brokerageAccount);

    void loadAccounts(const std::string& fileName); // load accounts
    void writeDataBase(std::ofstream& out); // write account

    TransactionReceipt transactionHistory(BankAccount* bankAccount);
    TransactionReceipt clientInformation(BankAccount* BankAccount);

    // Fommatter
    std::string printFormattedBalance(double balance);
```

```

private:
    std::vector<std::unique_ptr<BankAccount>> accountDatabase; // data base of accounts
    // Static Variables
    static int totalNumberOfAccounts; // total number of accounts
    static double valueOfAllAccounts; // value of all accounts
    static int totalNumberOfCheckingAccounts; // total number of checking accounts
    static double valueOfAllCheckingAccounts; // value of all checking accounts
    static int totalNumberOfSavingsAccounts; // number of savings accounts
    static double valueOfAllSavingsAccounts; // value of all saving accounts
    static int totalNumberOfCDAccounts; // number of cd accounts
    static double valueOfAllCDAccounts; // value of all cd account
    static int totalNumberOfCommercialAccounts; // number of commerical accounts
    static double valueOfAllCommercialAccounts; // value of all commercial accounts
    static int totalNumberOfBrokerageAccounts; // number of brokerage accounts
    static double valueOfAllBrokerageAccounts; // value of all brokerage accounts
    static int totalNumberOfCreditAccounts; // total number of credit accounts
    static double valueOfAllCreditAccounts; // value of all credit accounts

    BankAccount* findAccountByNumber(std::string socialSecurityNumber);
};

```

24.8.2 Bank.cpp

The entire Bank class source file should look like this:

```

#include "Bank.hpp"
#include "RandomGenerator.hpp"
#include "Name.hpp"
#include "Depositor.hpp"
#include "BankAccount.hpp"
#include "TransactionReceipt.hpp"
#include "Check.hpp"
#include "SavingsAccount.hpp"
#include "CheckingAccount.hpp"
#include "CDAccount.hpp"
#include "CreditAccount.hpp"
#include "CommercialAccount.hpp"
#include "BrokerageAccount.hpp"

using namespace std;

void unitTest(){
    testName(); // test Name
    testDepositor(); // test depositor
    testTransactionReceipt(); // test transaction receipt
    testTransactionTicket(); // test Transaction Ticket
    testBankAccount(); // test bank Account
    testCalendar(); // test calendar
    testCheck(); // test Check
    testSavingsAccount(); // Test Savings aAccount
    testCheckingAccount(); // test checking Account
    testCDAccount(); // test CD Account
    testCreditAccount(); // test Credit Account
    testCommercialAccount(); // test Commercial Account
    testStock(); // test stock
    testBond(); // test bon
    testOption(); // test option
    testBrokerageAccount(); // test brokerage account
    testRandomGenerator(); // test random generator
}

```

```

}

// Intialize static variables
int Bank::totalNumberOfAccounts = 0;
int Bank::totalNumberOfSavingsAccounts = 0;
int Bank::totalNumberOfBrokerageAccounts = 0;
int Bank::totalNumberOfCDAccounts = 0;
int Bank::totalNumberOfCheckingAccounts = 0;
int Bank::totalNumberOfCommercialAccounts = 0;
int Bank::totalNumberOfCreditAccounts = 0;
double Bank::valueOfAllAccounts = 0.0;
double Bank::valueOfAllBrokerageAccounts = 0.0;
double Bank::valueOfAllCDAccounts = 0.0;
double Bank::valueOfAllCommercialAccounts = 0.0;
double Bank::valueOfAllCheckingAccounts = 0.0;
double Bank::valueOfAllSavingsAccounts = 0.0;
double Bank::valueOfAllCreditAccounts = 0.0;

/* Vank Constructor Series */
Bank::Bank(int numberOfTransactions){
    // Perform unit test
    unitTest();
    // Initialize output file name
    string outputFile = "BankApplicationOutput.html";
    // Load Accounts
    loadAccounts("bankAccounts.txt");

    // Re open file to append
    ofstream programOutput(outputFile);
    if (!programOutput){
        throw runtime_error("Could not open output file");
    }

    // Write HTML Header
    programOutput << "<!DOCTYPE html>\n"
        << "<html>\n"
        << "<head>\n"
        << "<title>Bank Database</title>\n"
        << "<style>\n"
        << "body { font-family: monospace; }\n"
        << "pre { margin-bottom: 30px; }\n"
        << "</style>\n"
        << "</head>\n"
        << "<body>\n";
    programOutput << "<b>Number Of Transactions: </b>"
        << numberOfTransactions << "\n\n";

    // Print database
    writeDataBase(programOutput);

    // Random Generator
    RandomGenerator randomGenerator;
    for (int i = 0; i < numberOfTransactions; i++){
        size_t index = randomGenerator.randomInteger(
            0,
            accountDatabase.size() - 1
        );
        // Get random account

```

```

BankAccount* randomAccount = accountDatabase[index].get();
// Initialize transaction receipt
TransactionReceipt transactionReceipt;

TransactionTicket transactionTicket =
    randomGenerator.generateBankAccountTransaction(
        randomAccount->getAccountNumber()
    );

if (transactionTicket.getType() == TransactionType::Balance){
    transactionReceipt = randomAccount->getBalance(transactionTicket);
} else {
    switch (randomAccount->getType()){
        case AccountType::Commercial:
            transactionReceipt = commercialAccountTransaction(
                randomAccount
            );
            break;
        case AccountType::Brokerage:
            transactionReceipt = brokerageAccountTransaction(
                randomAccount
            );
            break;
        case AccountType::Checking:
            transactionReceipt = checkingAccountTransaction(
                randomAccount
            );
            break;
        default:
            transactionReceipt = bankAccountTransaction(
                randomAccount
            );
    }
}

// Adjust bank statistics
if (transactionReceipt.getTransactionSuccess()){
    // Copy ticket and account type
    TransactionTicket ticket = transactionReceipt.getTransactionTicket();
    string accountType = transactionReceipt.getAccountType();
    // Adjust individual account Balances
    switch(transactionReceipt.getTransactionTicket().getType()){
        case TransactionType::Withdrawl:
        case TransactionType::ClearCheck:
            // Decerement all accounts
            valueOfAllAccounts -= ticket.getTransactionAmount();
            if (accountType == "Savings"){
                // adjust savings
                valueOfAllSavingsAccounts -= ticket.getTransactionAmount();
            } else if (accountType == "Checking"){
                // adjust checking
                valueOfAllCheckingAccounts -= ticket.getTransactionAmount();
            } else if (accountType == "CD"){
                // Adjust CD
                valueOfAllCDAccounts -= ticket.getTransactionAmount();
            } else if (accountType == "Commercial"){
                // Adjust commercial
            }
    }
}

```

```

        valueOfAllCommercialAccounts-=ticket.getTransactionAmount();
    } else if (accountType == "Brokerage"){
        // Adjust Brokerage
        valueOfAllBrokerageAccounts -= ticket.getTransactionAmount();
    }
    break;
case TransactionType::Deposit:
    // Increment all accounts
    valueOfAllAccounts+= ticket.getTransactionAmount();
    if (accountType == "Savings"){
        // Increment savings
        valueOfAllSavingsAccounts+=ticket.getTransactionAmount();
    } else if (accountType == "Checking"){
        // increment checking
        valueOfAllCheckingAccounts+=ticket.getTransactionAmount();
    } else if (accountType == "CD"){
        // incremenegt CD
        valueOfAllCDAccounts += ticket.getTransactionAmount();
    } else if (accountType == "Commercial"){
        // Increment Commercial
        valueOfAllCommercialAccounts+=ticket.getTransactionAmount();
    } else if (accountType == "Brokerage"){
        // increment brokerage
        valueOfAllBrokerageAccounts += ticket.getTransactionAmount();
    }
    break; // stop
case TransactionType::PayBalance:
    // Increment credit balance
    valueOfAllCreditAccounts-=ticket.getTransactionAmount();
    valueOfAllAccounts+=ticket.getTransactionAmount();
    break;
case TransactionType::CreditTransaction:
    // decrement credit balance
    valueOfAllCreditAccounts+=ticket.getTransactionAmount();
    valueOfAllAccounts-=ticket.getTransactionAmount();
    break;
default:
    break;
}

}

// write receipt to the HTML file using print method
programOutput << "<pre>\n"
    << "Transaction Requested: "
    << transactionReceipt.getTransactionTicket().getTransactionType()
    << "\n"
    << randomAccount->printAccountReceipt() << "\n"
    << transactionReceipt.printTransactionReceipt()
    << "\n</pre>\n";

}
// Print the data base
writeDataBase(programOutput);

programOutput << "\n</body>\n</html>\n";
programOutput.close();
}

```



```

TransactionReceipt Bank::bankAccountTransaction(BankAccount* randomAccount){
    // Bank class bank account transaction method. Performs a regular transaction
    RandomGenerator randomGenerator; // initialize random generator
    TransactionReceipt transactionReceipt;
    TransactionTicket transactionTicket = randomGenerator
        // Generate Transaction Ticket
        .generateBankAccountTransaction(
            randomAccount->getAccountNumber()
        );

    // Perform transaction
    switch(transactionTicket.getType()){
        case TransactionType::Balance:
            // Get balance
            return transactionReceipt = randomAccount -> getBalance(
                transactionTicket
            );
        case TransactionType::Withdrawl:
            try {
                return transactionReceipt = randomAccount->makeWithdrawl(
                    // Make Withdrawl
                    transactionTicket
                );
            } catch (const InvalidAmountException& e){
                return transactionReceipt = TransactionReceipt(
                    transactionTicket, // transaction ticket
                    randomAccount->getAccountType(), // account type
                    randomAccount->getAccountBalance(), // account balance
                    randomAccount->getAccountBalance(), // post balance
                    false, // fasle
                    e.what() // error message
                );
            } catch (const InsufficientFundsException& e) {
                return transactionReceipt = TransactionReceipt(
                    transactionTicket, // transaction ticket
                    randomAccount->getAccountType(), // account type
                    randomAccount->getAccountBalance(), // pre balance
                    randomAccount->getAccountBalance(), // post balance
                    false, // false
                    e.what() // error message
                );
            } catch (const CDMaturityDateException& e) {
                return transactionReceipt = TransactionReceipt(
                    // maturitty date exception
                    transactionTicket, // transactionTicket
                    randomAccount->getAccountType(), // account type
                    randomAccount->getAccountBalance(), // pre balance
                    randomAccount->getAccountBalance(), // post balance
                    false, // failure
                    e.what() // error message
                );
            }
        case TransactionType::Deposit:
            try {
                return transactionReceipt = randomAccount->makeDeposit(
                    // Make deposit
                    transactionTicket
                );
            }
    }
}

```

```

    );
} catch (const InvalidAmountException& e) {
    return transactionReceipt = TransactionReceipt(
        transactionTicket, // transaction Ticket
        randomAccount->getAccountType(), // account type
        randomAccount->getAccountBalance(), // pre balance
        randomAccount->getAccountBalance(), // post balance
        false, // false
        e.what() // error message
    );
} catch (const InvalidAccountException& e) {
    return transactionReceipt = TransactionReceipt(
        transactionTicket, // transaction ticket
        randomAccount->getAccountType(), // random account type
        randomAccount->getAccountBalance(), // pre balance
        randomAccount->getAccountBalance(), // post balance
        false, // failed
        e.what() // error
    );
}
}
case TransactionType::Close:
    try {
        return transactionReceipt = randomAccount->closeAccount(
            // close account
            transactionTicket
        );
    } catch (const AccountClosedException& e){
        return transactionReceipt = TransactionReceipt(
            transactionTicket, // transaction tikcet
            randomAccount->getAccountType(), // account type
            randomAccount->getAccountBalance(), // pre balance
            randomAccount->getAccountBalance(), // post balance
            false, // failure
            e.what() // error message
        );
    }
    break;
case TransactionType::Open:
    try {
        return transactionReceipt = randomAccount -> reOpenAccount(
            // open account
            transactionTicket
        );
    } catch (const AccountOpenException& e){
        return transactionReceipt = TransactionReceipt(
            transactionTicket, // tranastion ticket
            randomAccount->getAccountType(), // account type
            randomAccount->getAccountBalance(), // pre balance
            randomAccount->getAccountBalance(), // post balance
            false, // failed
            e.what() // error message
        );
    }
    break;
case TransactionType::TransactionHistory:
    // return transaction history
    return transactionHistory(randomAccount);
case TransactionType::ClientInfo:

```

```

        // return client info
        return clientInformation(randomAccount);
    default:
        return TransactionReceipt(); // return null receipt
    }
}

TransactionReceipt Bank::checkingAccountTransaction(BankAccount* checkingAccount){
    // Bank Account Checking Account transaction either runs a regular transaction
    // clear check
    CheckingAccount* randomCheckingAccount = // dynamic cast
        dynamic_cast<CheckingAccount*>(checkingAccount);

    // Perform checks
    if(!randomCheckingAccount ||
        randomCheckingAccount->getType() != AccountType::Checking){
        return TransactionReceipt(); // return null receipt
    }

    // Initialize random generator
    RandomGenerator randomGenerator;
    int transaction = randomGenerator.randomInteger(1, 2);
    Check checkToClear = randomGenerator.generateCheckingTransaction(
        // generate check to clear
        randomCheckingAccount->getAccountNumber()
    );

    if (transaction == 1){
        try{
            // return clear check transaction receipt
            return randomCheckingAccount->clearCheck(
                checkToClear // check to clear
            );
        } catch (const CheckTooOldException& e) {
            return TransactionReceipt(
                TransactionTicket(
                    TransactionType::ClearCheck,
                    randomCheckingAccount->getAccountNumber(),
                    checkToClear.getCheckAmount(),
                    Calendar(), // time now
                    0
                ),
                randomCheckingAccount->getAccountType(), // account type
                randomCheckingAccount->getAccountBalance(), // pre balance
                randomCheckingAccount->getAccountBalance(), // post balance
                false, // failed
                e.what() // error
            );
        } catch (const InsufficientFundsException& e){
            return TransactionReceipt(
                // Return in sufficient funds receipt
                TransactionTicket(
                    TransactionType::ClearCheck, // transaction type
                    randomCheckingAccount->getAccountNumber(), // account number
                    checkToClear.getCheckAmount(), // transaction amount
                    Calendar(), // time now
                    0
                ),
            );
        }
    }
}

```

```

        randomCheckingAccount->getAccountType(), // account type
        randomCheckingAccount->getAccountBalance(), // pre balance
        randomCheckingAccount->getAccountBalance(), // post balance
        false, // failed
        e.what() // error
    );
} catch (const PostDatedCheckException& e) {
    return TransactionReceipt(
        // Return post dated check exception
        TransactionTicket(
            TransactionType::ClearCheck, // transaction type
            randomCheckingAccount->getAccountNumber(), // account number
            checkToClear.getCheckAmount(), // check amount
            Calendar(), // time now
            0
        ),
        randomCheckingAccount->getAccountType(), // account type
        randomCheckingAccount->getAccountBalance(), // pre balance
        randomCheckingAccount->getAccountBalance(), // post balance
        false, // failed
        e.what() // error
    );
}
}
// Return regular transaction
return bankAccountTransaction(randomCheckingAccount);
}

TransactionReceipt Bank::commercialAccountTransaction(BankAccount* commercialAccount){
    // Bank Class commercial account Transaction. generates a transaction
    // for a commercial account
    RandomGenerator randomGenerator;
    TransactionReceipt transactionReceipt;
    int type = randomGenerator.randomInteger(1, 3);
    CommercialAccount* randomCommercialAccount =
        dynamic_cast<CommercialAccount*>(commercialAccount);

    if(!randomCommercialAccount){
        return TransactionReceipt(); // return null receipt
    }

    if (type == 1){
        // Perform checking transaction
        return checkingAccountTransaction(
            randomCommercialAccount
        );
    }

    if (type == 2){
        // Perform regular banking transaction
        return bankAccountTransaction(
            randomCommercialAccount
        );
    }
    // Generate Credit transaction
    CreditCard creditCard = randomGenerator
        .generateCreditAccountTransaction(
            randomCommercialAccount->getAccountNumber() // account number

```

```

    );

    try {
        if (randomCommercialAccount->getCreditAccount().getBalanceDue() > 0){
            creditCard = CreditCard(
                randomCommercialAccount->getAccountNumber(),
                Calendar(
                    randomGenerator.randomInteger(1, 12),
                    randomGenerator.randomInteger(
                        2025,
                        2030
                    )
                ),
                randomGenerator.generateCardLevel(),
                randomGenerator.randomDouble(
                    0,
                    randomCommercialAccount->getCreditAccount().getBalanceDue()
                )
            );
            return randomCommercialAccount->getCreditAccount()
                .payBalanceDue(
                    creditCard);
        } else {
            return randomCommercialAccount->getCreditAccount()
                .creditTransaction(creditCard);
        }
    } catch (const InvalidAmountException& e){
        return TransactionReceipt(
            TransactionTicket(
                // return invalid amount receipt
                TransactionType::PayBalance, // transaction type
                randomCommercialAccount->getAccountNumber(), // get account number
                creditCard.getCreditCharge(), // transaction amount
                Calendar(), // time now
                0 // no term
            ),
            randomCommercialAccount->getAccountType(), // account type
            randomCommercialAccount->getAccountBalance(), // prebalance
            randomCommercialAccount->getAccountBalance(), // post balance
            false, // failed
            e.what() // error message
        );
    } catch (const SpendingLimitException& e){
        return TransactionReceipt(
            TransactionTicket(
                // Return spending limit exception
                TransactionType::CreditTransaction, // trans type
                randomCommercialAccount->getAccountNumber(), // account number
                creditCard.getCreditCharge(), // transaction amount
                Calendar(), // now
                0 // no term
            ),
            randomCommercialAccount->getAccountType(),
            randomCommercialAccount->getAccountBalance(), // pre balance
            randomCommercialAccount->getAccountBalance(), // post balance
            false, // failed
            e.what() // error
        );
    }

```

```

    } catch (const CardExpiredException& e){
        return TransactionReceipt(
            // Return card expired exception
            TransactionTicket(
                TransactionType::CreditTransaction, // transaction type
                randomCommercialAccount->getAccountNumber(),
                creditCard.getCreditCharge(), // transaction amount
                Calendar(), // time now
                0
            ),
            randomCommercialAccount->getAccountType(), // account type
            randomCommercialAccount->getAccountBalance(), // pre balance
            randomCommercialAccount->getAccountBalance(), // post balance
            false, // failed
            e.what() // error
        );
    }
}

TransactionReceipt Bank::brokerageAccountTransaction(BankAccount* bankAccount){
    // Bank Brokerage account Transaction generator. Returns a transaction
    // receipt of a brokerage transaction. Could print portfolio
    BrokerageAccount* brokerageAccount = // Dynamic cast
        dynamic_cast<BrokerageAccount*>(bankAccount);

    // Perform checks
    if (!brokerageAccount ||
        brokerageAccount->getType() != AccountType::Brokerage){
        return TransactionReceipt(); // return null receipt
    }

    // Initialize random generator
    RandomGenerator randomGenerator;
    int transaction = randomGenerator.randomInteger(1, 2);

    if (transaction == 1){
        // Print portfolio
        TransactionTicket transactionTicket(
            TransactionType::Portfolio, // transaction type
            brokerageAccount->getAccountNumber(),
            0.0,
            Calendar(), // time now
            0
        );

        return TransactionReceipt(
            transactionTicket, // transactionticket
            brokerageAccount->getAccountType(), // account type
            brokerageAccount->getAccountBalance(), // pre balance
            brokerageAccount->getAccountBalance(), // post balance
            true, // true
            brokerageAccount->printPortfolio() // portfolio
        );
    }

    // else return nominal commercial potential
    return commercialAccountTransaction(brokerageAccount);
}

```

```

BankAccount* Bank::findAccountByNumber(string socialSecurityNumber){
    // Bank Find account by number method. finds an account
    // by the account number
    for (const auto& bankAccount: accountDatabase){
        if (bankAccount->getDepositor().getDepositorSSN()
            == socialSecurityNumber){
            return bankAccount.get(); // return address
        }
    }
    return nullptr; // not found
}

void Bank::loadAccounts(const string& fileName){
    // Bank Load accounts method. Reads the first name,
    // last name and ssn of an account form a file
    // generates an account and loads it into the bank database
    ifstream inFile(fileName); // initialize file
    RandomGenerator RandomGenerator; // initialize random generator
    if (!inFile){
        // Error handling
        throw runtime_error("Could not open input file");
    }

    string firstName, lastName, ssn;

    while (inFile >> firstName >> lastName >> ssn){
        // Initialize depositor
        Depositor depositor(
            Name(
                firstName, // first name
                lastName // last name
            ),
            ssn // social security number
        );
        // Generate account
        auto account = RandomGenerator.generateAccount(
            depositor // depositor
        );

        // Compute bank statistics
        totalNumberOfAccounts++; // increment total
        valueOfAllAccounts += account->getAccountBalance(); // increment total balance

        switch(account->getType()){
            case AccountType::Savings:
                totalNumberOfSavingsAccounts++; // increment total number of savings accounts
                valueOfAllSavingsAccounts += account->getAccountBalance(); // add to savings total
                break;
            case AccountType::Checking:
                totalNumberOfCheckingAccounts++; // increment total number of checking
                valueOfAllCheckingAccounts += account->getAccountBalance();
                break;
            case AccountType::CD:
                // increment CD total
                totalNumberOfCDAccounts++; // total number of CD account
                // Increment total balance
                valueOfAllCDAccounts += account->getAccountBalance();
                break;
        }
    }
}

```

```

        case AccountType::Brokerage:
            // Increment brokerage total
            totalNumberOfBrokerageAccounts++;
            totalNumberOfCreditAccounts++;
            // Increment brokerage balance
            valueOfAllBrokerageAccounts += account->getAccountBalance();
            break;
        case AccountType::Commercial:
            // increment total number of commercial accounts
            totalNumberOfCommercialAccounts++;
            // commercial account balance
            valueOfAllCommercialAccounts += account->getAccountBalance();
            break;
        case AccountType::Credit:
            break;
    }

    accountDatabase.emplace_back(std::move(account)); // std needed
}

void Bank::writeDataBase(ofstream& out){
    // Bank Class write database. Writes the bank data bas and
    // transactions to a webpage
    // Headline
    out << "<h1>Bank Database</h1>\n";
    out << "<pre>\n";
    out << left
        << setw(27) << "<b>Name</b>"
        << setw(19) << "<b>SSN</b>"
        << setw(20) << "<b>Account#</b>"
        << setw(22) << "<b>Type</b>"
        << setw(16) << "<b>Status</b>"
        << setw(22) << "<b>Balance</b>"
        << setw(15) << "<b>Maturity Date</b>"
        << "\n";
    out << string(100, '*') << "\n";

    // Accounts (polymorphic)
    for (const auto& account : accountDatabase){
        out << account->printAccount(); // account should return a single line string
        out << "\n"; // single newline, no extra spacing
    }

    // padding
    out << string(100, '*') << "\n";
    // Print bank account statistics
    out << "Total Number of Checking Accounts: "
        << totalNumberOfCheckingAccounts << "\n"
        << "Total Number of Savings Accounts: "
        << totalNumberOfSavingsAccounts << "\n"
        << "Total Number of CD Accounts: "
        << totalNumberOfCDAccounts << "\n"
        << "Total Number of Commercial Accounts: "
        << totalNumberOfCommercialAccounts << "\n"
        << "Total Number of Brokerage Accounts: "
        << totalNumberOfBrokerageAccounts << "\n"

```



```

        << "Total Number of Credit Accounts: "
        << totalNumberOfCreditAccounts << "\n\n";
// Print bank account balances
out << right
    << "\t\t\t\t\t\t\tChecking Account Total Balance: "
    << printFormattedBalance(valueOfAllCheckingAccounts) << "\n"
    << "\t\t\t\t\t\t\tSavings Account Total Balance: "
    << printFormattedBalance(valueOfAllSavingsAccounts) << "\n"
    << "\t\t\t\t\t\t\tCD Account Total Balance: "
    << printFormattedBalance(valueOfAllCDAccounts) << "\n"
    << "\t\t\t\t\t\t\tCommercial Account Total Balance: "
    << printFormattedBalance(valueOfAllCommercialAccounts) << "\n"
    << "\t\t\t\t\t\t\tBrokerage Account Total Balance: "
    << printFormattedBalance(valueOfAllBrokerageAccounts) << "\n"
    << "\t\t\t\t\t\t\tCredit Account Total Balance: "
    << printFormattedBalance(valueOfAllCreditAccounts) << "\n";
// Print Bank Totals
out << string(100, '*') << "\n";
out << "\t\t\t\t\t\t\tTotal Number of Accounts: "
    << totalNumberOfAccounts << "\n"
    << "\t\t\t\t\t\t\tTotal Assests Under Management: "
    << printFormattedBalance(valueOfAllAccounts) << "\n"
    << string(100, '*') << "\n";
}

TransactionReceipt Bank::transactionHistory(BankAccount* bankAccount){
    // Bank Calls transaction history. Returns the account receipts
    if (!bankAccount){
        return TransactionReceipt(); // return null receipt
    }

    TransactionTicket transactionTicket(
        TransactionType::TransactionHistory, // transaction type
        bankAccount->getAccountNumber(), // account number
        0.0, // no transaction Amount
        Calendar(), // time now
        0
    );

    // Return transaction receipt
    TransactionReceipt transactionReceipt = TransactionReceipt(
        transactionTicket, // transaction ticket
        bankAccount->getAccountType(), // account Type
        bankAccount->getAccountBalance(), // pre balance
        bankAccount->getAccountBalance(), // post balance
        true, // success
        bankAccount->printTransactionHistory()
    );
    bankAccount->addTransactionReceipt(transactionReceipt);
    return transactionReceipt;
}

TransactionReceipt Bank::clientInformation(BankAccount* bankAccount){
    // Bank class client information. returns the accounts of a
    // particular client
    if (!bankAccount){
        return TransactionReceipt(); // return null receipt
    }
}

```

[illegible]

```

// Bank account print account balance method. Prints a fomatted
// balance
ostringstream bankAccountBalance; // write string stream
bankAccountBalance << "$" << fixed << setprecision(2) << balance;
// convert to string
string formattedBalance = bankAccountBalance.str();

// Insert commas manually
int dot = formattedBalance.find('.');
for (int i = dot - 3; i > 1; i -= 3){
    formattedBalance.insert(i, ",");
}

return formattedBalance; // return balance
}

```

25 Main.cpp

25.1 Overview

The `main.cpp` file serves as the entry point for the banking application. Its primary responsibilities include:

- Initializing the bank system.
- Simulating a sequence of banking transactions.
- Measuring and reporting the total execution time of the program.

25.2 Libraries Used

- `Bank.hpp`
Provides access to the `Bank` class, account management, and transaction handling.
- `<chrono>`
Used for high-resolution timing of program execution.
- `<iostream>`
Provides input/output streams for printing execution time to the console.

25.3 Main Function

- `int main()`
 1. Initializes a high-resolution clock to measure execution time:

```
using clock = std::chrono::high_resolution_clock;
auto start = clock::now();
```
 2. Instantiates a `Bank` object to simulate the banking application with 500 transactions:

```
Bank bankingApplication(500);
```
 3. Stops the clock after completing all transactions:

```
auto end = clock::now();
```
 4. Calculates the elapsed time in seconds:

```
auto elapsed = std::chrono::duration<double>(end - start);
```

5. Prints the total execution time to the console:

```
std::cout << "\nTotal execution time: "
           << elapsed.count()
           << " seconds\n";
```

25.4 Execution Flow

1. The program starts and records the current time.
2. The `Bank` class constructor is called, which performs:
 - Unit tests for all account types.
 - Loading of accounts from `bankAccounts.txt`.
 - Execution of 500 random transactions.
 - Writing of the account database and transaction receipts to an HTML file.
3. After completing all transactions, the program records the end time.
4. The total elapsed execution time is calculated and printed to the console.

25.5 Notes

- The number of transactions (500) can be modified by changing the argument passed to the `Bank` constructor.
- The timing mechanism uses `std::chrono::high_resolution_clock` for precise measurement.
- The program outputs transaction results in HTML format, while performance statistics are printed to the console.

25.6 Main Implementation

This is the main program:

Main.cpp

```
#include "Bank.hpp"
#include <chrono>
#include <iostream>

using namespace std;

int main() {
    // Initialize clock
    using clock = std::chrono::high_resolution_clock;
    auto start = clock::now(); // start timer

    // Banking Application
    Bank bankingApplication(500);

    // Stop timer
    auto end = clock::now();

    // Print time results
    auto elapsed = std::chrono::duration<double>(end - start);
    std::cout << "\nTotal execution time: "
```

```
        << elapsed.count()  
        << " seconds\n";  
    }
```